

Distributed Database

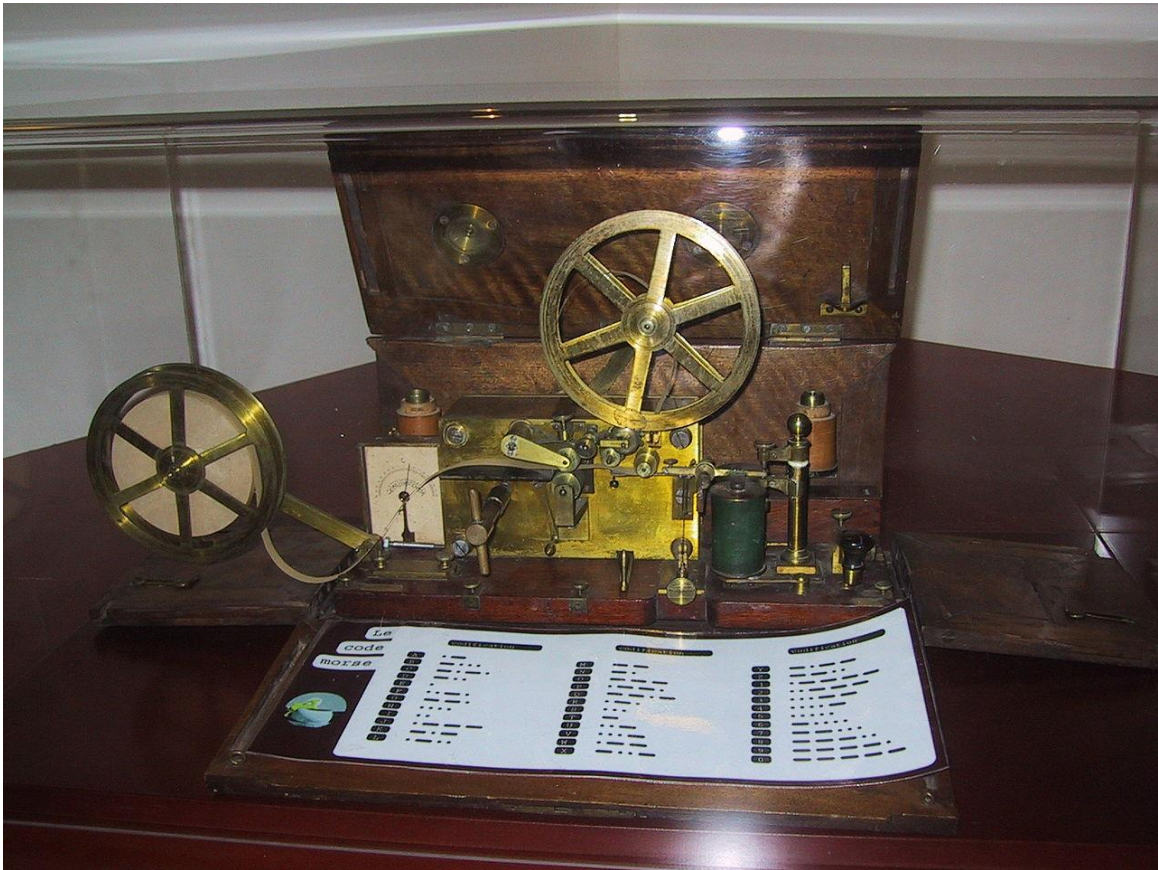
Ibrahim Elsemman

Information Systems

Faculty of Computers and information

Lecture 1: IS Infrastructure

Telegraph



Developed in the 1830s and 1840s by Samuel Morse (1791-1872) and other inventors, the telegraph revolutionized long-distance communication. It worked by transmitting electrical signals over a wire laid between stations.

Fax

- Can send images and papers



Client/server (1983–now)

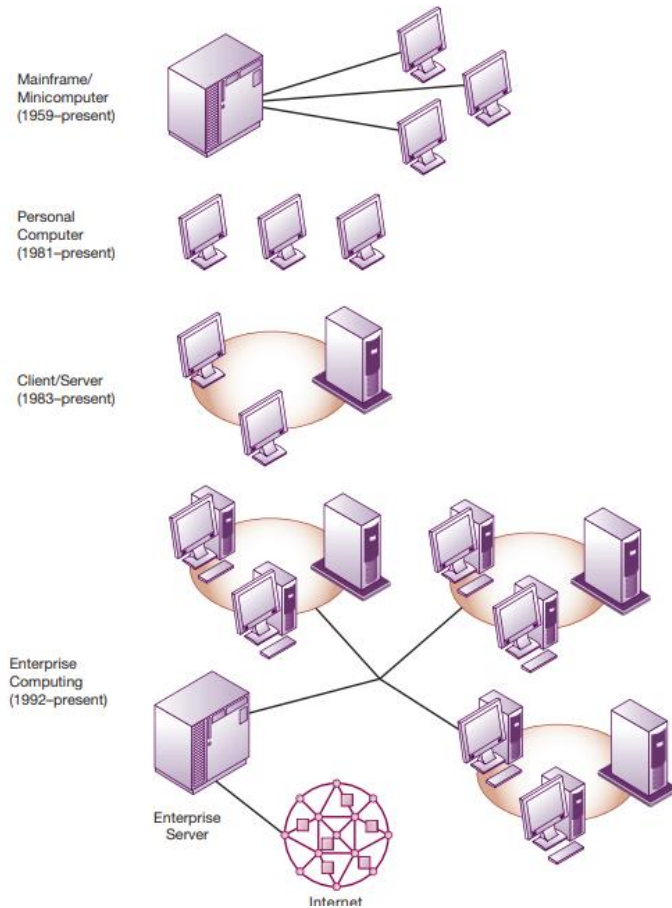
- Client–server systems began to emerge in the United States in the early 1980s as computing transitioned from large mainframes to distributed processing using multiple workstations or personal computers. Corporations quickly adopted client–server systems, which became the backbones of their office automation and communication infrastructure.

Network in any organization

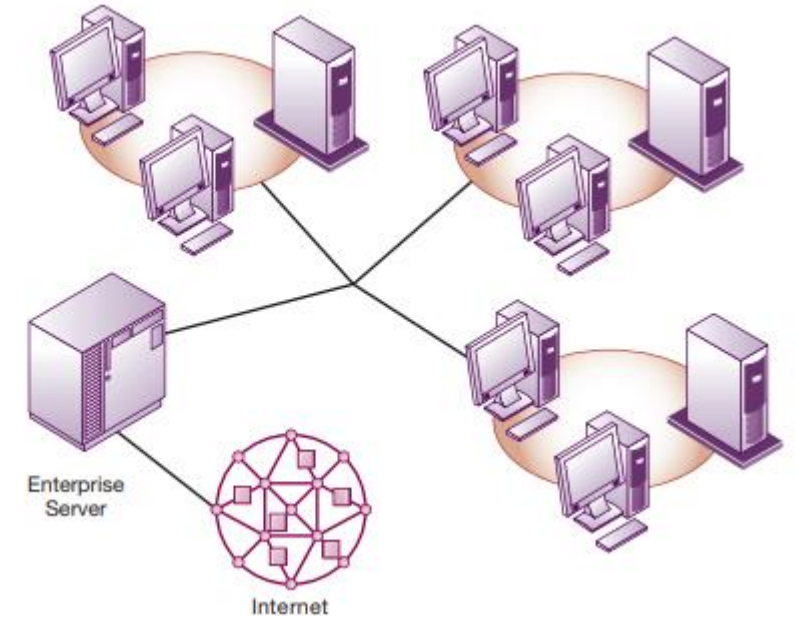
FIGURE 5.2 ERAS IN IT INFRASTRUCTURE EVOLUTION

Illustrated here are the typical computing configurations characterizing each of the five eras of IT infrastructure evolution.

Stages in IT Infrastructure Evolution



Enterprise Computing
(1992–present)



Cloud and Mobile Computing
(2000–present)



The curent network

FIGURE 7.1 COMPONENTS OF A SIMPLE COMPUTER NETWORK

Illustrated here is a simple computer network consisting of computers, a network operating system (NOS) residing on a dedicated server computer, cable (wiring) connecting the devices, switches, and a router.

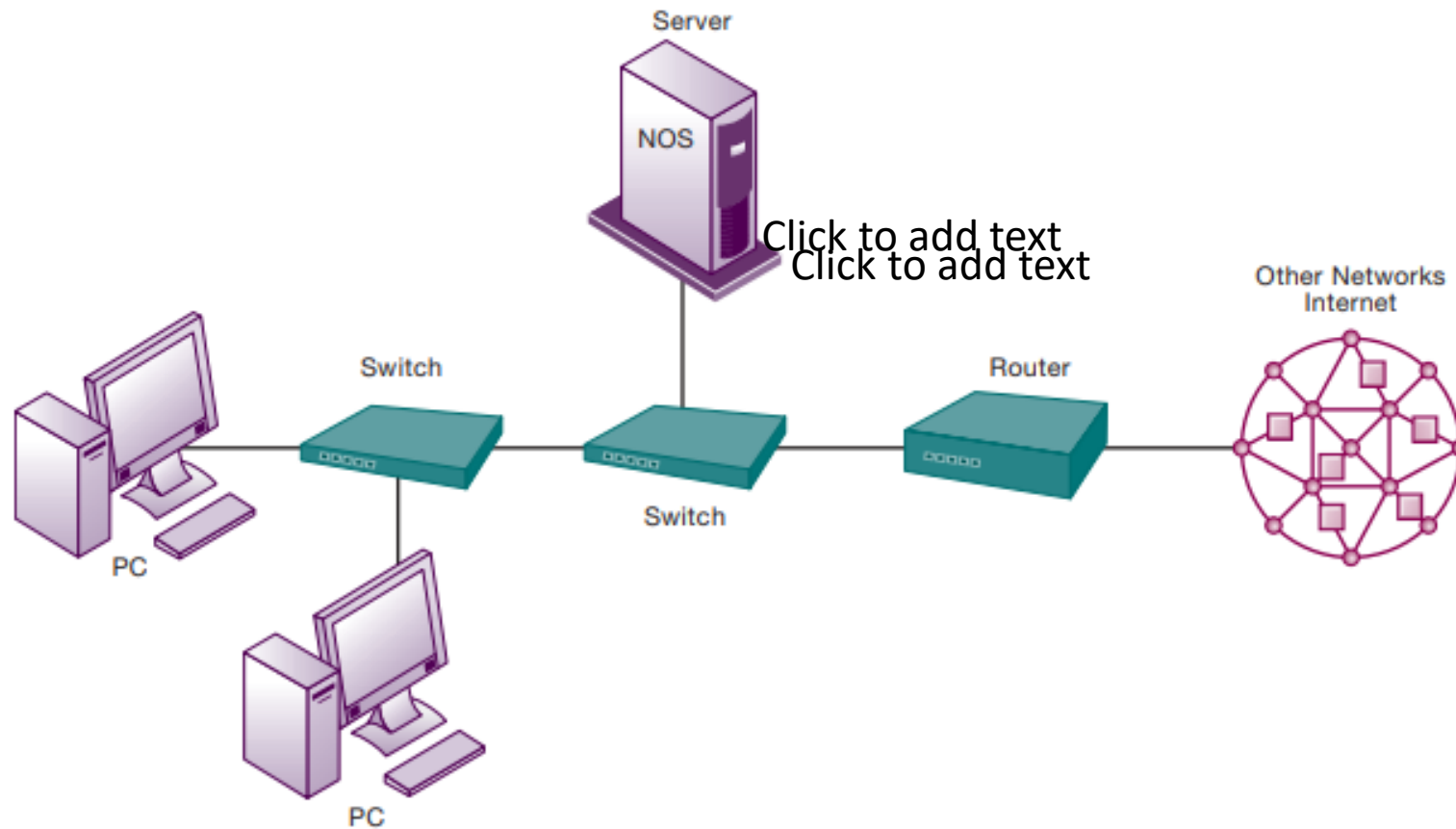
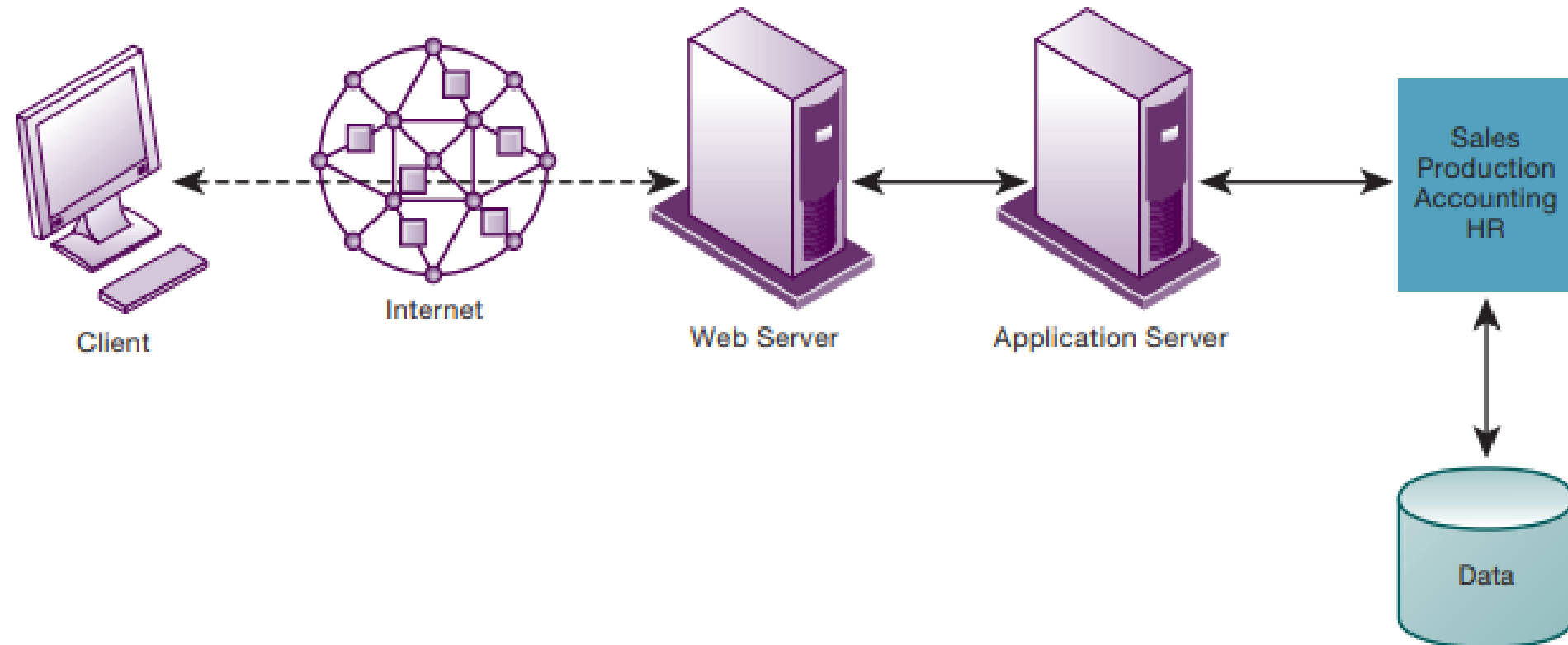
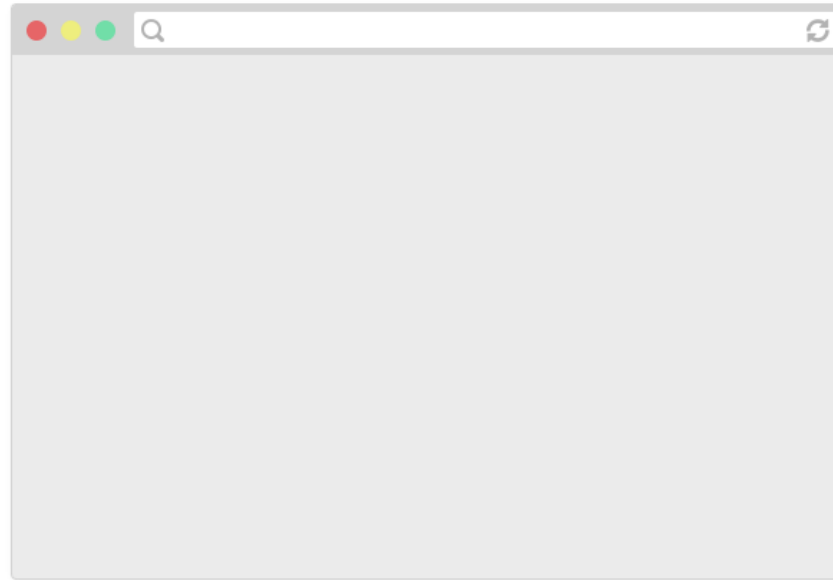


FIGURE 5.3 A MULTITIERED (N-TIER) CLIENT/SERVER NETWORK

In a multitiered client/server network, client requests for service are handled by different levels of servers.

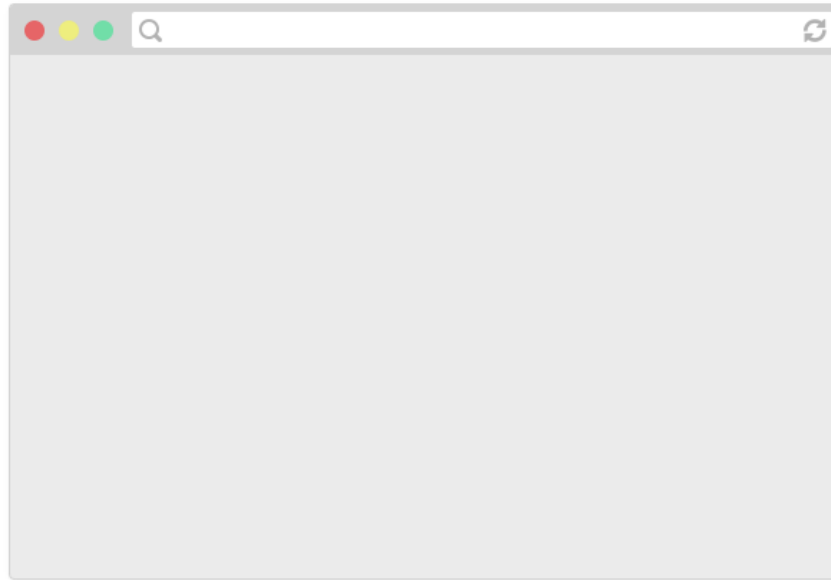


How do web pages work?



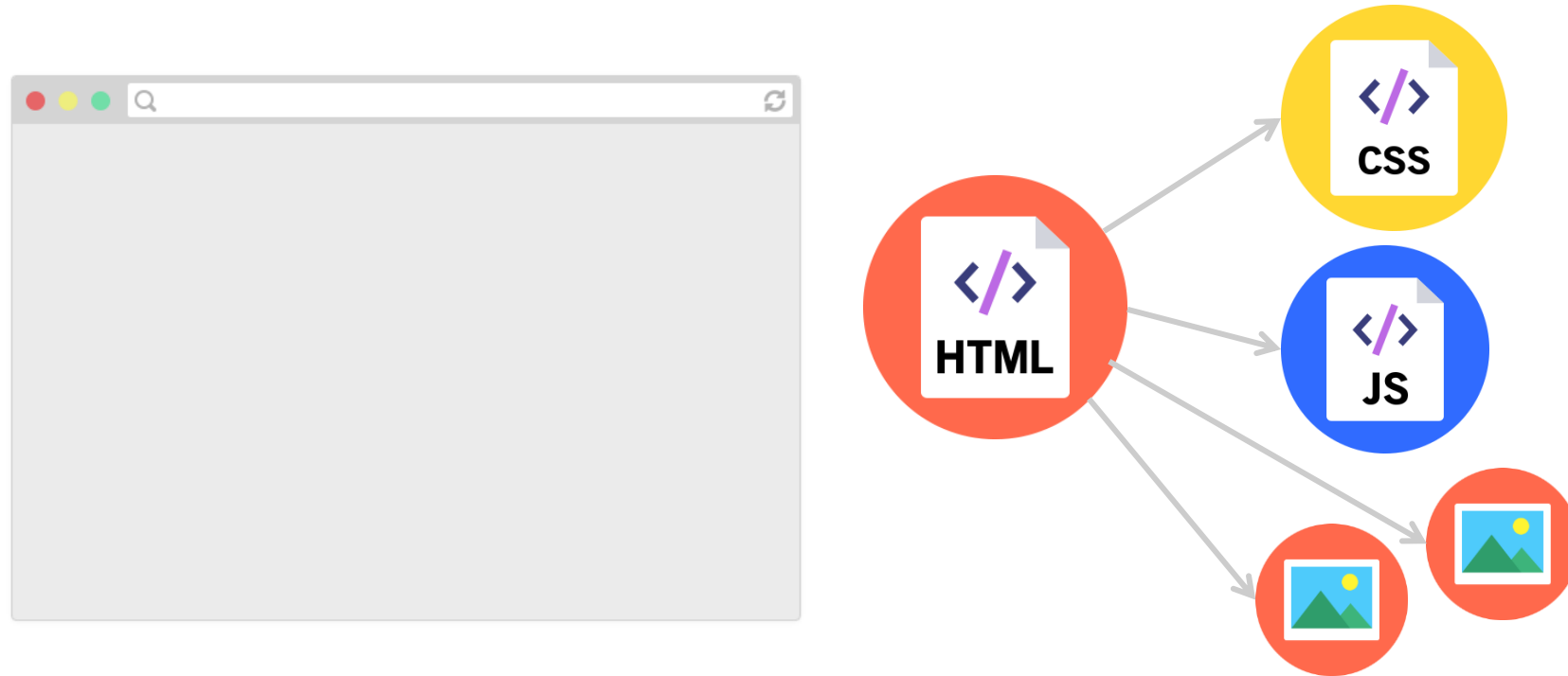
Browsers are applications that can display web pages.
E.g. Chrome, Firefox, Safari, Internet Explorer, Edge, etc.

How do web pages work?



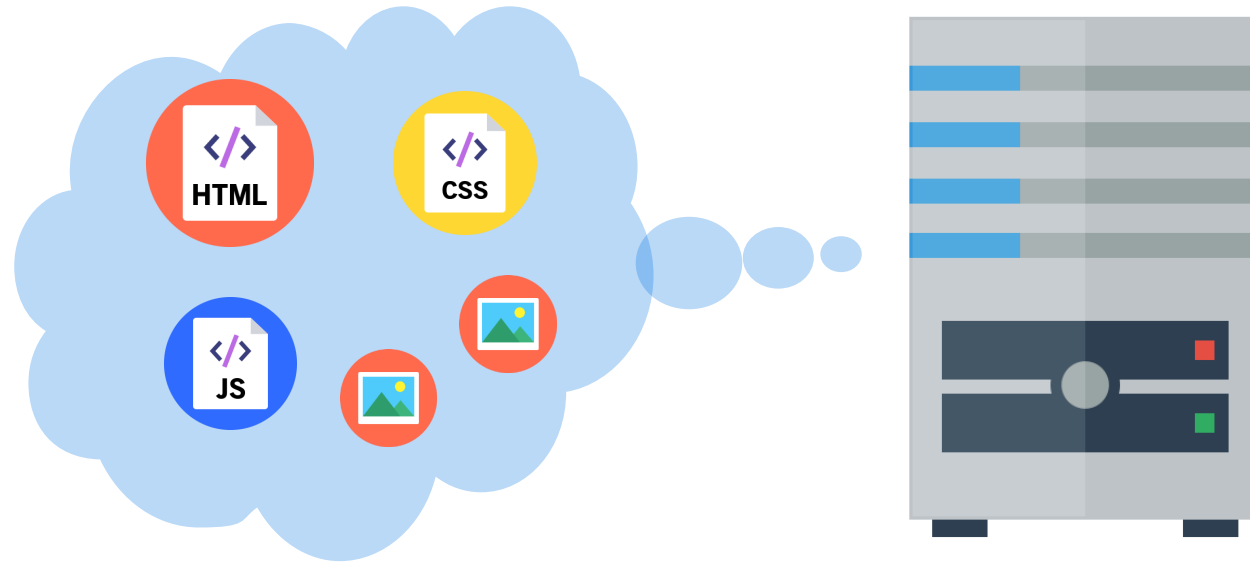
Web pages are written in a markup language called **HTML**, so browsers display a web page by reading and interpreting its HTML.

How do web pages work?



The HTML file might link to other resources, like images, videos, as well as **JavaScript** and **CSS** (stylesheet) files, which the browser then also loads.

How do web pages work?

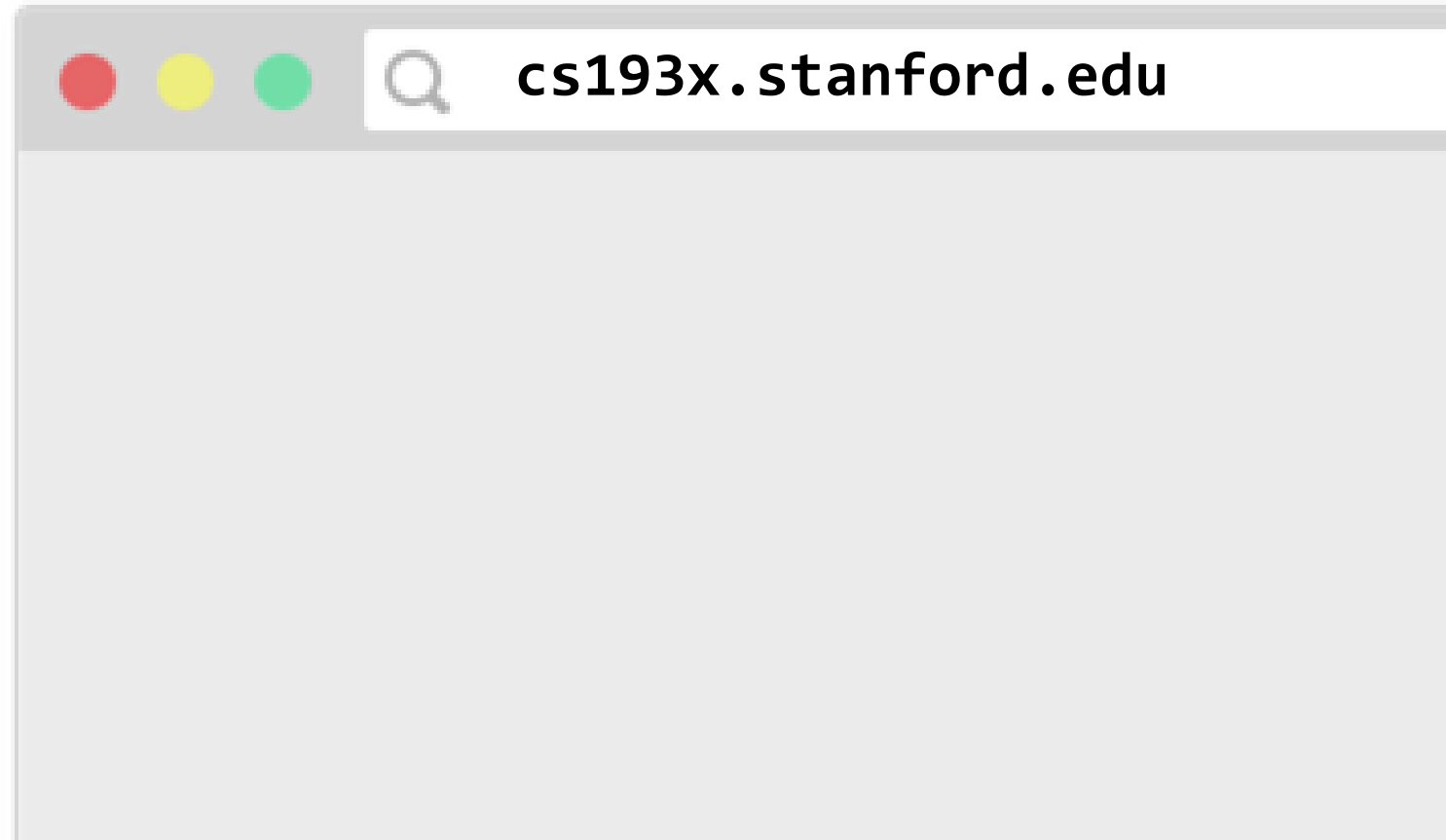


A **web server** is a program running on a computer that delivers web pages in response to requests.

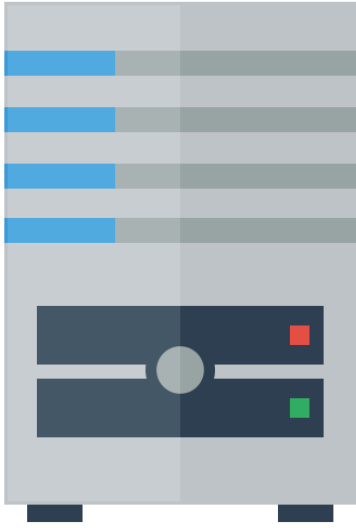
It either stores or generates the web page returned.

How do web pages work?

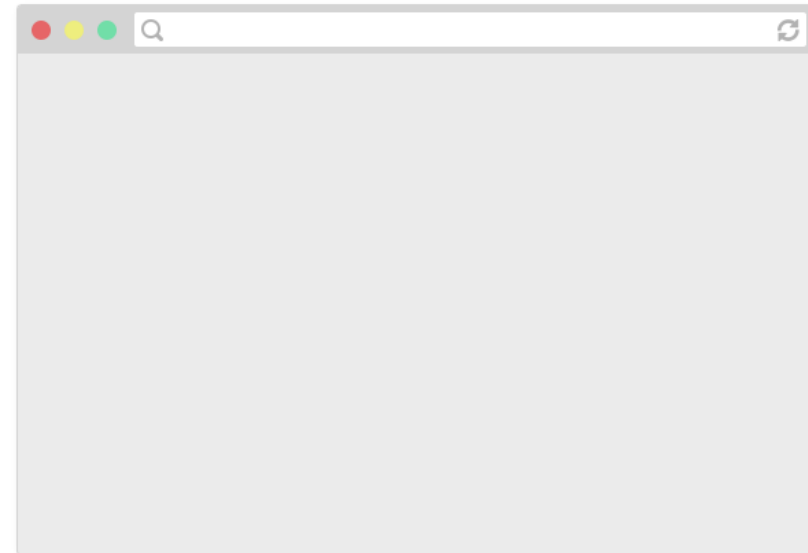
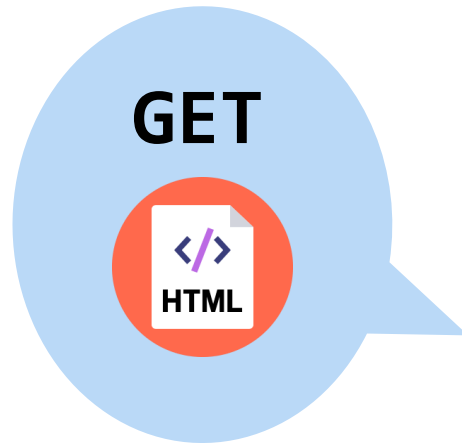
1. You type in a URL, which is the address of the HTML file on the internet.



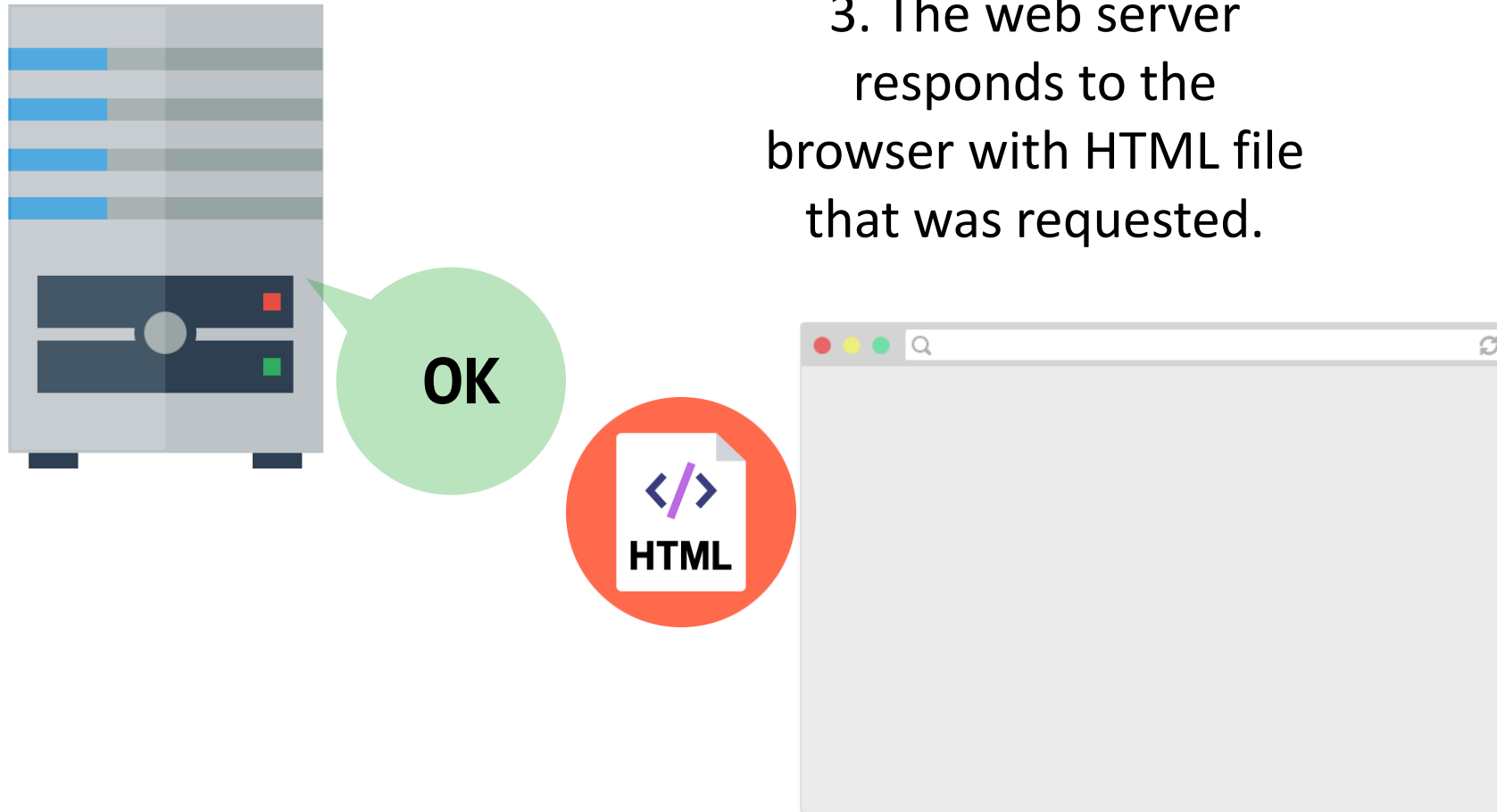
How do web pages work?



2. The browser asks the web server that hosts the document to send that document.

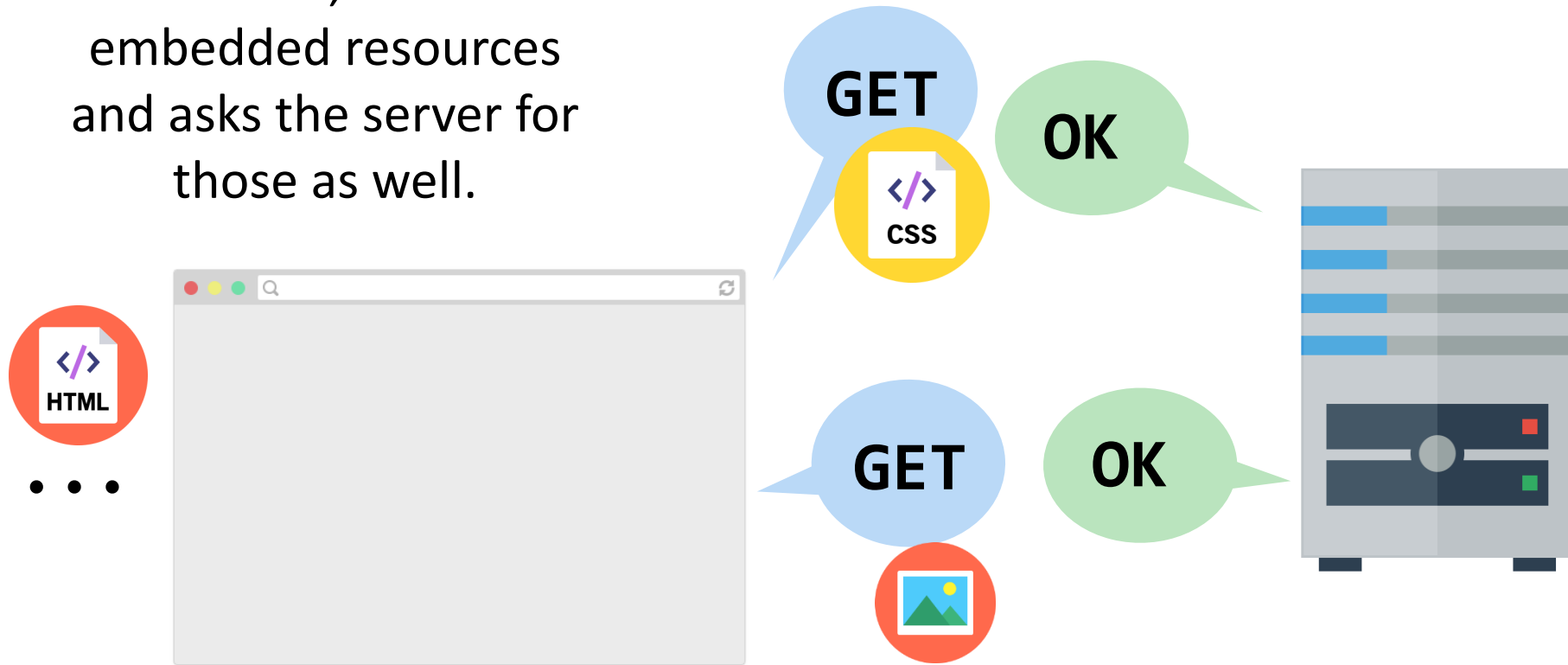


How do web pages work?



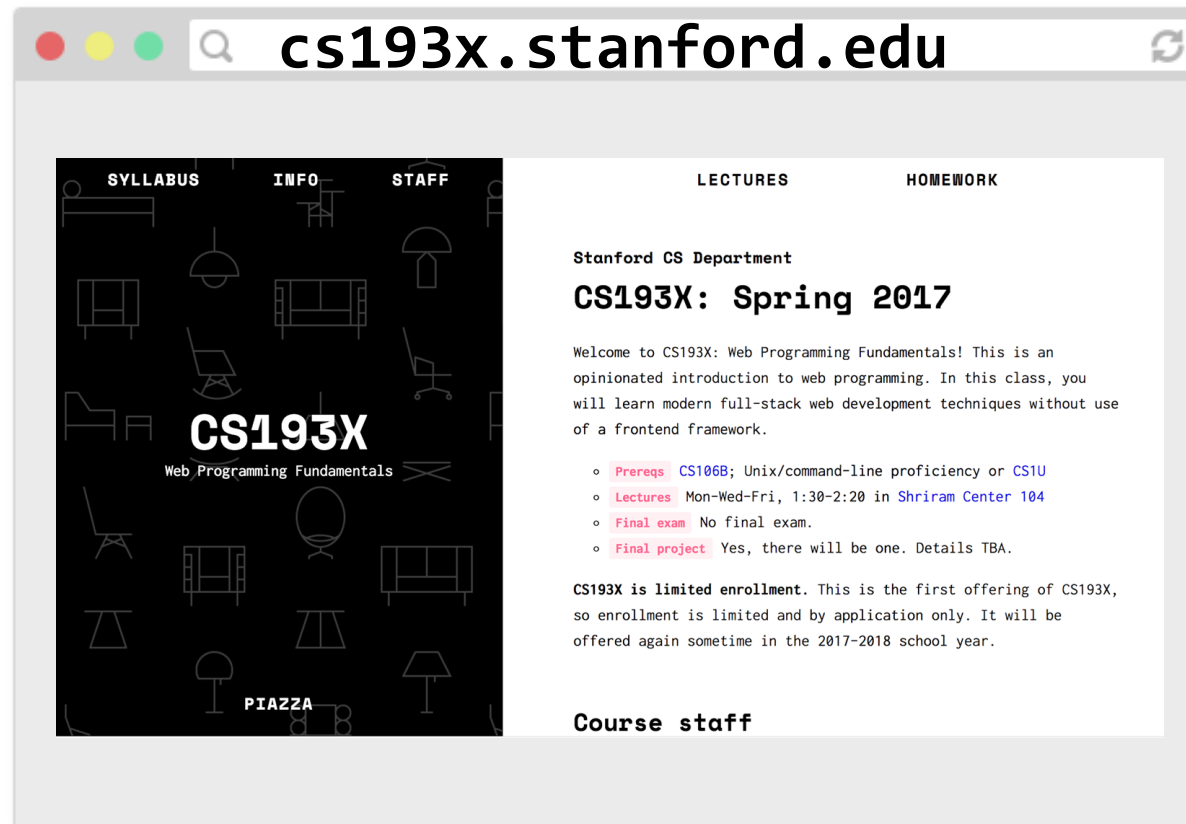
How do web pages work?

4. The browser reads the HTML, sees the embedded resources and asks the server for those as well.

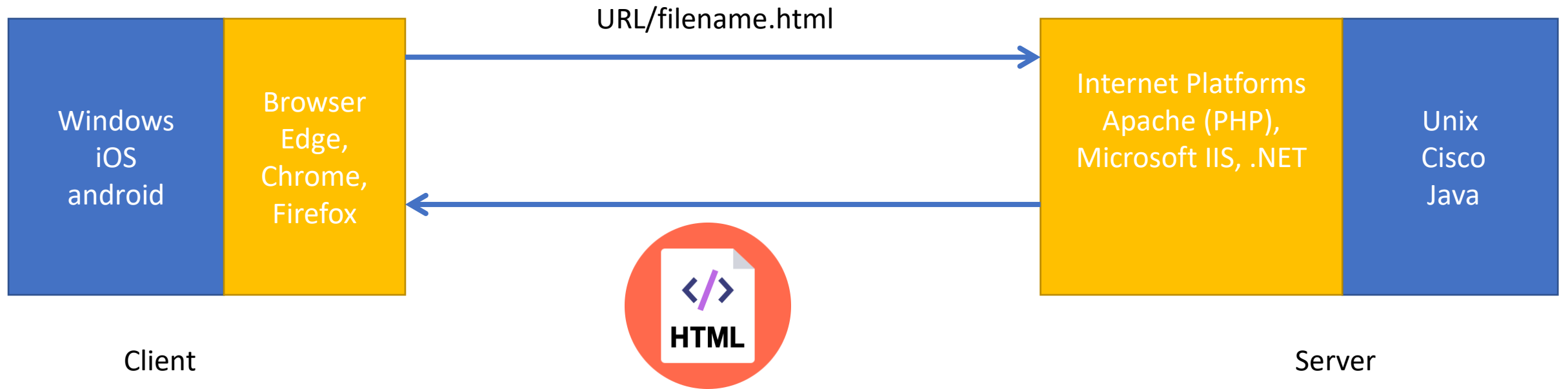


How do web pages work?

5. The web page is loaded when all the resources are fetched and displayed.



What is the main problem in this client/server platform?



HTML is static, no dynamics

Amazon.com 1995 July 16th



Welcome to Amazon.com Books!

*One million titles,
consistently low prices.*

(If you explore just one thing, make it our personal notification service. We think it's very cool!)

SPOTLIGHT! -- AUGUST 16TH

These are the books we love, offered at Amazon.com low prices. The spotlight moves **EVERY** day so please come often.

ONE MILLION TITLES

Search Amazon.com's [million title catalog](#) by author, subject, title, keyword, and more... Or take a look at the [books we recommend](#) in over 20 categories... Check out our [customer reviews](#) and the [award winners](#) from the Hugo and Nebula to the Pulitzer and Nobel... and [bestsellers](#) are 30% off the publishers list...

EYES & EDITORS, A PERSONAL NOTIFICATION SERVICE

Like to know when that book you want comes out in paperback or when your favorite author releases a new title? Eyes, our tireless, automated search agent, will send you mail. Meanwhile, our human editors are busy previewing galleys and reading advance reviews. They can let you know when especially wonderful works are published in particular genres or subject areas. Come in, [meet Eyes](#), and have it all explained.

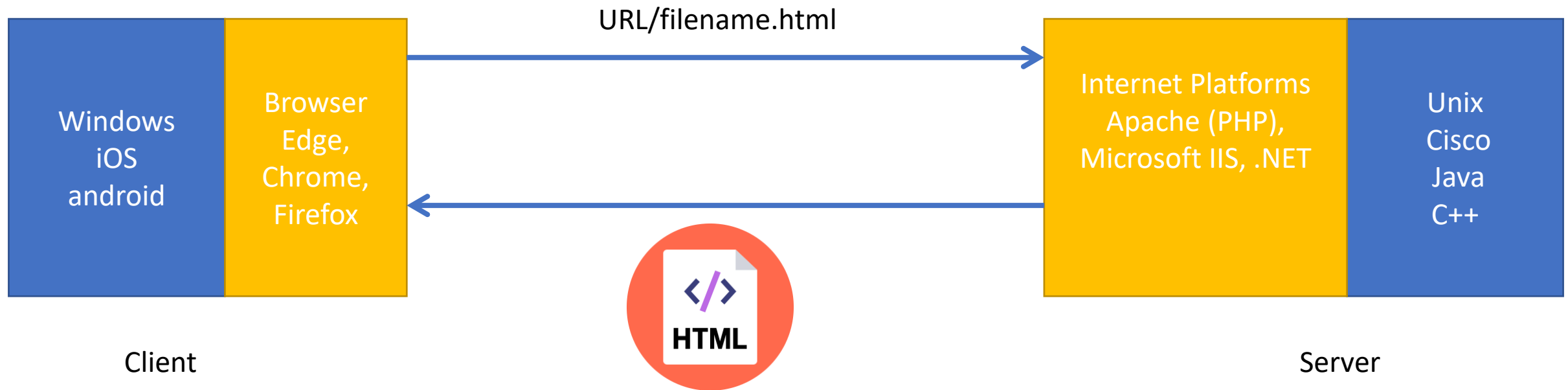
YOUR ACCOUNT

Check the status of your orders or change the email address and password you have on file with us. Please note that you **do not** need an account to use the store. The first time you place an order, you will be given the opportunity to create an account.

Link and information

Common-gateway interface

The client can run c++/java program in the client. This program generates outputs as HTML format



HTML is static, no dynamics

The main problem, is we need professional programmers

Active-pages

- PHP, ASP, .NET, JSP

Server side

```
The server's current time:  
<%  
Response.Write Now()  
%>
```

Client Side

```
The server's current time:  
8/11/2015 6:24:45 PM
```

insert_data.php × ls_one.php

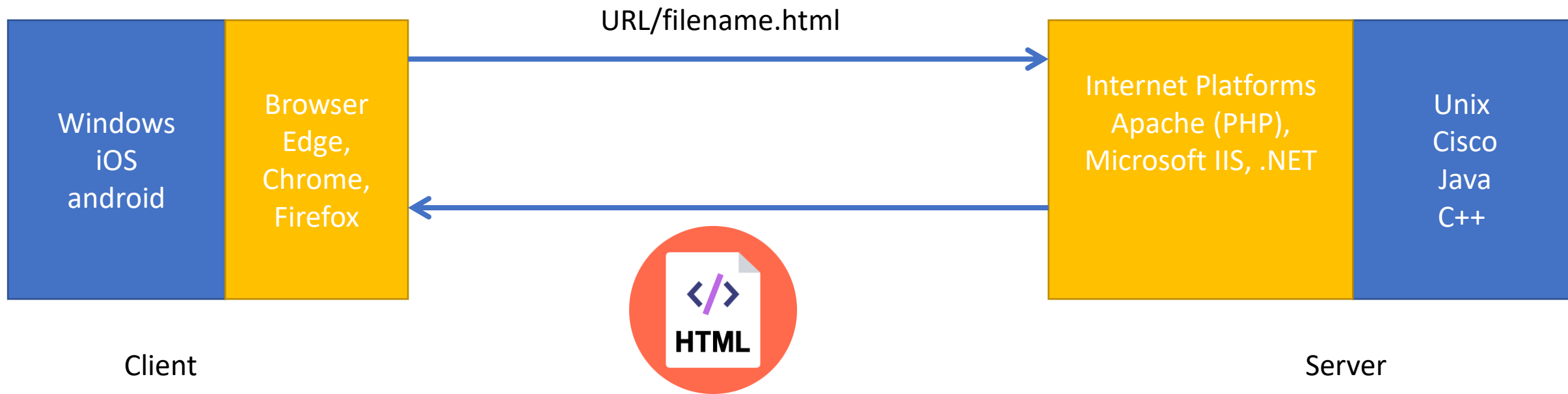
sql_basics > insert_data.php

```
1  <?php  
2  
3  //Create Connection  
4  //mysqli_connect('HOSTNAME','DATABASE USER','DATABASE PASSWORD','DATABASE NAME');  
5  $conn = mysqli_connect('localhost','root','','myfirstdb');  
6  
7  //CHECK CONNECTION  
8  if (!$conn) {  
9      die("Connection failed: ".mysqli_connect_error());  
10 }  
11  
12 $sql = "INSERT INTO users (email, password) VALUES ('c@c.com', 'hello')";  
13 $conn->query($sql);  
14  
15 echo "New User Added";  
16 ?>
```

Easy to connect to Data Management and Storage
IBM DB2 Oracle SQL Server
Sybase
MySQL
Apache Hadoop

Progress in the client

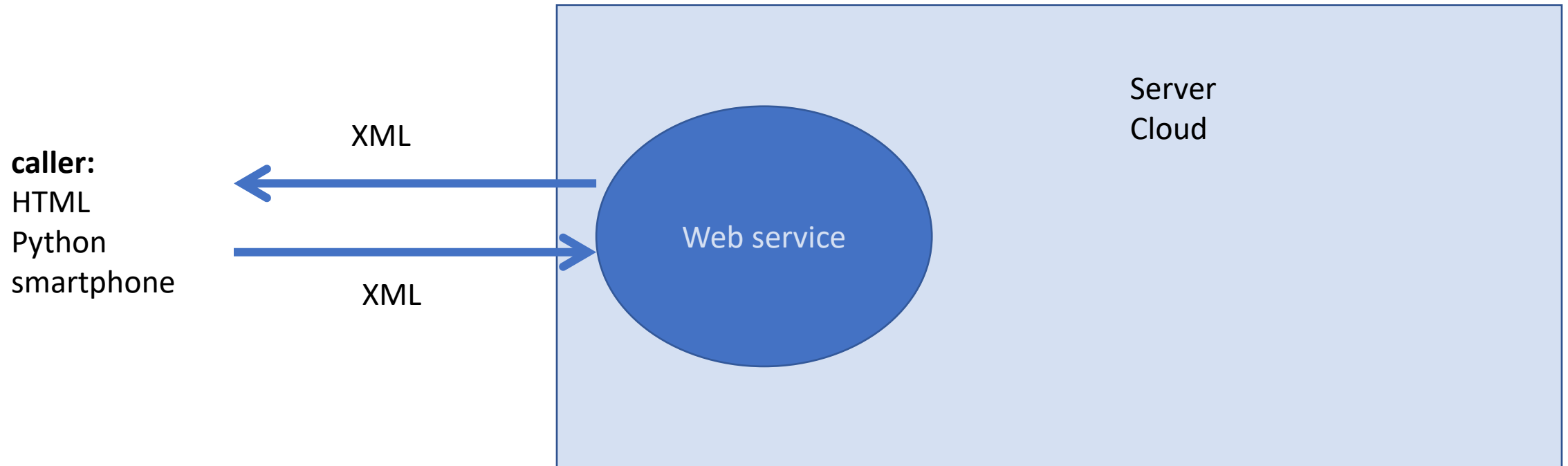
JavaScript, programming in client
Jquery, AJAX, Angular, ..., etc



HTML is static, no dynamics

The current technology: Web service/microservice

Web services refer to a set of loosely coupled software components that exchange information with each other using universal web communication standards and languages. They can exchange information between two different systems regardless of the operating systems or programming languages on which the systems are based. They can be used to build open standard web-based applications linking systems of two different organizations, and they can also be used to create applications that link disparate systems within a single company. Different applications can use web services to communicate with each other in a standard way without time-consuming custom coding.



- The collection of web services that are used to build a firm's software systems constitutes what is known as a service-oriented architecture. **A service oriented architecture (SOA) is set of self-contained services that communicate with each other to create a working software application.** Business tasks are accomplished by executing a series of these services. Software developers reuse these services in other combinations to assemble other applications as needed.

Large companies can have 1000s of microservices

FIGURE 5.10 AMAZON WEB SERVICES

Amazon Web Services (AWS) is a collection of web services that Amazon provides to users of its cloud platform. AWS is the largest provider of cloud computing services in the United States.



Microservice in reality

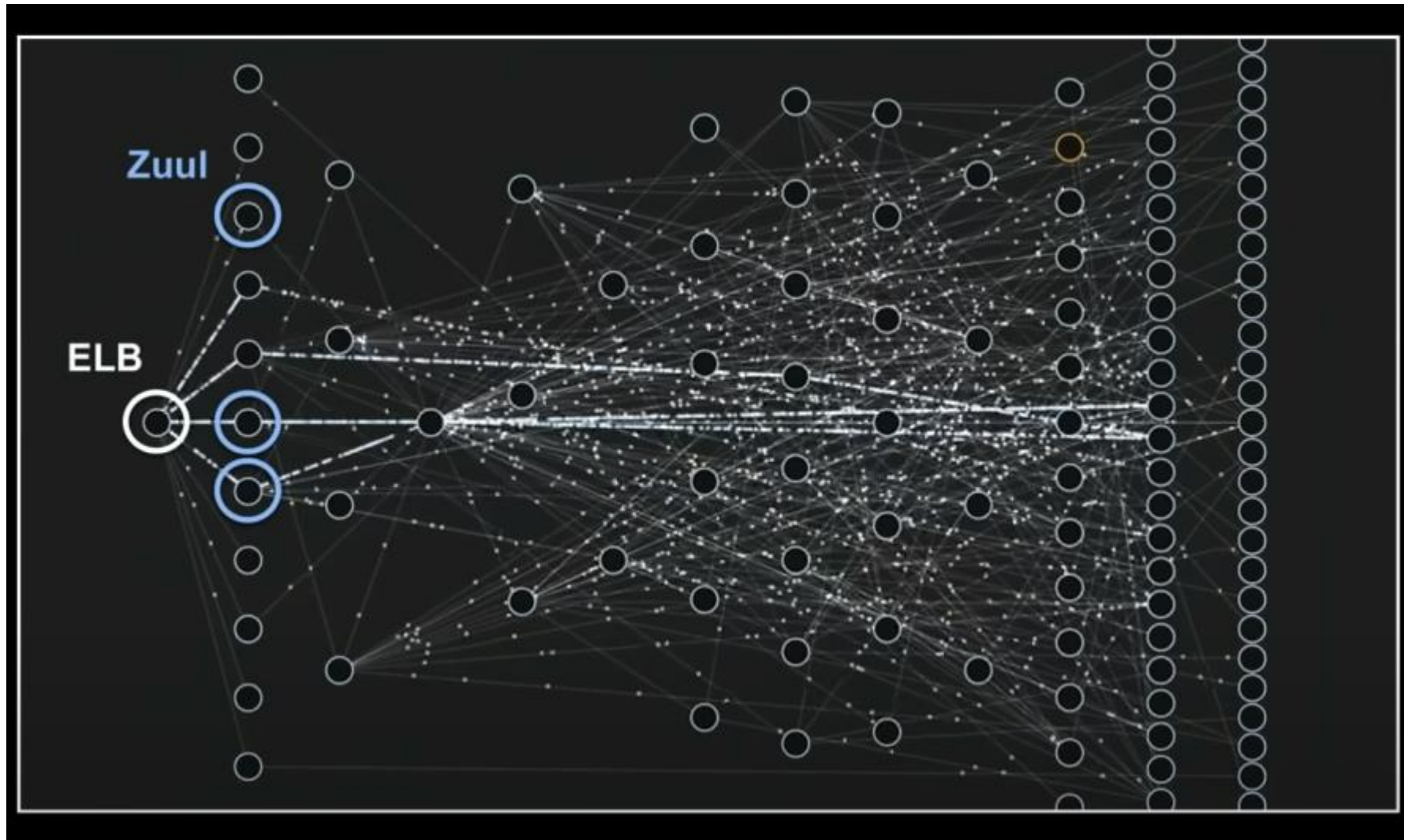
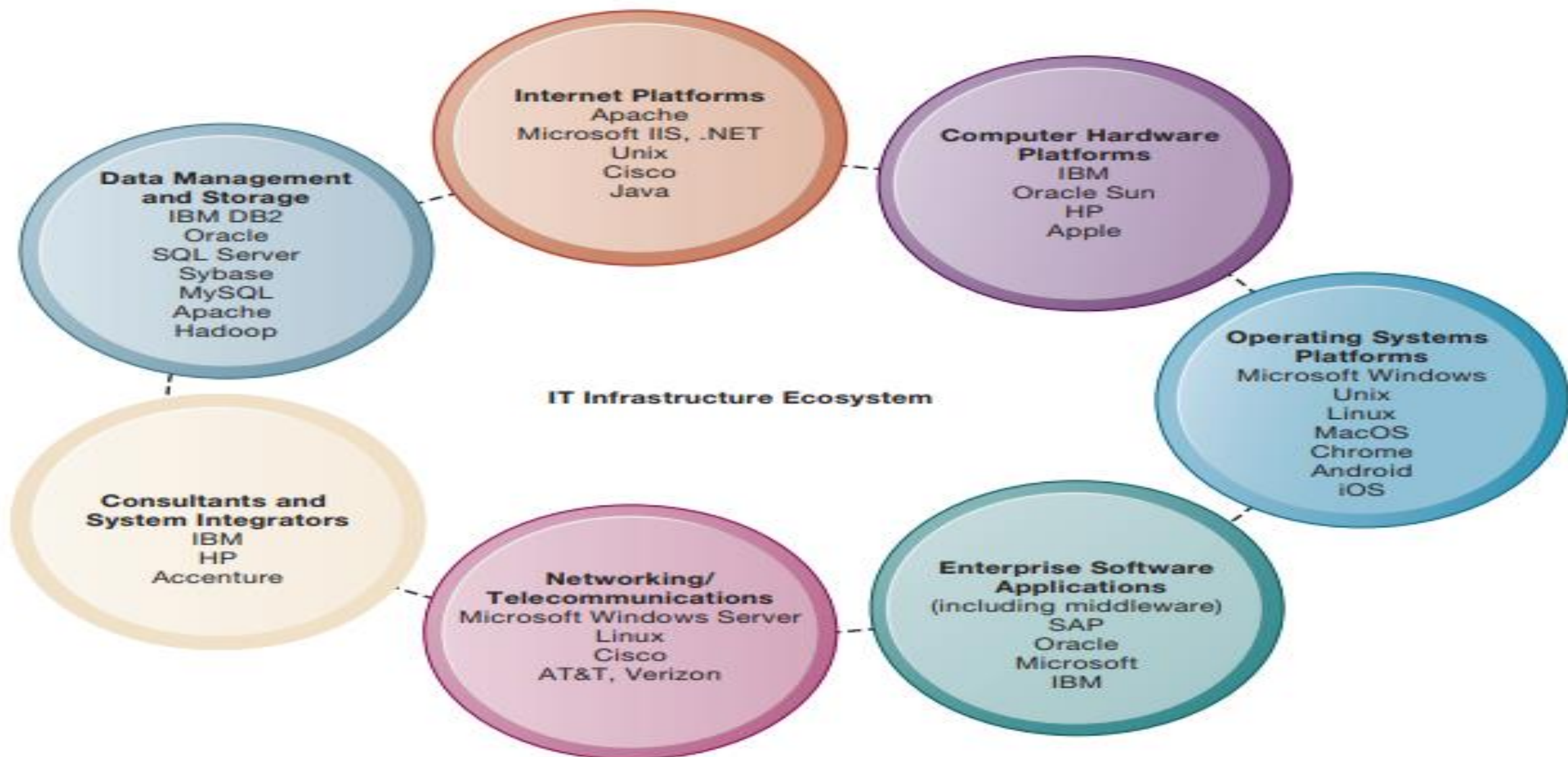


FIGURE 5.8 THE IT INFRASTRUCTURE ECOSYSTEM

There are seven major components that must be coordinated to provide the firm with a coherent IT infrastructure. Listed here are major technologies and suppliers for each component.

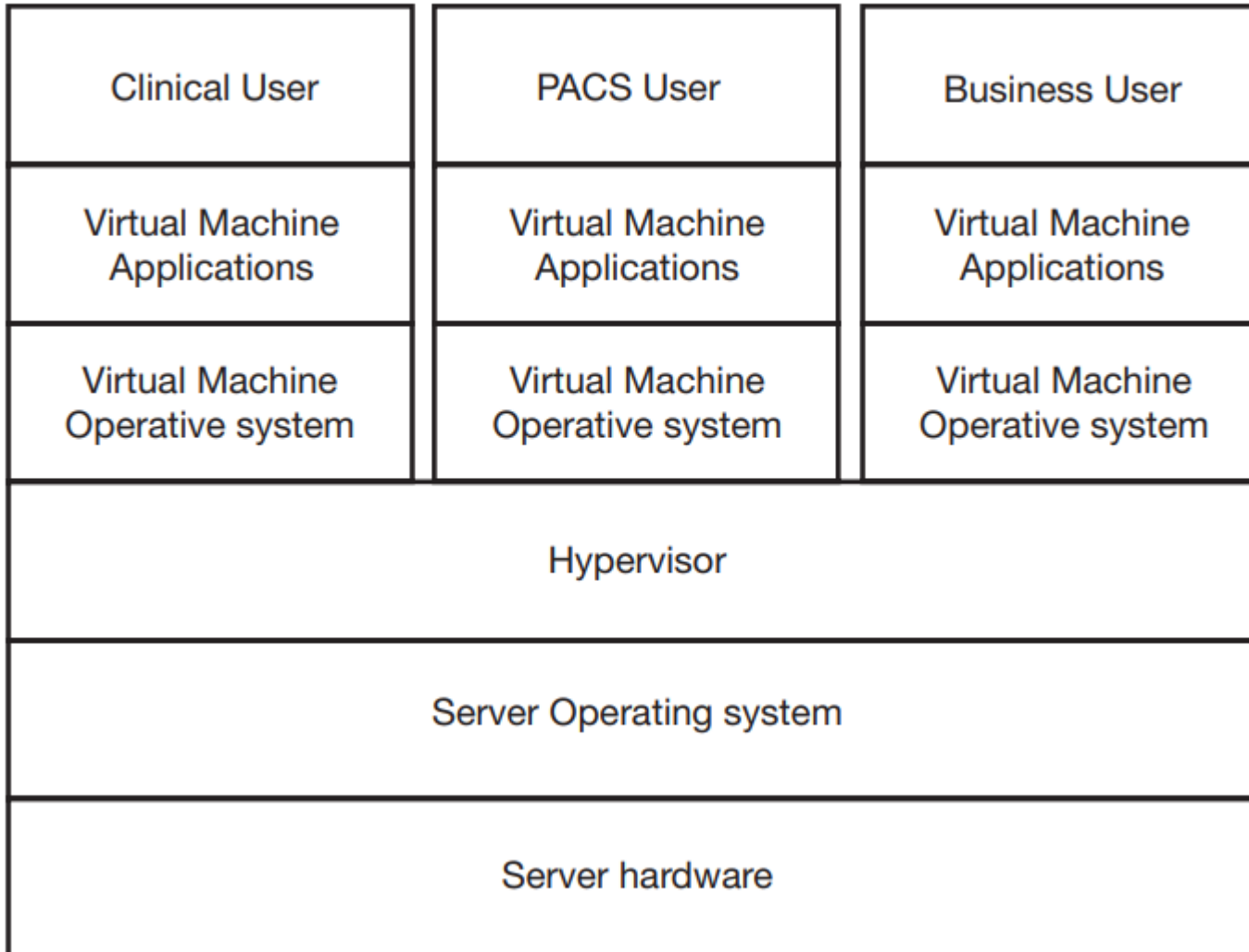


Homework

- summarize this video

<https://www.youtube.com/watch?v=CZ3wluvmHeM>

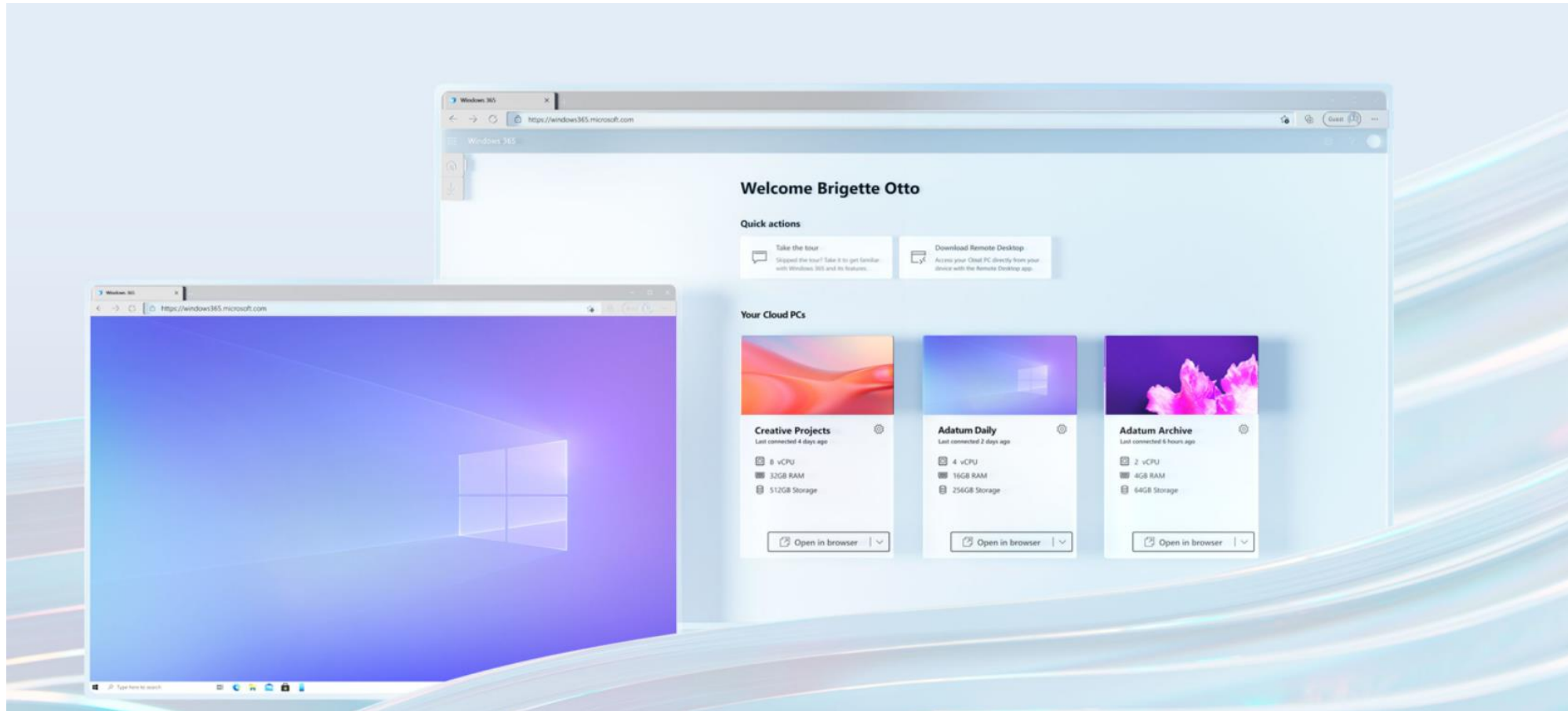
Server Virtualization



[Read more VMware](#)

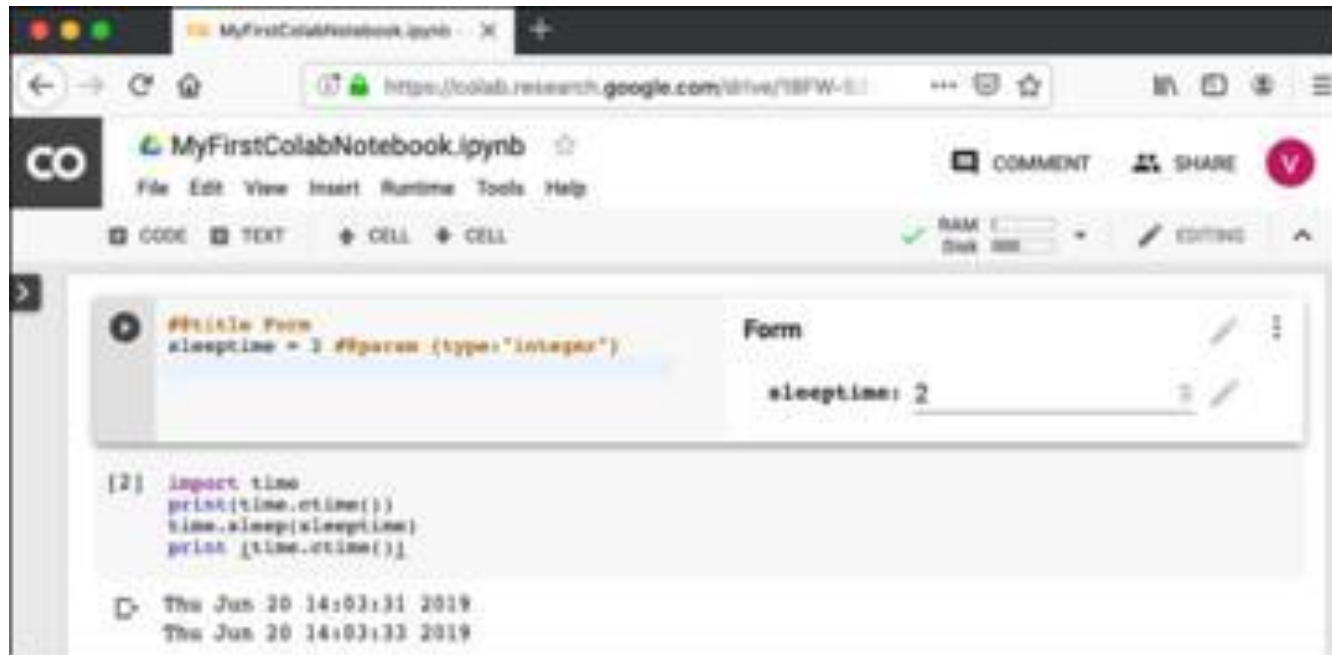
Cloud computing: windows-365

<https://www.microsoft.com/en-us/windows-365> (summarize this as a homework)



Cloud computing: google Co-lab

- You can run your code in google computers



Cloud computing characteristics

- **Cloud Computing Provides *On-Demand Self-Service***
- **Cloud Computing Encompasses the Characteristics of Grid Computing**
- **Cloud Computing Encompasses the Characteristics of Utility Computing**
- **Cloud Computing Uses Broad Network Access**

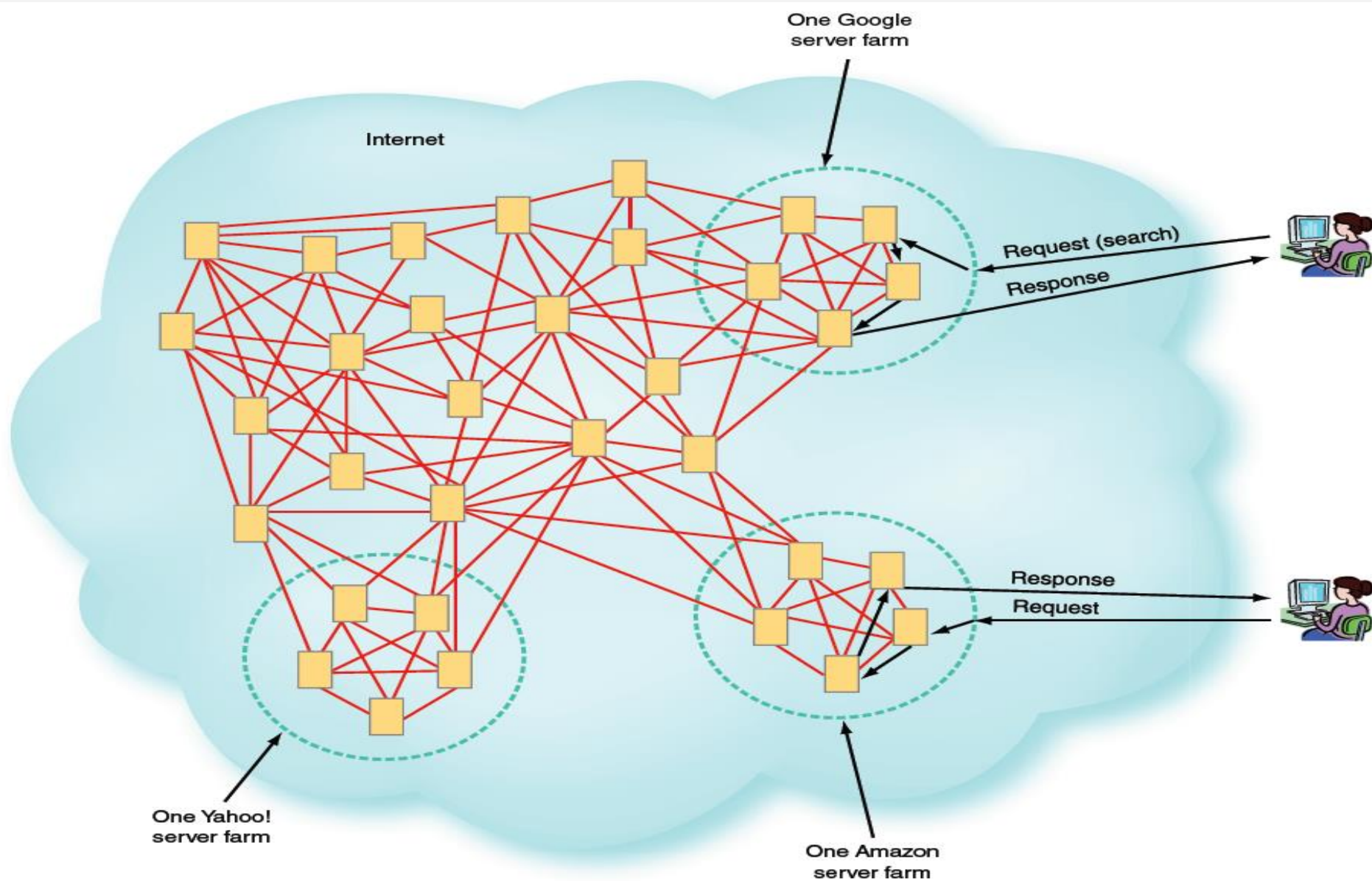
The cloud provider's computing resources are available over a network, accessed with a web browser, and they are configured so that they can be used with any computing device.

Cloud Computing Pools Computing Resources

Cloud Computing Often Occurs on Virtualized Servers



FIGURE TG 3.1 A server farm. Notice the ventilation in the racks and ceiling.



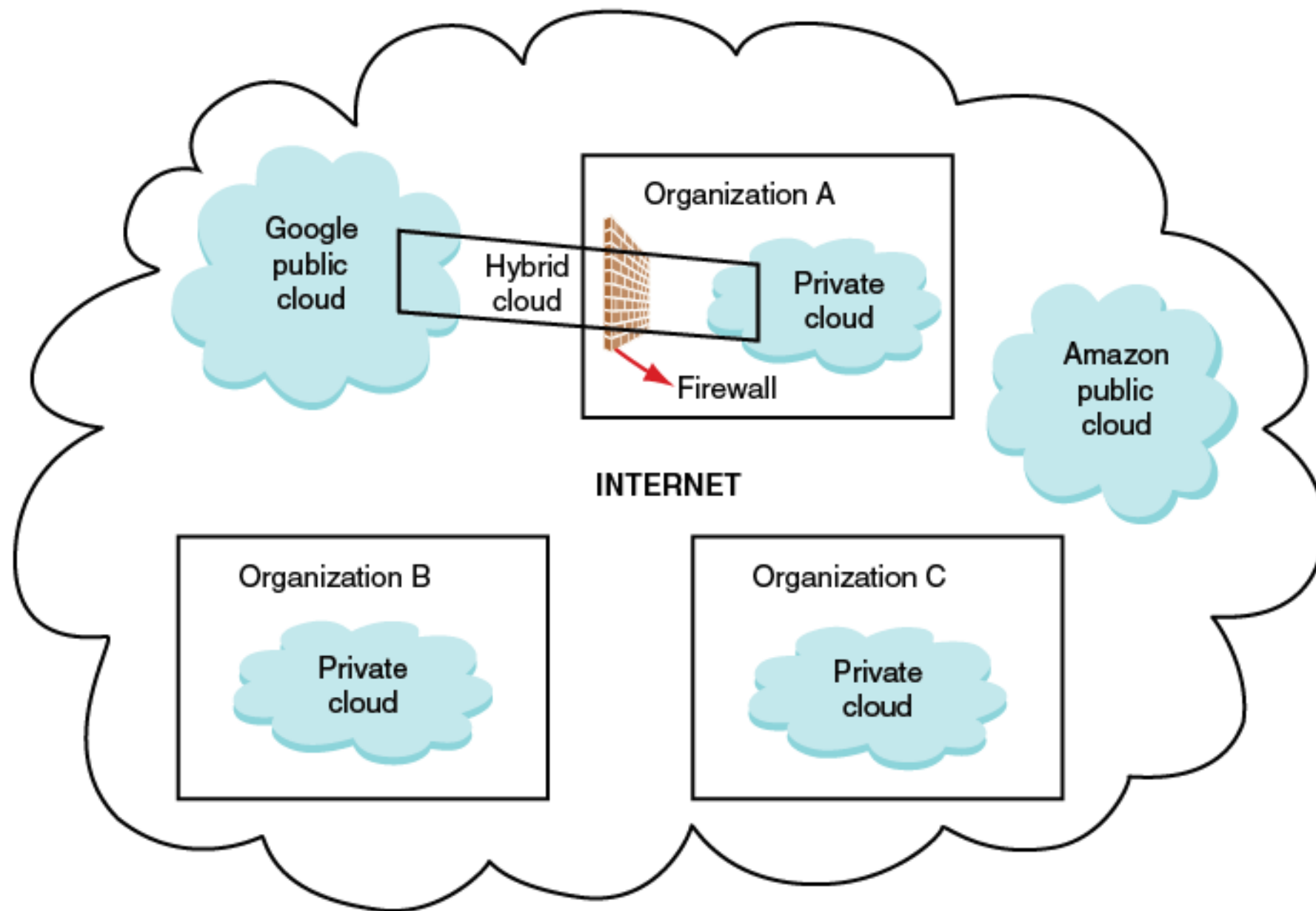


FIGURE TG 3.3 Public clouds, private clouds, and hybrid clouds.

Software-as-a-Service

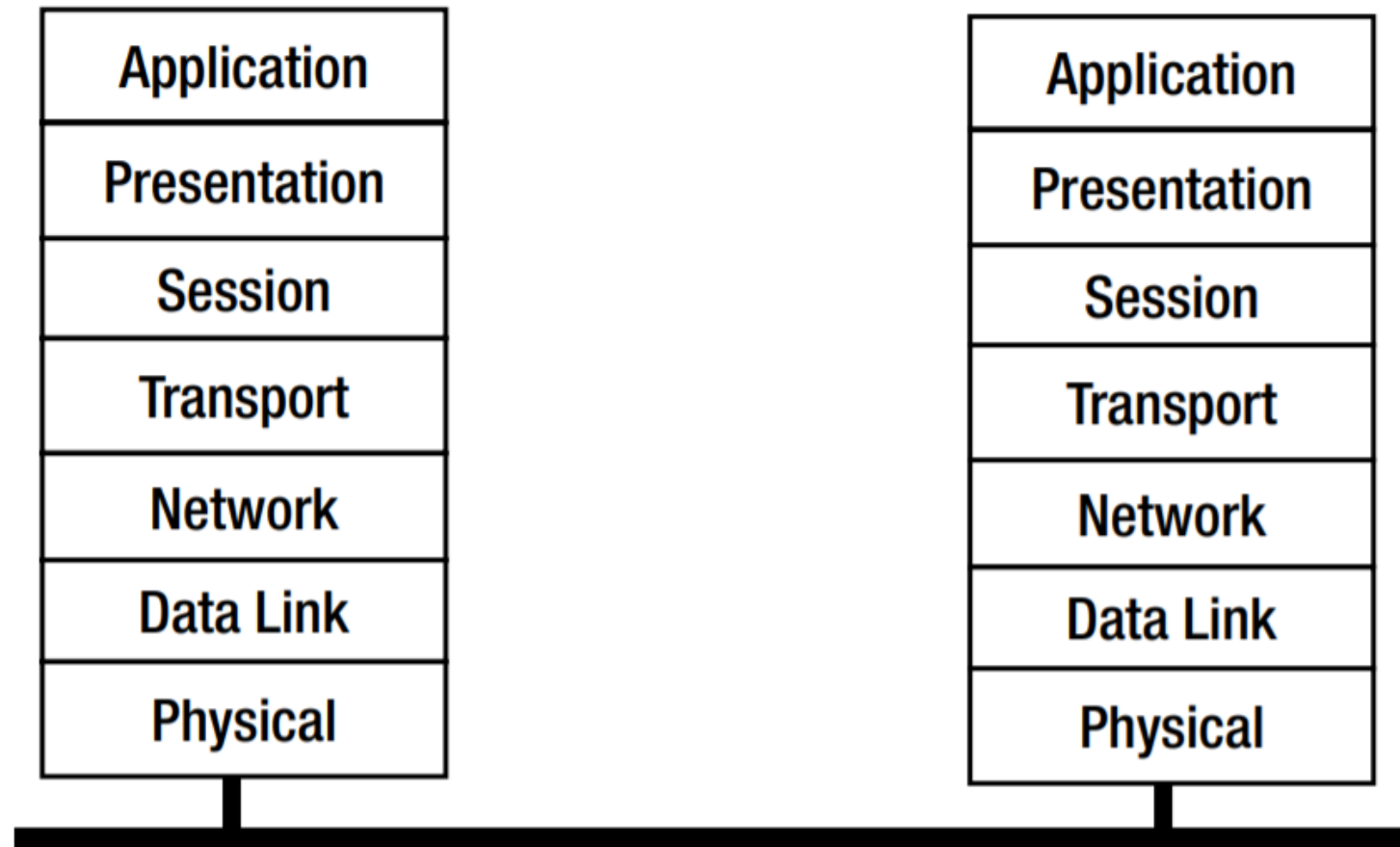
- With the **software-as-a-service (SaaS)** delivery model, cloud computing vendors provide software that is specific to their customers' requirements. SaaS is the most widely used service model, and it provides a broad range of software applications. SaaS providers typically charge their customers a monthly or yearly subscription fee.
- In the **platform-as-a-service (PaaS)** model, customers rent servers, operating systems, storage, a database, software development technologies such as Java and .NET, and network capacity over the Internet. The PaaS model allows the customer to both run existing applications and to develop and test new applications.
- With the **infrastructure-as-a-service (IaaS)** model, cloud computing providers offer remotely accessible servers, networks, and storage capacity. They supply these resources on demand from their large resource pools, which are located in their data centres.

ON-PREMISE SOFTWARE	INFRASTRUCTURE-AS-A-SERVICE	PLATFORM-AS-A-SERVICE	SOFTWARE-AS-A-SERVICE
CUSTOMER MANAGES Applications Data Operating system Servers Virtualization Storage Networking	CUSTOMER MANAGES Applications Data Operating system VENDOR MANAGES Servers Virtualization Storage Networking	CUSTOMER MANAGES Applications Data VENDOR MANAGES Operating system Servers Virtualization Storage Networking	VENDOR MANAGES Applications Data Operating system Servers Virtualization Storage Networking
Examples	Amazon, IBM, Google, Microsoft, Rackspace	Microsoft Windows Azure, Google App Engine, Force.com	Salesforce.com, Google Apps, Dropbox, Apple iCloud, Box.net

FIGURE TG 3.4 Comparison of on-premise software, infrastructure-as-a-service, platform-as-a-service, and software-as-a-service.

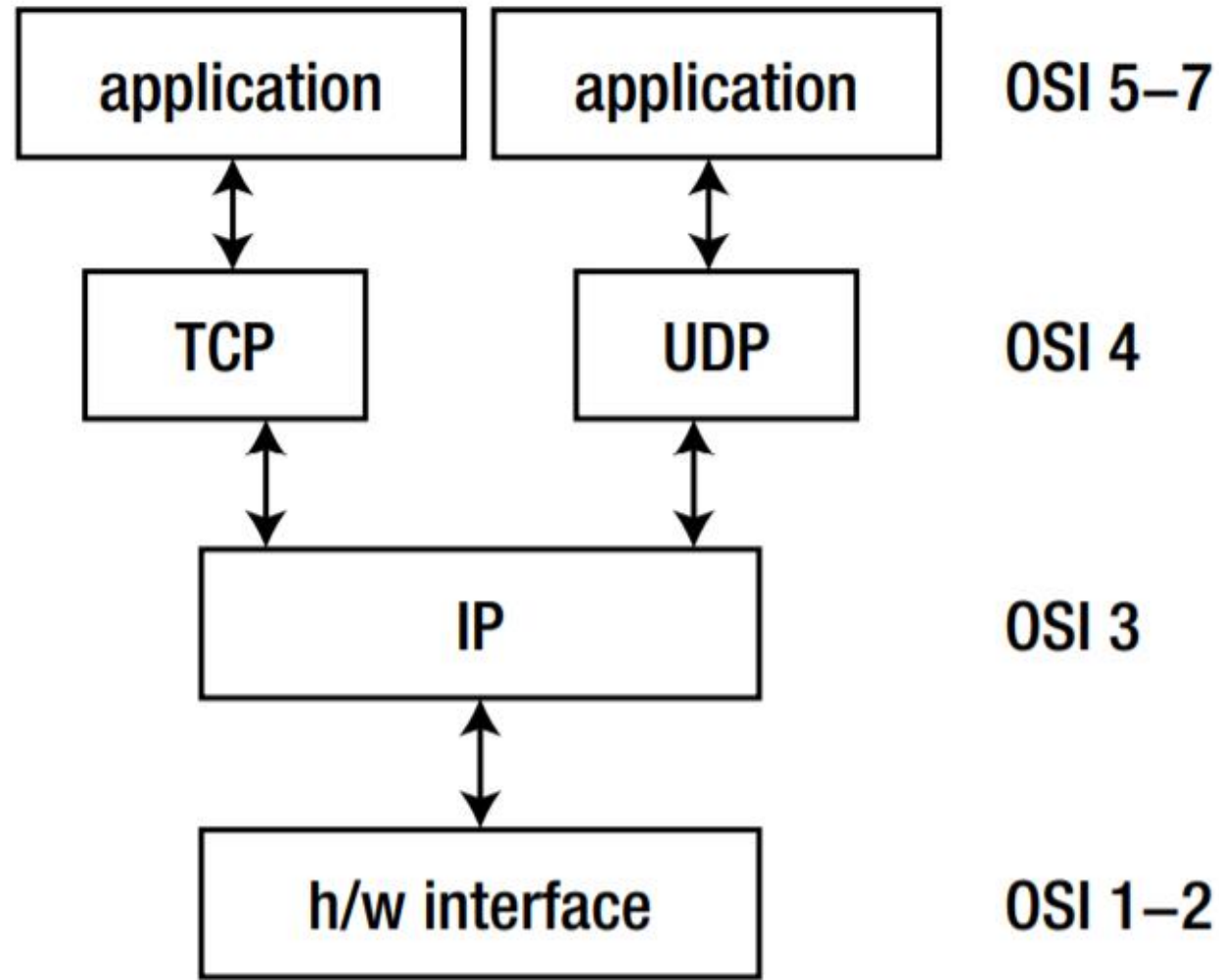
ISO OSI Protocol

Although it was never properly implemented, the OSI (Open Systems Interconnect) protocol has been a major influence in ways of talking about and influencing distributed systems design. It is commonly given as shown in Figure 1-1.



TCP/IP Protocol

While the OSI model was being argued, debated, partly implemented, and fought over, the DARPA Internet research project was busy building the TCP/IP protocols. These have been immensely successful and have led to *The Internet* (with capitals). This is a much simpler stack, as shown in Figure 1-2.



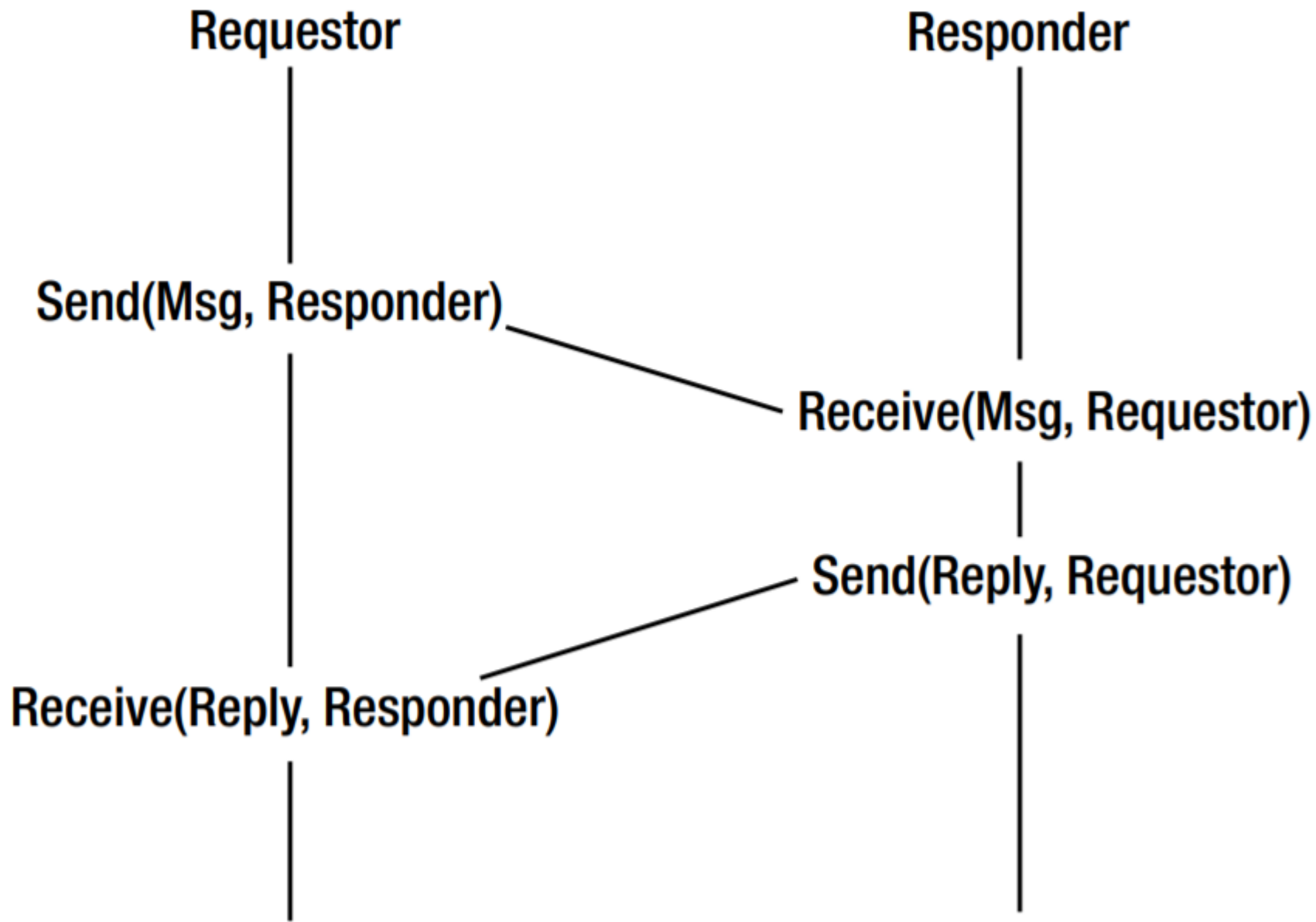


Figure 1-4. *The message passing communications model*

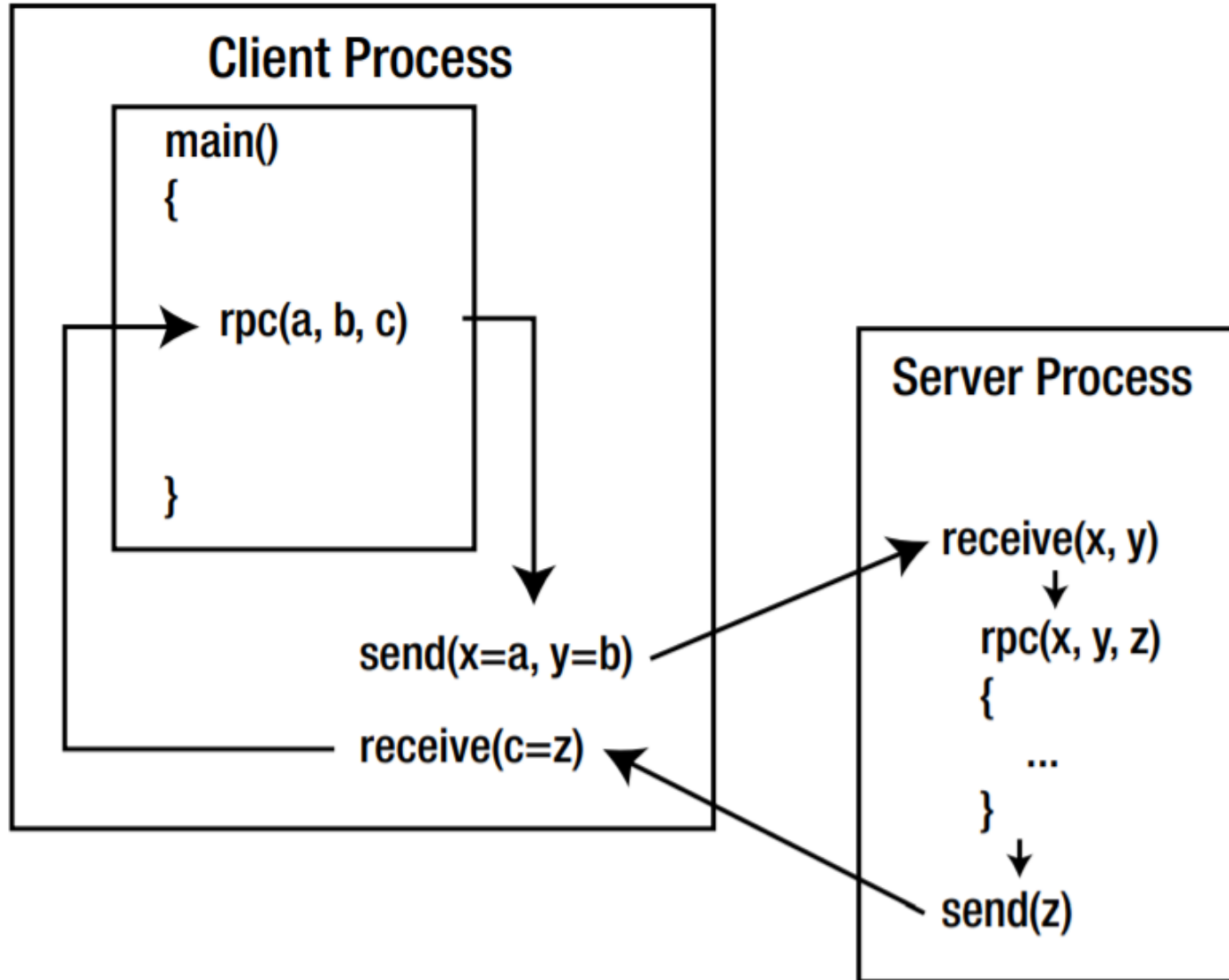
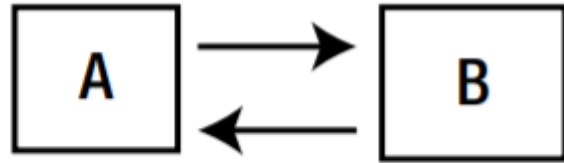


Figure 1-5. *The remote procedure call communications model*

Distributed Computing Models

client-server



peer-to-peer



If both components are equivalent, both able to initiate and to respond to messages, then we have a peer-to-peer system. Note that this is a logical classification: one peer may be a 16,000 core supercomputer, the other might be a mobile phone. But if both can act similarly, then they are peers.

Client-Server System

Another view of a client-server system is shown in Figure 1-7.

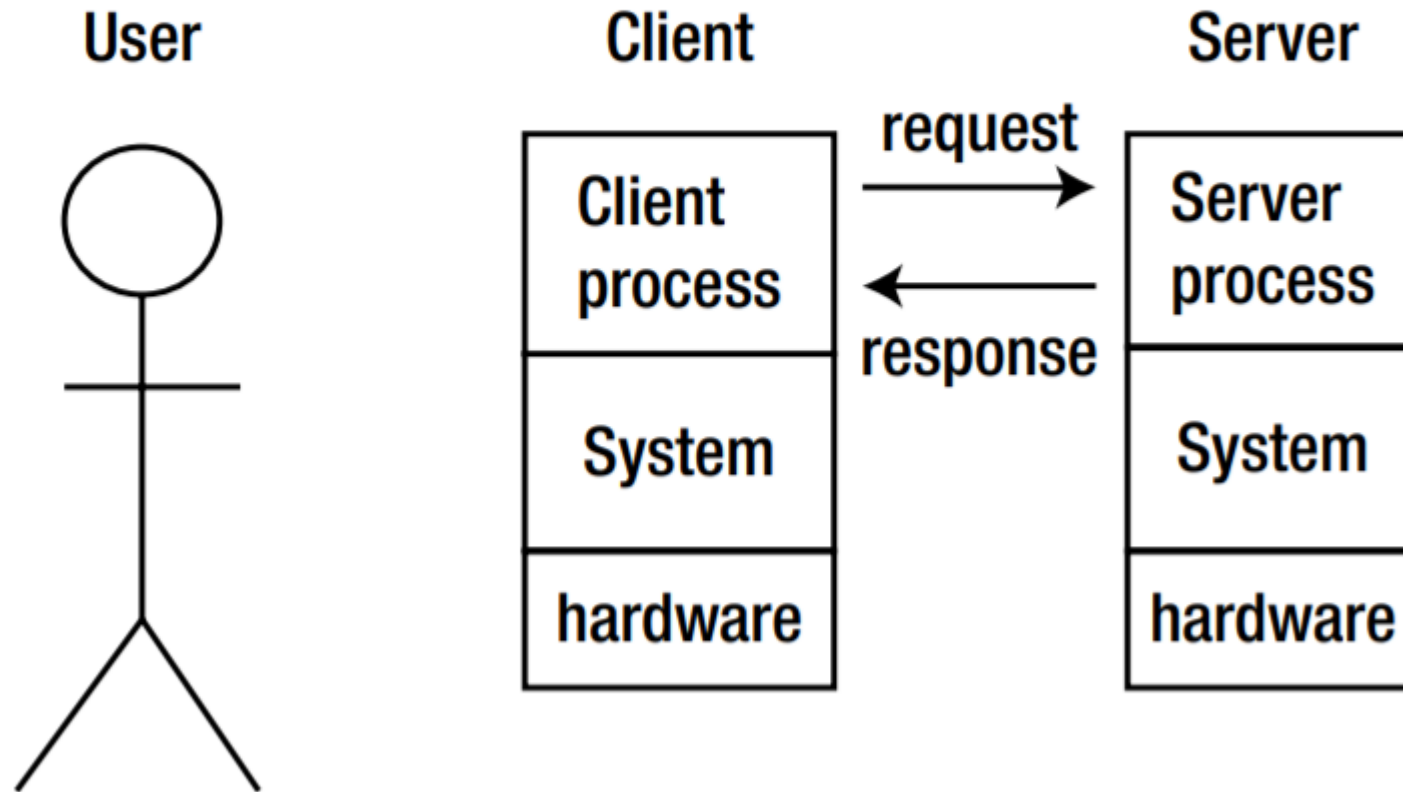


Figure 1-7. *The client-server system*

Server Distribution

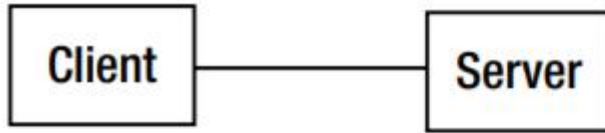


Figure 1-9. *The single client, single server system*

However, you can also have multiple clients, single server, as illustrated in Figure 1-10.

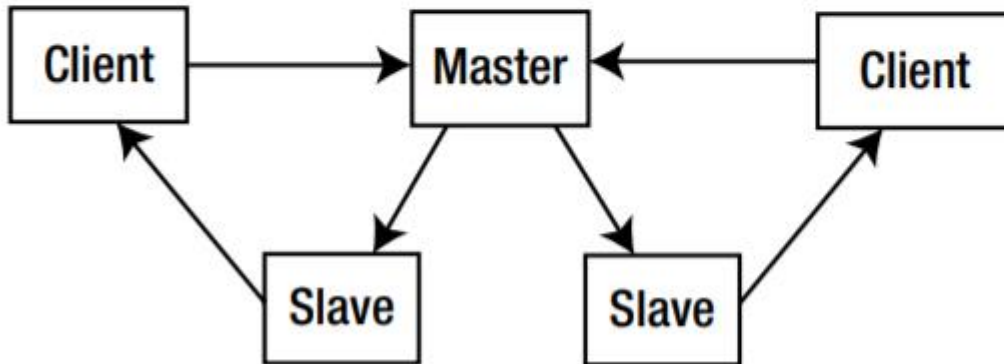


Figure 1-10. *The multiple clients, single server system*

Component Distribution

- A simple but effective way of decomposing many applications is to consider them as made up of three parts:
 - **Presentation component:** responsible for interactions with the user, both displaying data and gathering input. It may be a modern GUI interface with buttons, lists, menus, etc., or an older command-line style interface, asking questions and getting answers. It could also encompass wider interaction styles, such as the interaction with physical devices such as a cash register, ATM, etc. It could also cover the interaction with a non-human user, as in a machine-to-machine system. The details are not important at this level.
 - **Application logic:** responsible for interpreting the users' responses, for applying business rules, for preparing queries, and for managing responses from the third component.
 - **Data access:** responsible for storing and retrieving data. This will often be through a database, but not necessarily

Gartner Classification

Based on this threefold decomposition of applications, Gartner considered how the components might be distributed in a client-server system. They came up with five models, shown in Figure 1-12.

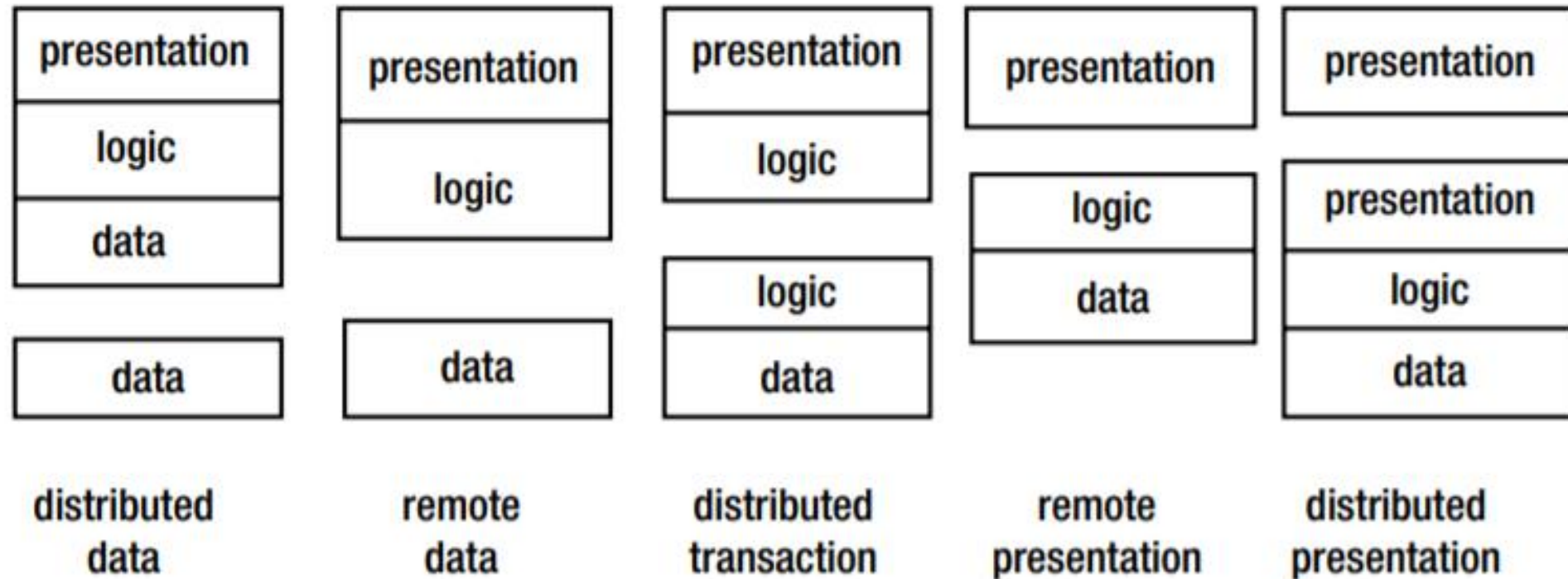


Figure 1-12. Gartner's five models

GO language

- Go was chosen both because it is easy to learn for students, and because is pretty well-suited for writing concurrent, distributed applications (goroutines come in particularly handy). Over the course of four labs, students build a fault-tolerant, sharded key-value store.

First Program in GO

```
package main

import "fmt"

func main() {
    fmt.Print("Hello World\n")
    fmt.Print("done")
}
```

package main In Go, you can write two types of programs: executable programs and a shared library. When you write executable programs, you must give main as the package name to make the package an executable program: the package main tells the Go compiler that the package should compile as an executable program.

When you write packages as shared libraries, there are no main packages or main functions in the package.

```
D:\GO>go run P1.go
Hello World
done
D:\GO>
```

Goroutine is not function nor procedure

```
package main
const N = 1
func c() {
    for i := 0; i < N; i++ {
        print("-")
    }
}
func main() {
    println("begin") // A
    go c()           // fork C
    for i := 0; i < N; i++ {
        print(".") // B
    }
    println("\nend") // D
}
```

```
D:\GO>go run P2.go
```

```
begin
```

```
.
```

```
end
```

```
D:\GO>
```

What is happening?

Why do not we see “–”?


```
const N = 10
```

```
D:\GO>go run P2.go
```

```
begin
```

```
.....
```

```
end
```

```
D:\GO>
```

```
const N = 100
```

```
D:\GO>go run P2.go
```

```
begin
```

```
.....
```

```
end
```

```
D:\GO>
```

Now, we start to see ----- . Why?

some points were printed, then -----, after that some points were printed. What does this mean?

Run again only one -. what is going on?

```
D:\GO>go run P2.go
```

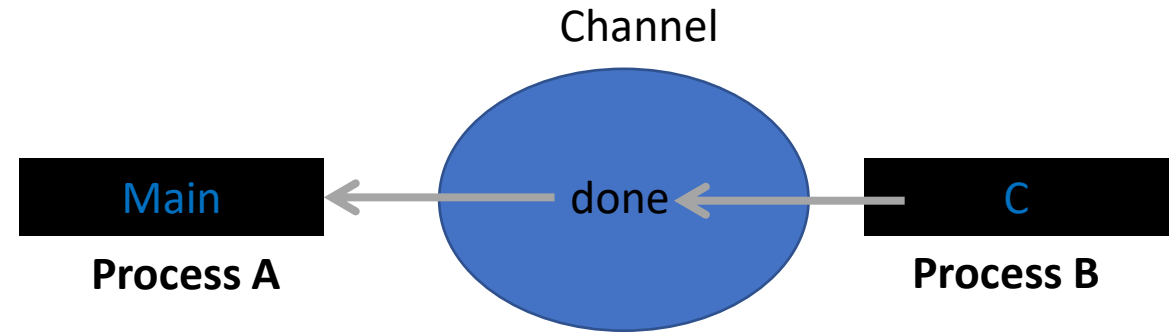
```
begin
```

```
.....-
```

```
end
```

Channel

```
package main
const N = 100
var done = make(chan int)
func c() {
    for i := 0; i < N; i++ {
        print("-")
    }
    done <- 0
}
func main() {
    println("begin") // A
    go c()           // fork C
    for i := 0; i < N; i++ {
        print(".") // B
    }
    <-done           // wait C
    println("\nend") // D
}
```



Main will wait until get something from the channel done

C will set value in done after it finishes, so C will complete.

```
D:\GO>go run P2.go
```

```
begin
```

```
.....
-----
-----
.....
```

```
end
```

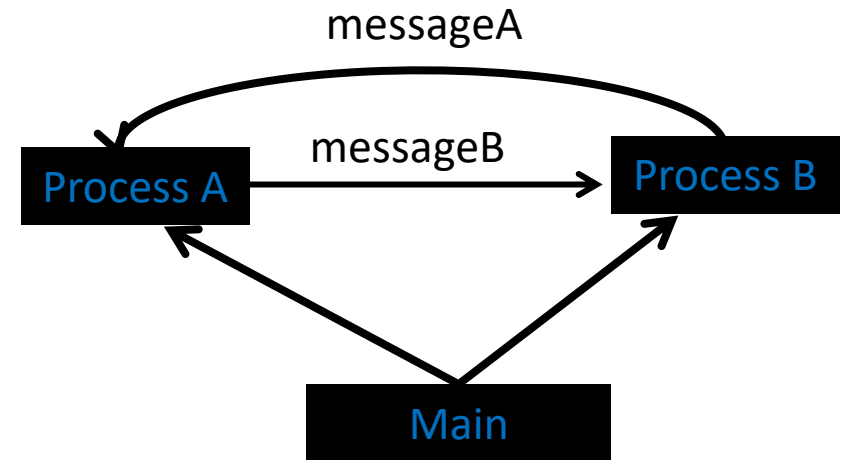
```
D:\GO>
```

```

package main
import "fmt"
var messageA = make(chan int)
var messageB = make(chan string)
func processA() {
    for { // this is the infinite loop
        <-messageA // processA waits until have value in messageA
        // we can process the messageA as we want
        // now we print it only
        fmt.Print("Process A receives a message:\n")
        messageB <- "done from A"
    }
}
func processB() {
    for { // this is the infinite loop
        fmt.Print("Process B sends an message to Process A\n")
        messageA <- 1 // processB sends a message to
        <-messageB // process A waits until having a message in messageB
        // we can process the messageA as we want
        // now we print it only
        fmt.Print("Process B receives the results from A\n")
    }
}
func main() {
    println("begin") // A
    go processB()
    go processA()
    for {

    }
}

```



Process A receives a message:
 Process B receives the results from A
 Process B sends an message to Process A
 Process A receives a message:
 Process B receives the results from A
 Process B sends an message to Process A
 Process A receives a message:
 Process B receives the results from A
 Process B sends an message to Process A
 Process A receives a message:
 Process B receives the results from A

.
 .
 .
 .

```

package main
import "fmt"
var messageA = make(chan int, 0)
var messageB = make(chan int, 0)
func processA() {
    for { // this is the infinte loop
        var input int = 0
        input = <-messageA // processA waits until have value in messageA
        // we can process the messageA as we want
        // now we print it only
        fmt.Print("Process A receives a message: ", input, "\n")
        messageB <- input + 1
    }
}

func processB() {

    var result int = 0
    for { // this is the infinte loop
        fmt.Print("Process B sends an message to Process A\n")
        messageA <- result // processB sends a message to
        result = <-messageB // process A waits until having a message in messageB
        // we can process the messageA as we want
        // now we print it only
        fmt.Print("Process B receives the results from A as ", result, "\n")
    }
}

func main() {
    println("begin") // A
    go processB()
    go processA()
    for {

    }
}

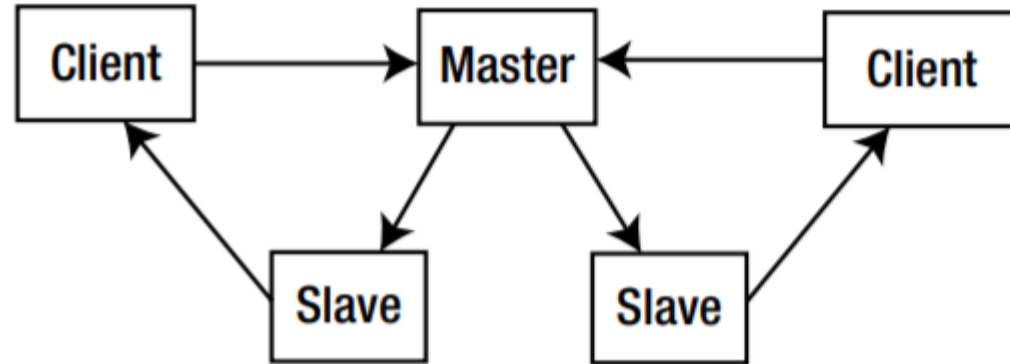
```

Pass value in the channel

Process A receives a message: 364
 Process B receives the results from A as 365
 Process B sends an message to Process A
 Process A receives a message: 365
 Process B receives the results from A as 366

.
 .
 .

Test your self: Can you do something like this with GO



Can we call processA from HTTP

```
func viewHandler(w http.ResponseWriter, r
*http.Request) {
    input_str := r.URL.Path[len("/runA/"):]
    var input_int int = 0

    fmt.Sscan(input_str, &input_int)

    inputB <- input_int
    out := <-outputB
    fmt.Fprintf(w, "<h1>%d</h1>", out)
}

func main() {
    println("begin") // A
    go processB()
    go processA()

    // now HTTP server is running
    http.HandleFunc("/runA/", viewHandler)
    log.Fatal(http.ListenAndServe(":8080", nil))
}
```

```
func processB() {

    var input int = 0
    var output int = 0

    for { // this is the infinite loop
        input = <-inputB
        fmt.Print("Process receives input ", input, "from web ")
        messageA <- input // processB sends a message to
        output = <-messageB // process A waits until having a message
        // we can process the messageA as we want
        // now we print it only
        outputB <- output
    }
}
```

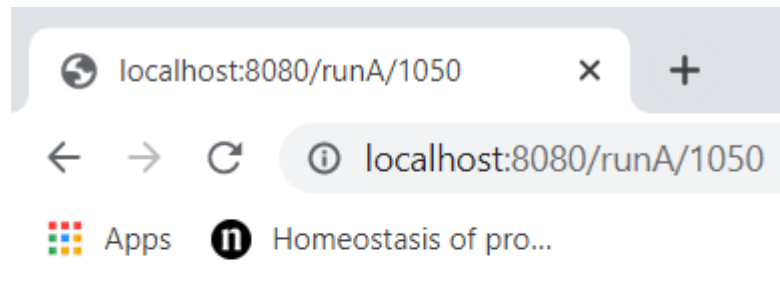
Now process A is called from Web Browser

```
D:\GO>p5
```

```
begin
```

```
Process receives input 1050 from web server and the process B sends it to Process A
```

```
Process A receives a message: 1050
```



1051

Let be more formal

Lecture 4: Google File Structure

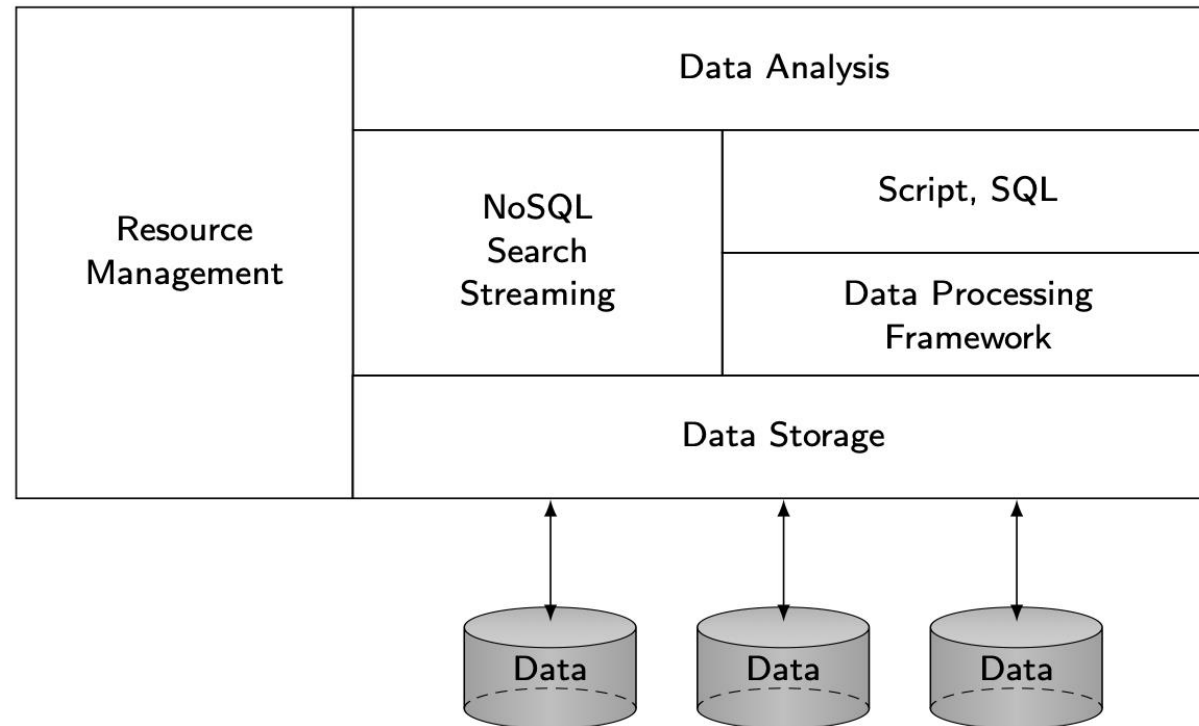
Outline

- Big Data Processing
 - Distributed storage systems
 - Processing platforms

Four Vs

- Volume
 - Increasing data size: petabytes (10^{15}) to zettabytes (10^{21})
- Variety
 - Multimodal data: structured, images, text, audio, video
 - 90% of currently generated data unstructured
- Velocity
 - Streaming data at high speed
 - Real-time processing
- Veracity
 - Data quality

Big Data Software Stack



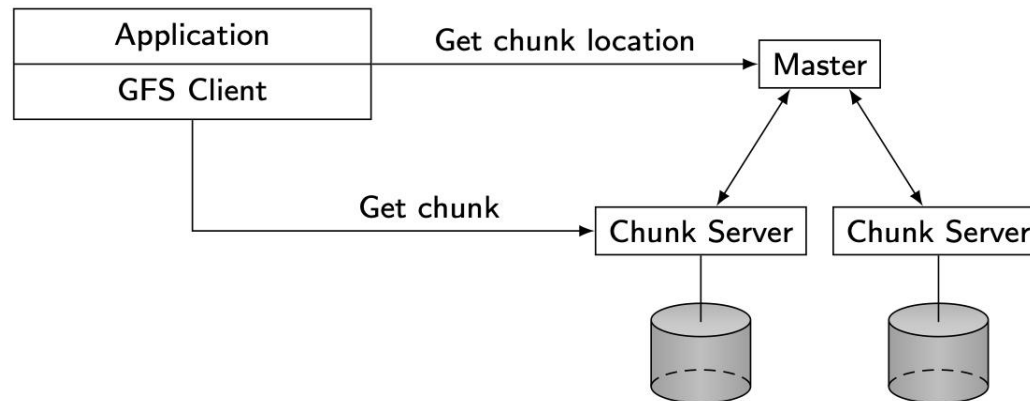
Distributed Storage System

Storing and managing data across the nodes of a shared-nothing cluster

- Object-based
 - Object = $\langle \text{oid}, \text{data}, \text{metadata} \rangle$
 - Metadata can be different for different object
 - Easy to move
 - Flat object space $\rightarrow$ billions/trillions of objects
 - Easily accessed through REST-based API (get/put)
 - Good for high number of small objects (photos, mail attachments)
- File-based
 - Data in files of fixed- or variable-length records
 - Metadata-per-file stored separately from file
 - For large data, a file needs to be partitioned and distributed

Google File System (GFS)

- Targets shared-nothing clusters of thousands of machines
- Targets applications with characteristics:
 - Very large files (several gigabytes)
 - Mostly read and append workloads
 - High throughput more important than low latency
- Interface: create, open, read, write, close, delete, snapshot, record append



Big Data Processing Platforms

- Applications that do not need full DBMS functionality
 - Data analysis of very large data sets
 - Highly dynamic, irregular, schemaless, ...
- “Embarrassingly parallel problems”
- MapReduce/Spark
- Advantages
 - Flexibility
 - Scalability
 - Efficiency
 - Fault-tolerance
- Disadvantage
 - Reduced functionality
 - Increased programming effort

MapReduce Basics

- Simple programming model
 - Data structured as (key, value) pairs
 - E.g. (doc-id, content); (word, count)
 - Functional programming style with two functions
 - $\text{map}(k1, v1) \rightarrow \text{list}(k2, v2)$
 - $\text{reduce}(k2, \text{list}(v2)) \rightarrow \text{list}(v3)$
- Implemented on a distributed file system (e.g. Google File System) on very large clusters

map Function

- User-defined function
 - Processes input (key, value) pairs
 - Produces a set of **intermediate** (key, value) pairs
 - Executes on multiple machines (called **mapper**)
- **map** function I/O
 - **Input:** read a from distributed file system (DFS)
 - **Output:** Write to intermediate file on local disk
- MapReduce library
 - Execute **map** function
 - Groups together all intermediate values with same key
 - Passes these lists to **reduce** function
- Effect of **map** function
 - Processes and partitions input data
 - Builds a distributed map (transparent to user)
 - Similar to “group by” operator in SQL

reduce Function

- User-defined function
 - Accepts one intermediate key and a set of values for that key (i.e. a list)
 - Merges these values together to form a (possibly) smaller set
 - Computes the reduce function generating, typically, zero or one output per invocation
 - Executes on multiple machines (called **reducer**)
- **reduce** function I/O
 - **Input:** read from intermediate files using remote reads on local files of corresponding mappers
 - **Output:** Write result back to DFS
- Effect of map function
 - Similar to aggregation function in SQL

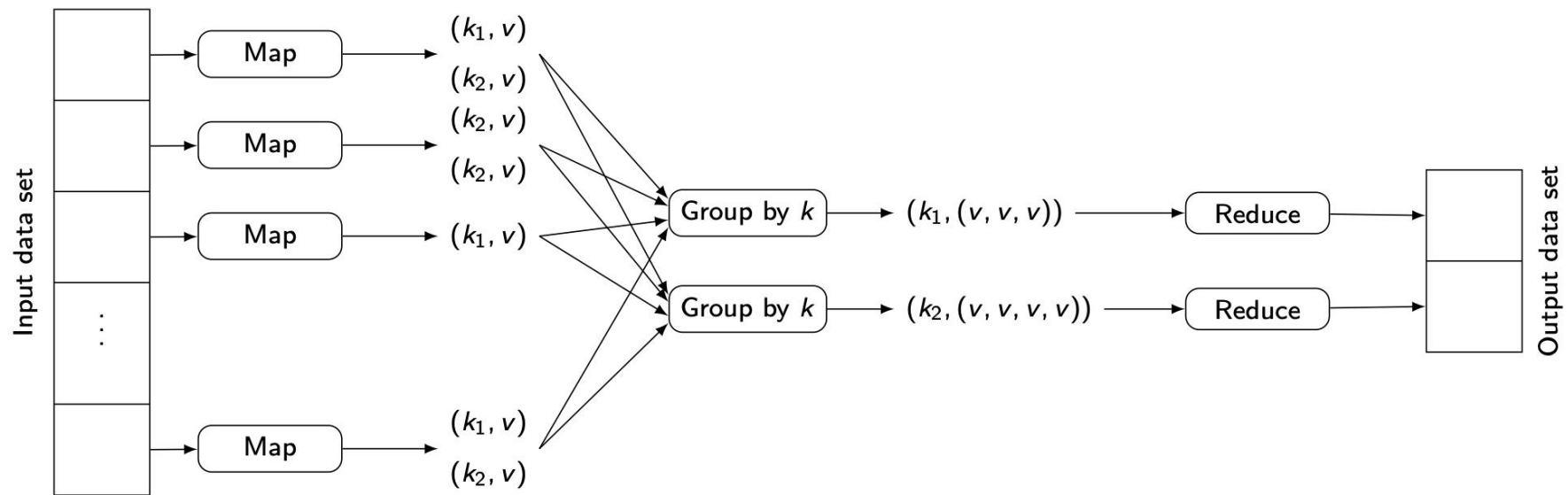
Example

Consider EMP (ENO, ENAME, TITLE, CITY)

```
SELECT    CITY, COUNT (*)  
FROM      EMP  
WHERE     ENAME LIKE "%Smith"  
GROUP BY CITY
```

```
map (Input: (TID,EMP), Output: (CITY, 1)  
    if EMP.ENAME like ``\%Smith'' return (CITY, 1)  
reduce (Input: (CITY, list(1)), Output: (CITY, SUM(list)))  
    return (CITY, SUM(1))
```

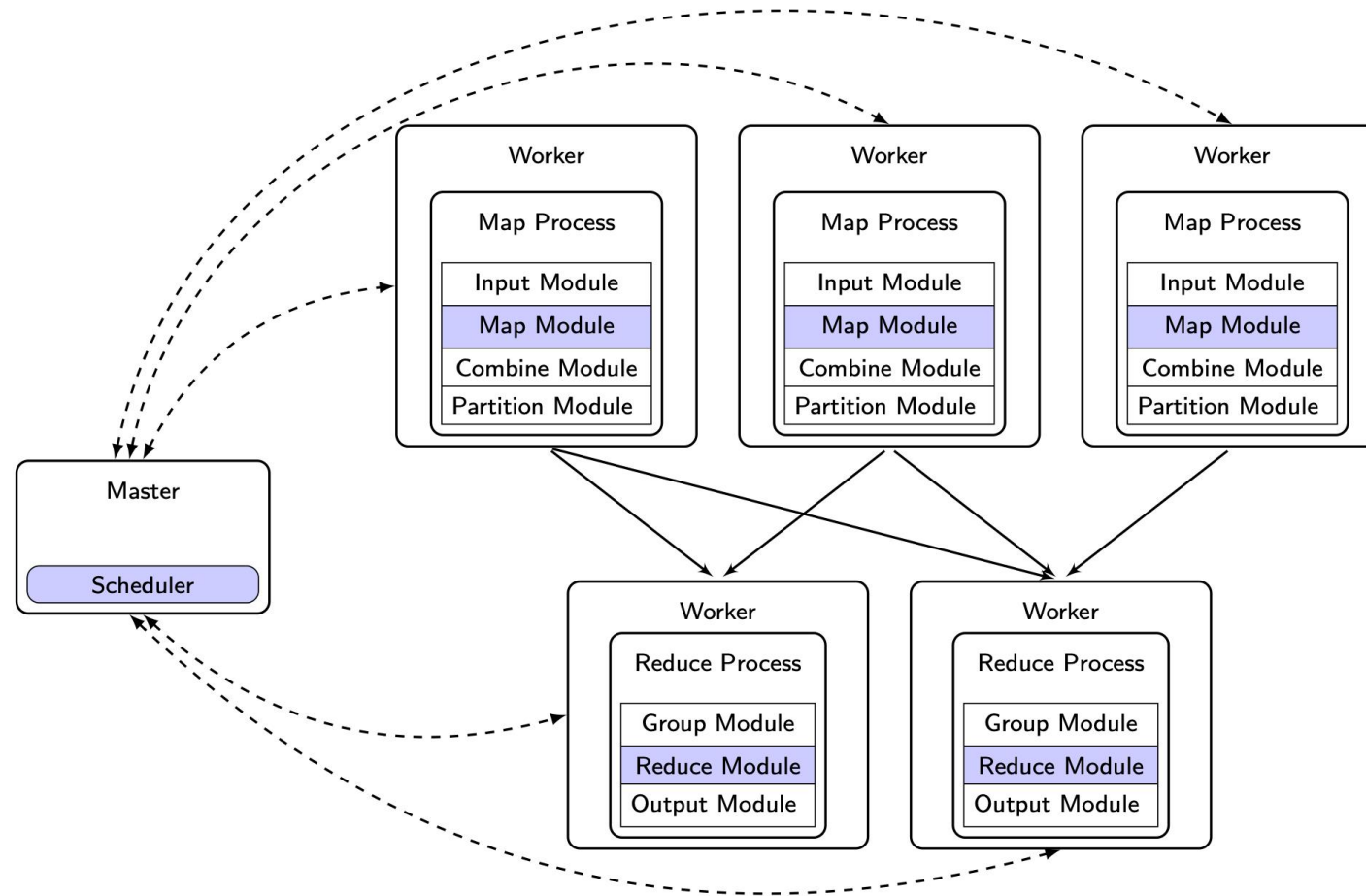
MapReduce Processing



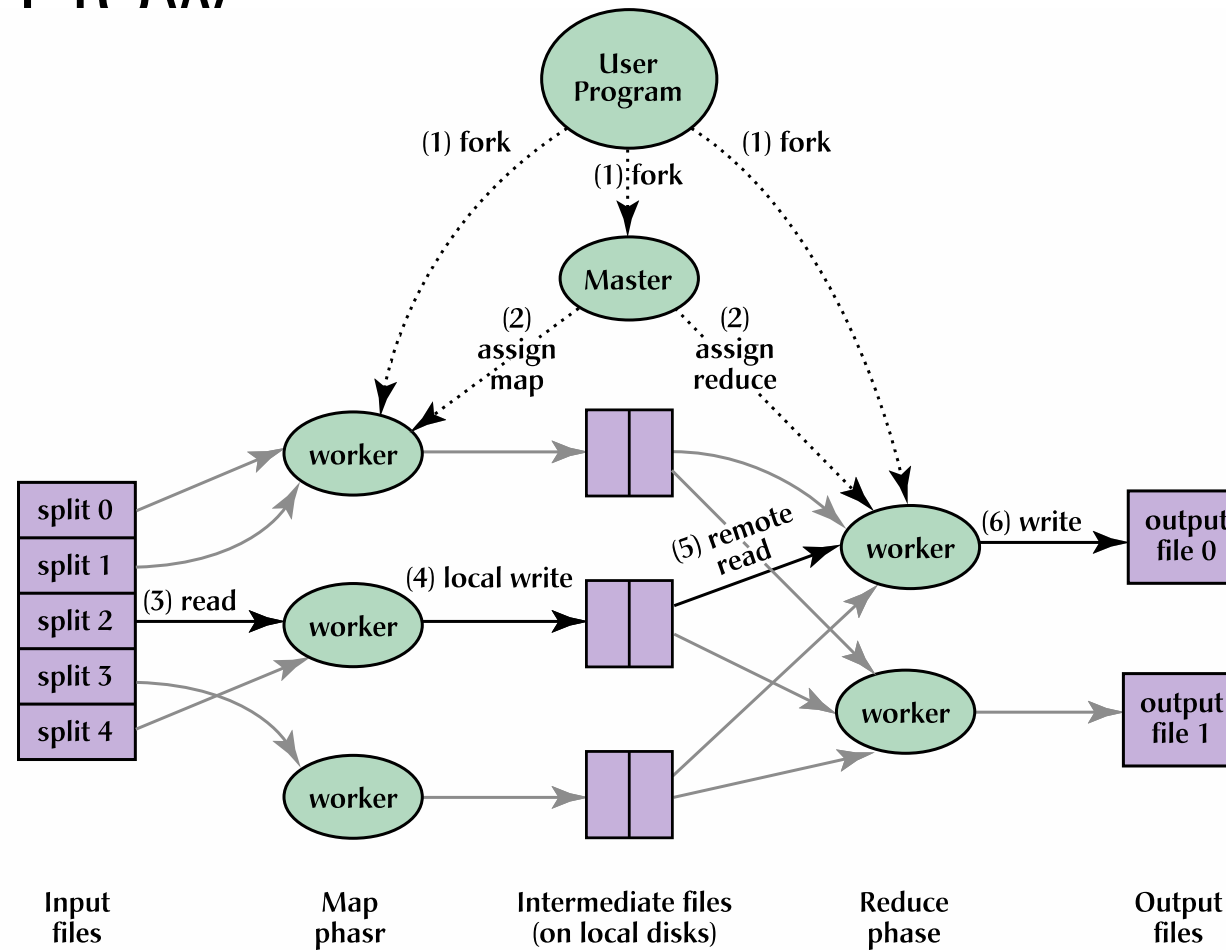
Hadoop Stack

Yarn	Third party analysis tools R (statistics), Mahout (machine learning), ...	
	Hbase	Hive & HiveQL
		Hadoop (MapReduce engine)
	Hadoop Distributed File System (HDFS)	

Master-

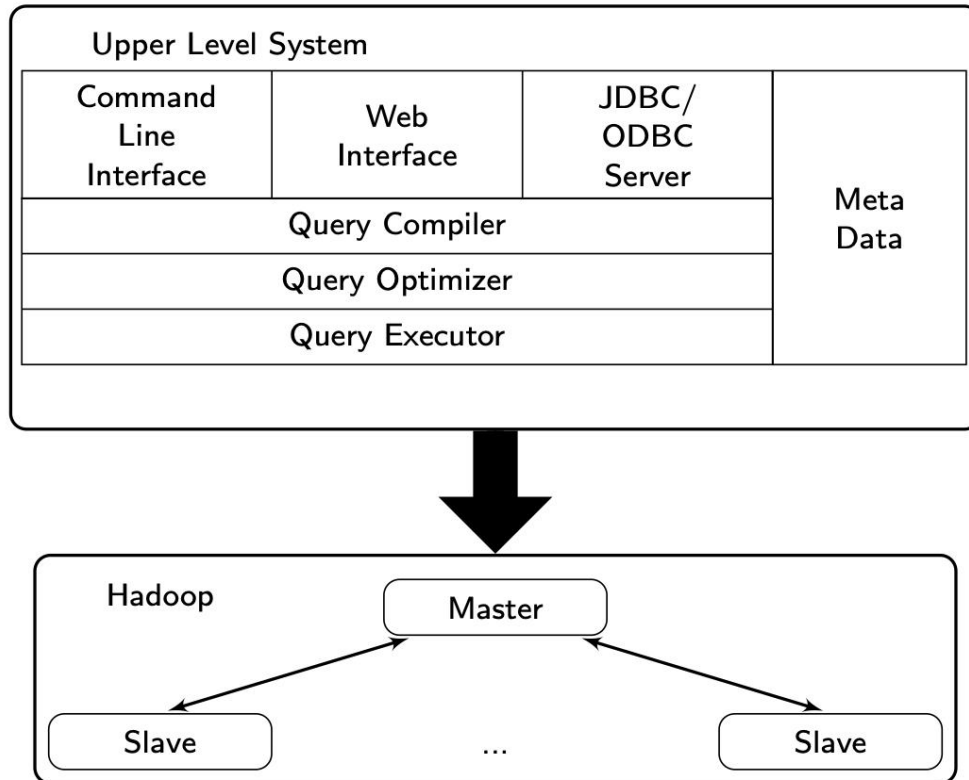


Execution Flow



From: J. Dean and S. Ghemawat. MapReduce: Simplified data processing on large clusters, *Comm. ACM*, 51(1), 2008.

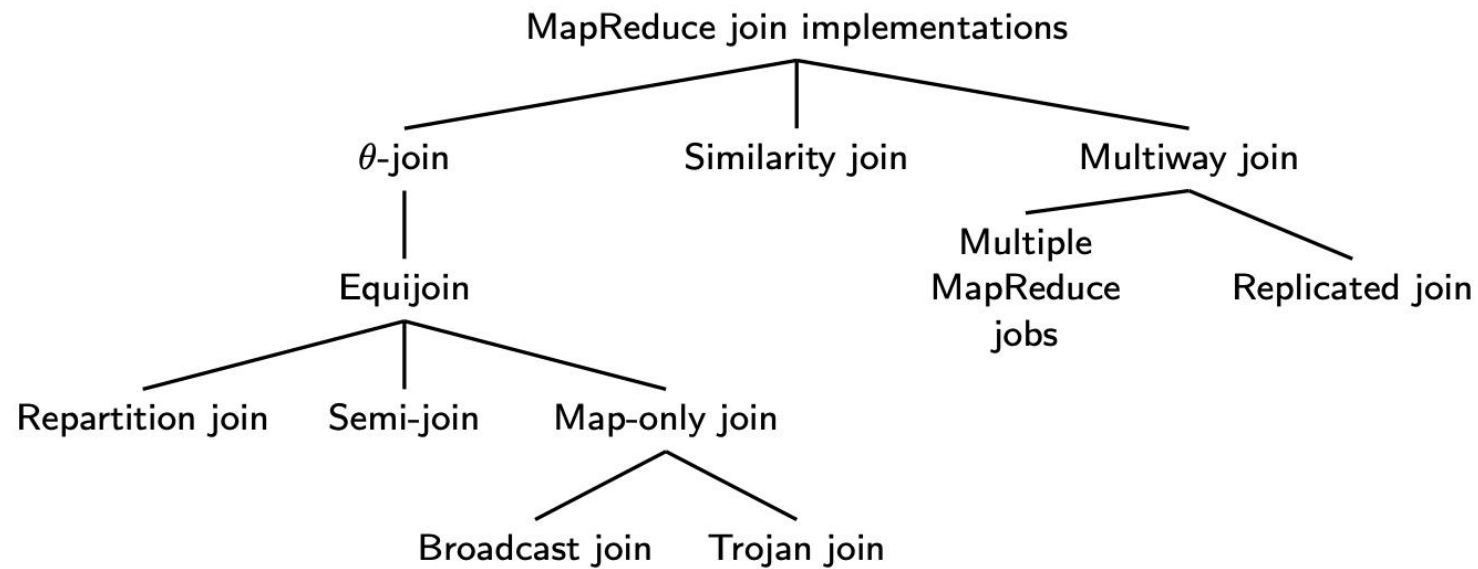
High-Level MapReduce Languages



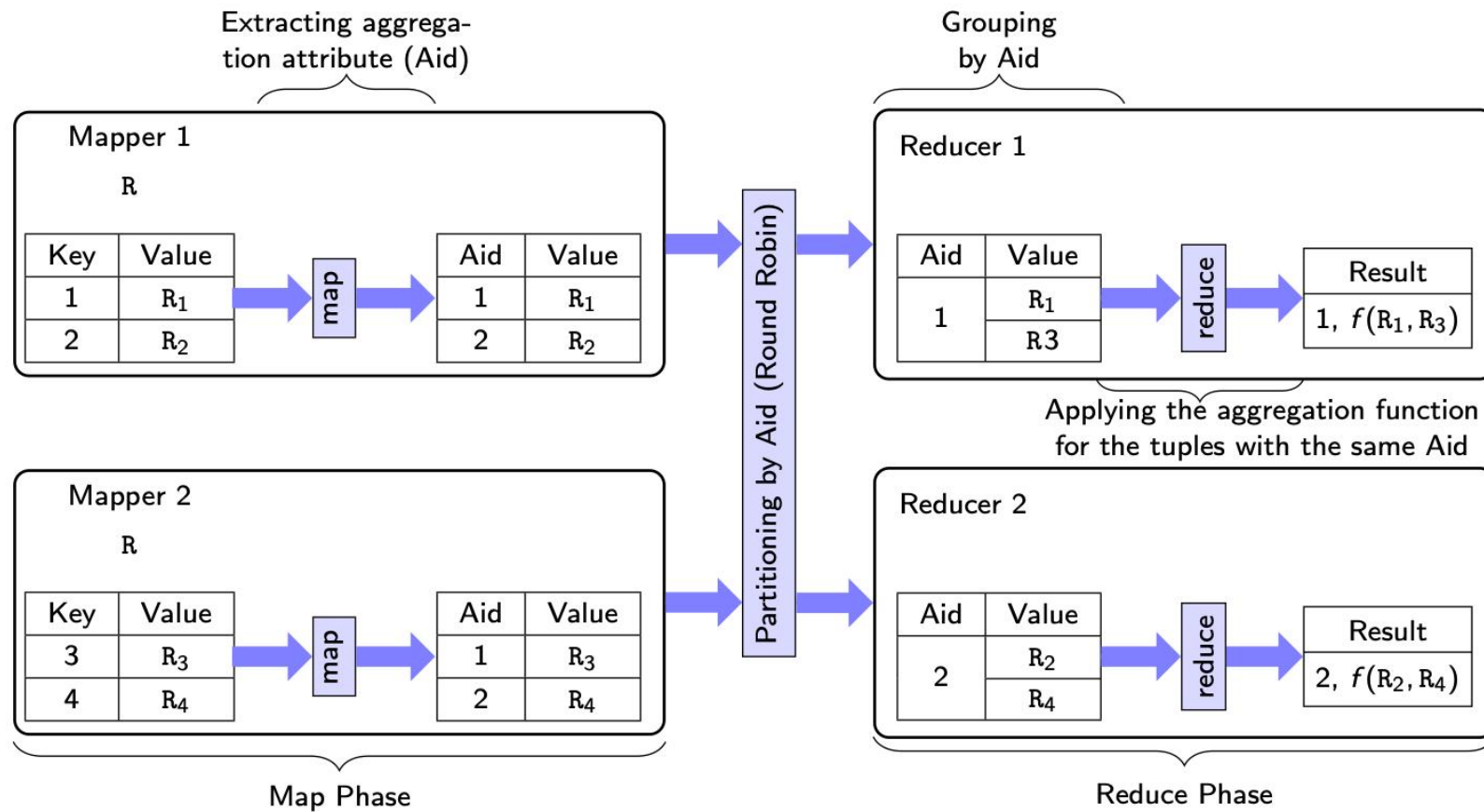
- Declarative
 - HiveQL
 - Tenzing
 - JAQL
- Data flow
 - Pig Latin
- Procedural
 - Sawzall
- Java Library
 - FlumeJava

MapReduce Implementations of DB Ops

- **Select** and **Project** can be easily implemented in the map function
- **Aggregation** is not difficult (see next slide)
- **Join** requires more work



Aggregation



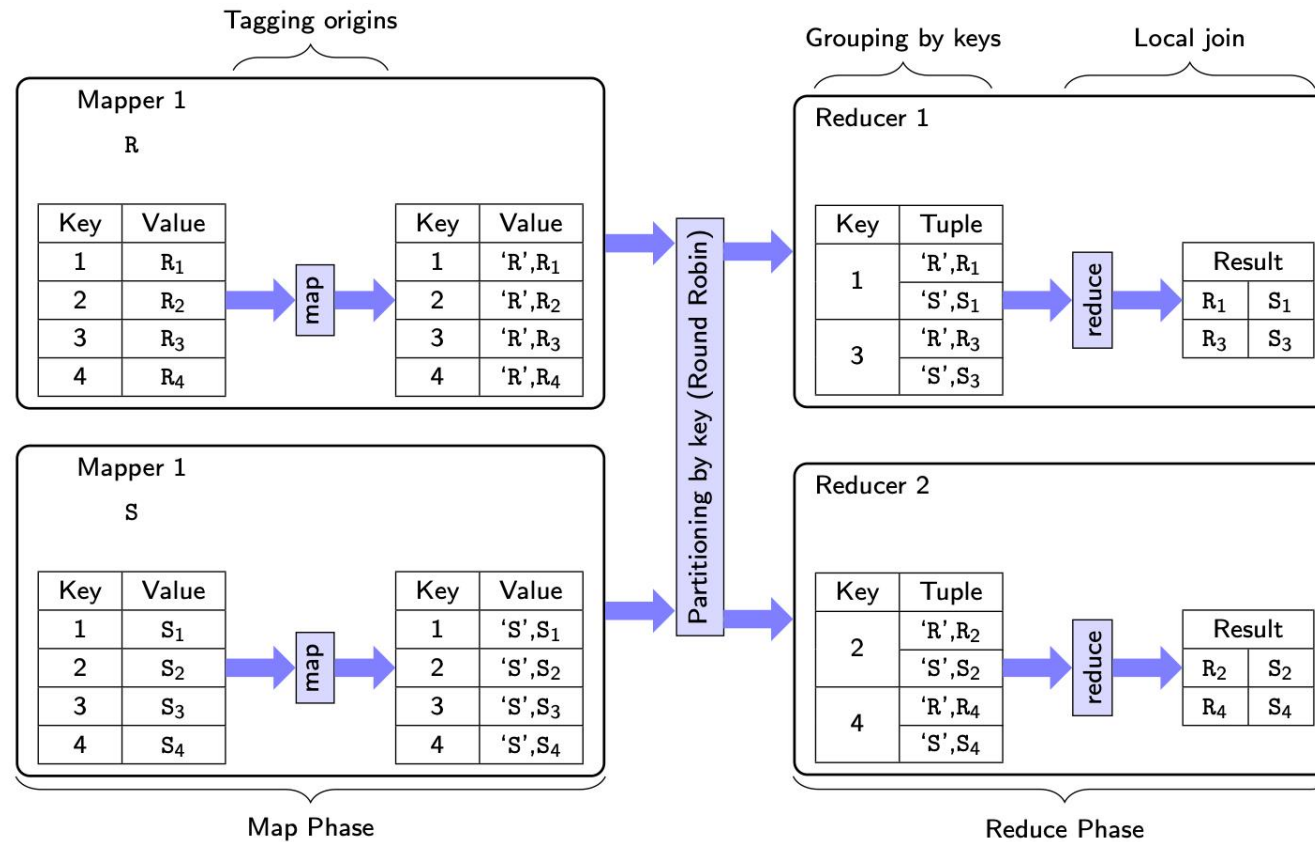
θ -Join

Baseline implementation of $R(A,B) \bowtie S(B,C)$

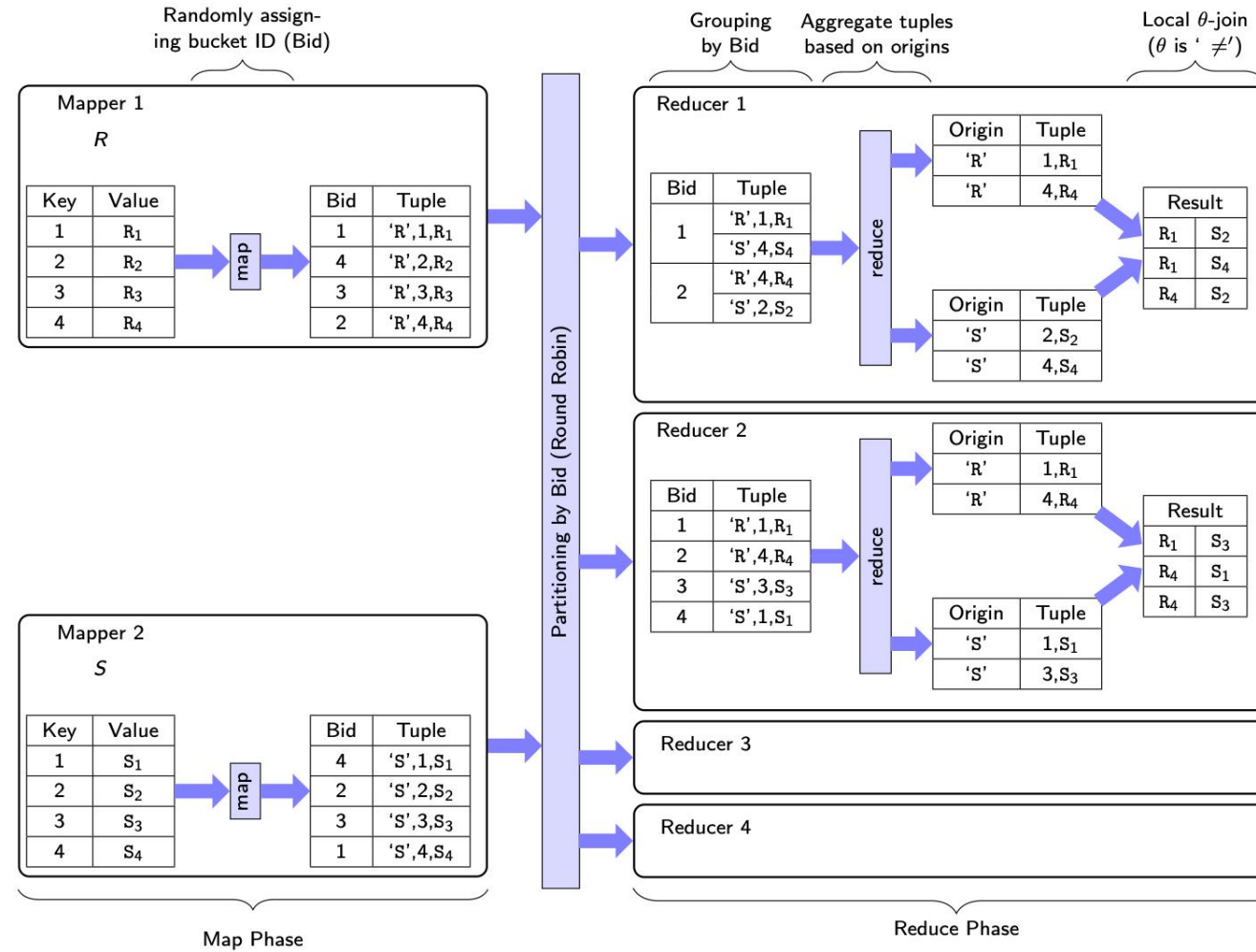
- 1) Partition R and assign each partition to mappers
- 2) Each mapper takes $\langle a, b \rangle$ tuples and converts them to a list of key-value pairs of the form $(b, \langle a, R \rangle)$
- 3) Each reducer pulls the pairs with the same key
- 4) Each reducer joins tuples of R with tuples of S

θ -Join (θ is =)

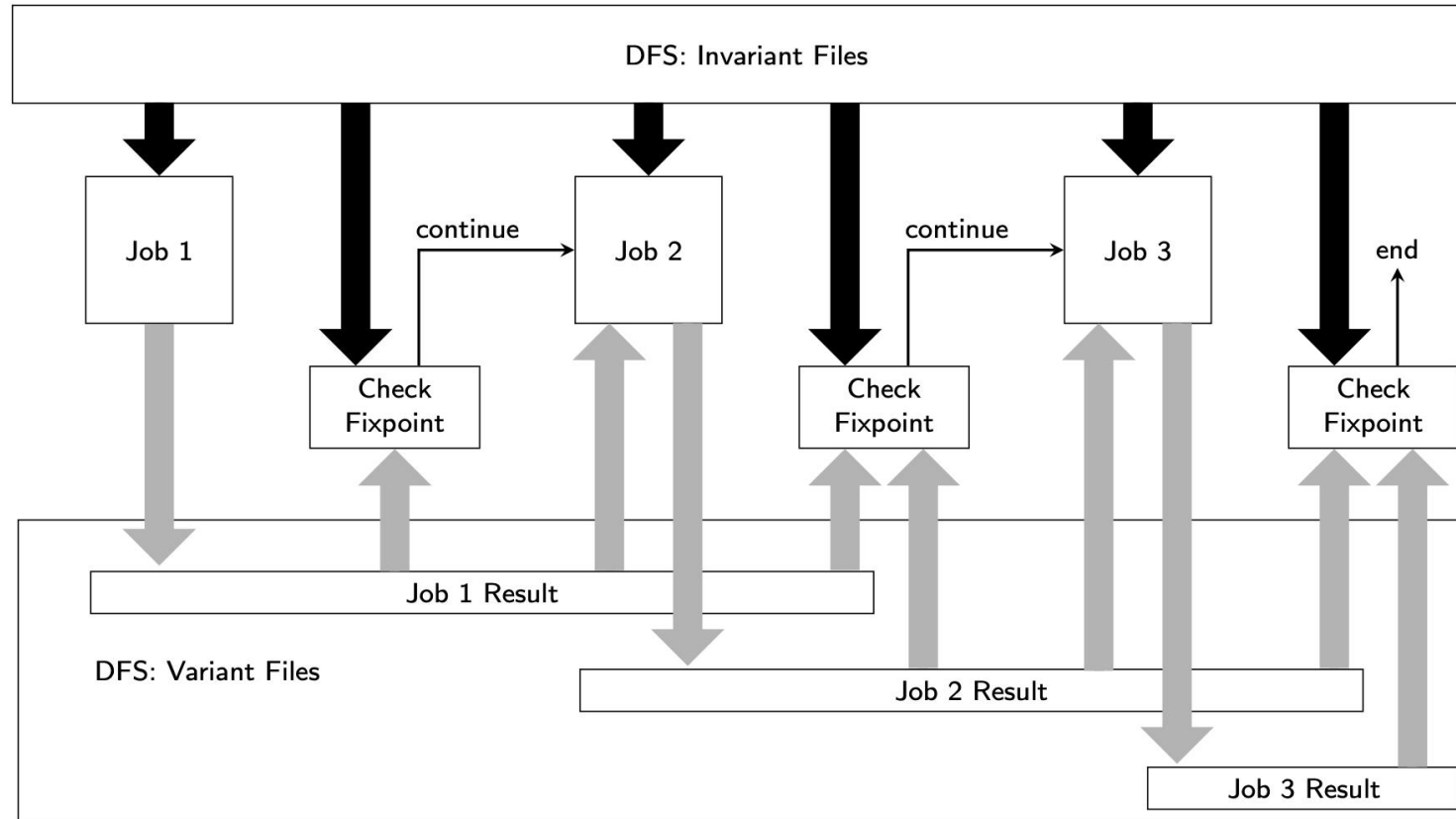
- Repartition join



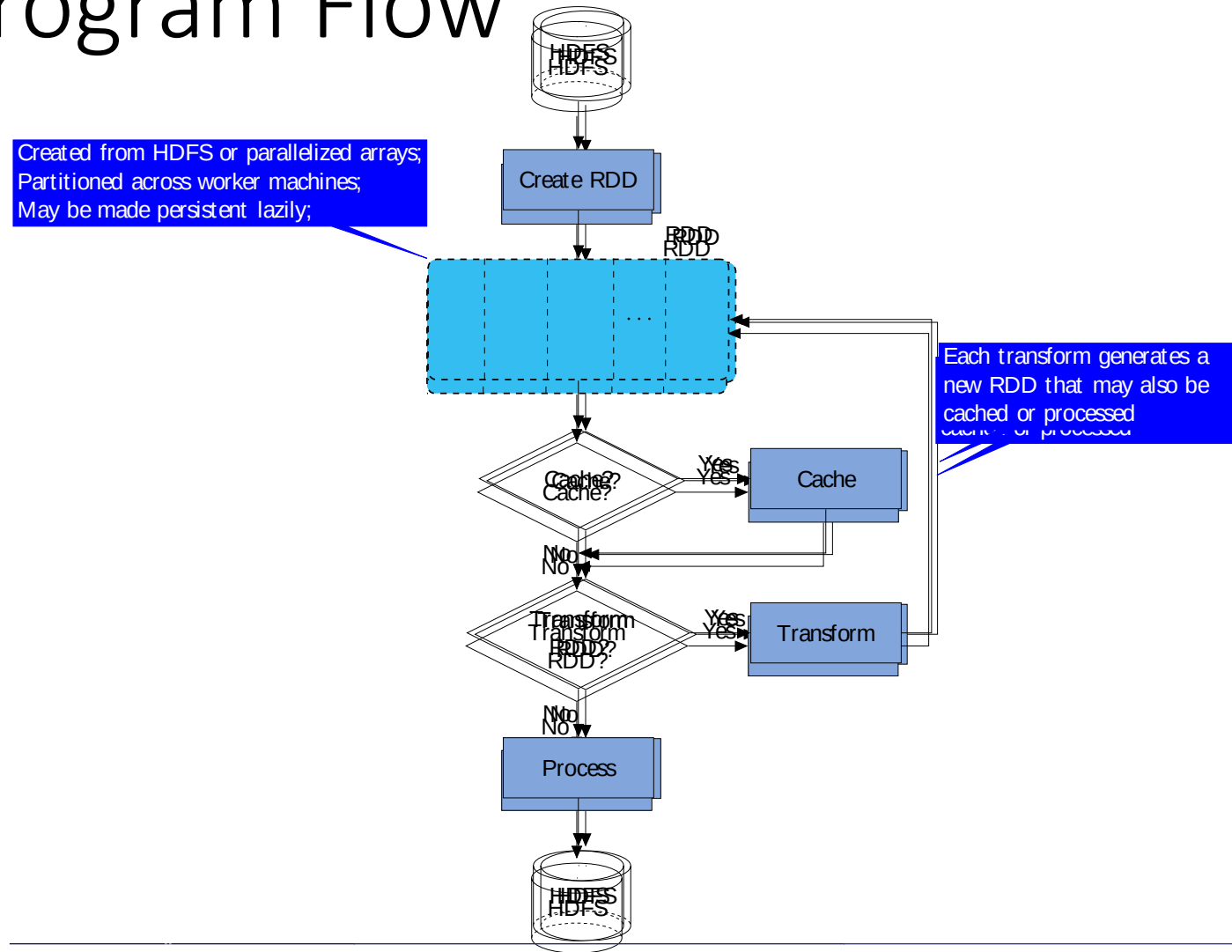
θ -Join (θ is $\neq$)



MapReduce Iterative Computation

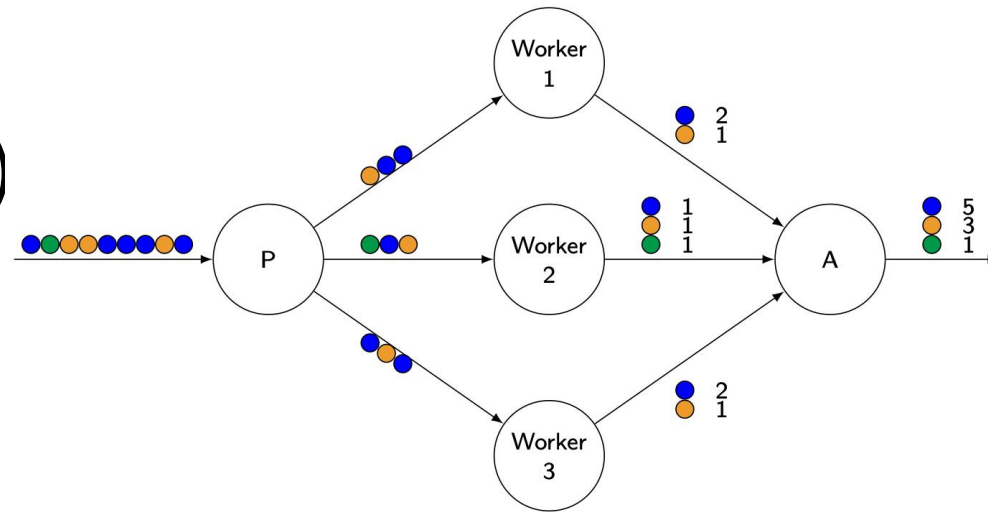


Spark Program Flow

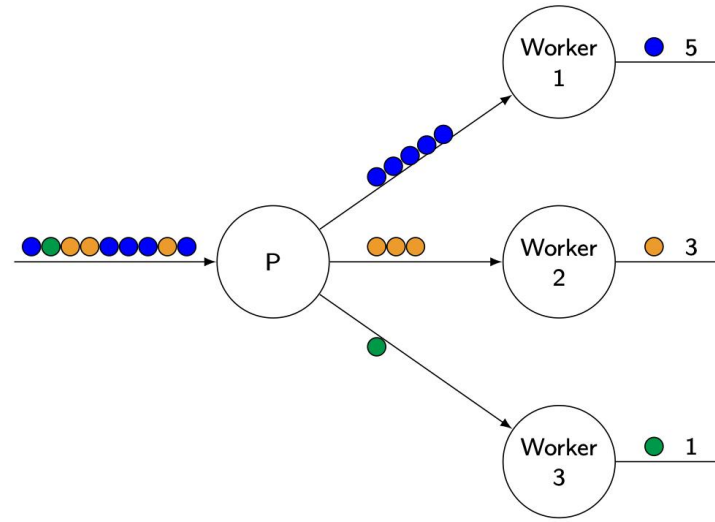


Stream Partitioning

- Shuffle (round-robin) partitioning



- Hash partitioning



Lecture 5: Load balancer Load with Go

Lecture 6: Working with Database

Lecture 6-7: Fault tolerance

Some examples of fault

- Hardware doesn't work (packages in network are lost)
- Bugs in Software

One or more workers do not work. How can the system reply to the clients with these faults?

Our aims are :

Availability: still useable despite [some class of] failures

Correctness: act just like a single server to clients and handle network failures and repair

Assuming that

- The fault are independents.
- Example: Japan Nulear power system

Google file structure with replicates

- How many copies of chunks are stored in different chunk servers?
- You have a service for a bank, you should be care about the users.

Replications can solve faults

- State machine replication:
 - replicate state (e.g., key/value store) apply operations in the same order view each replica has a state machine each replica starts in the same state applies all operations in the same order
 - > they end up in the same state

The MACHINE SENDS THE CONTENT OF RAM TO REPLICA

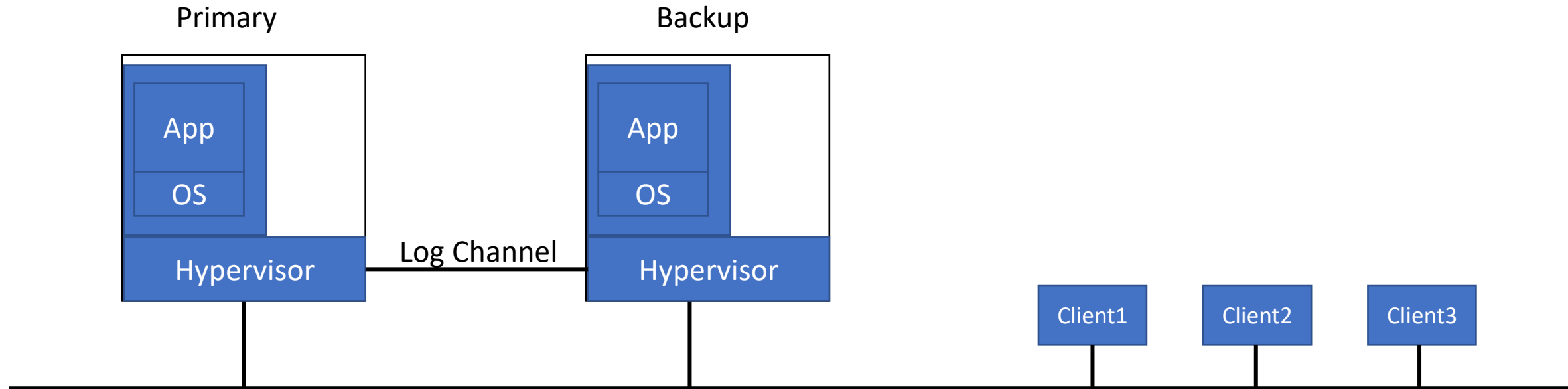
assumption: operations are deterministic

- what is operation needs the time?
- if two replicas call `gettime()` they may get different answers many variants of state machine replication

today: primary/backup approach

The MACHINE SENDS THE CLIENT INPUT TO REPLICA

Primary/Backup fault tolerance in VMware



Multiple clients, sending requests concurrently to two server roles: primary, backup which server is primary may change due to failures. Clients send operations to primary (e.g. Put, PutHash, Get). Primary chooses the order, primary forwards operations (or results) to backup, primary replies to client.

Some Problems in primary/Backup tolerance

- Non-deterministic events
 - Inputs – packets contain data + interrupt
 - Multi-core
 - Weird instructions (if two replicas call *gettime()* they may get different answers many variants of state machine replication)
- Network partition
- New backups after backup becomes primary

The Raft Consensus Algorithm

- <http://thesecretlivesofdata.com/raft/>
- We will discuss this visualization tool about RAFT protocol for fault tolerance

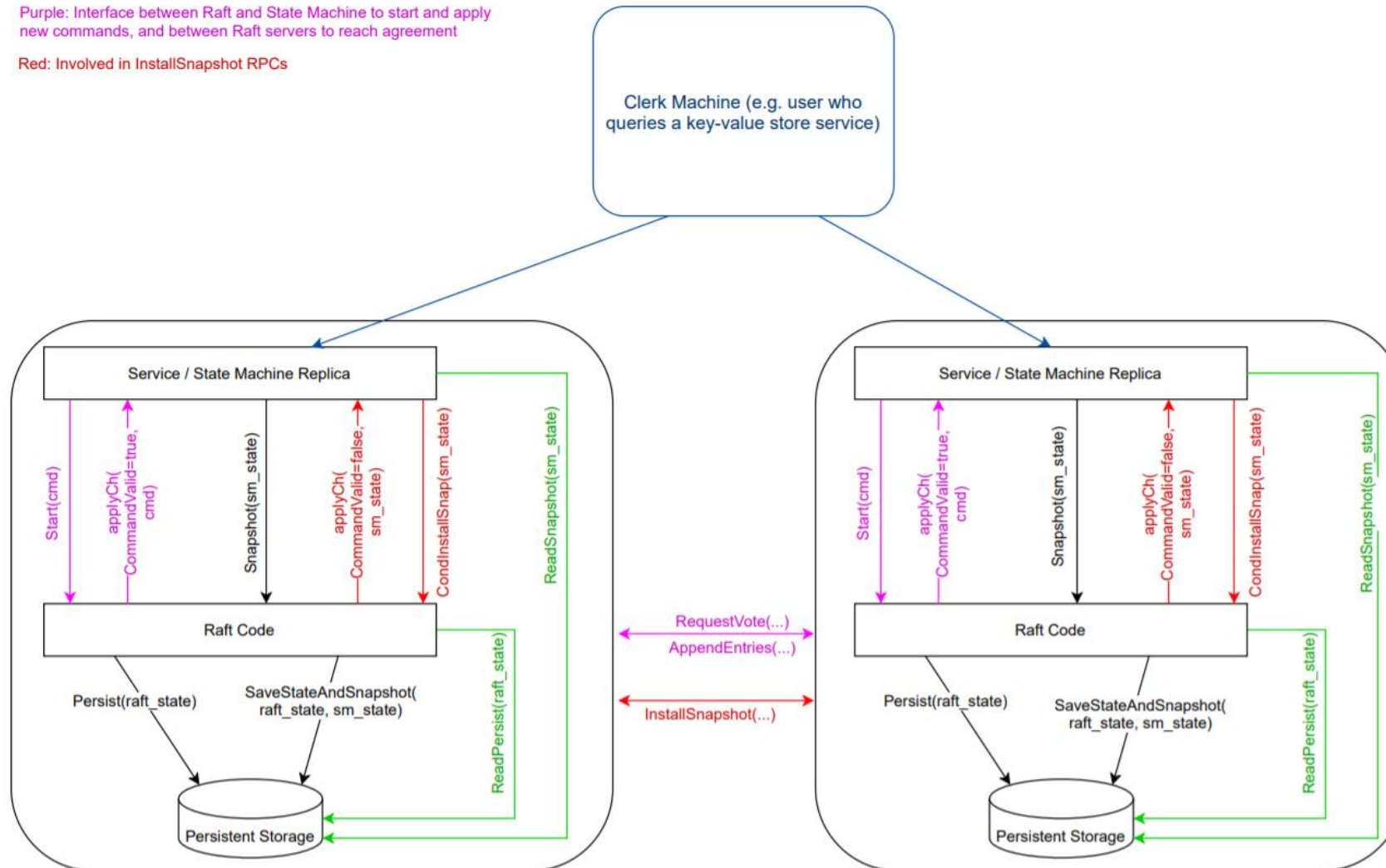
RAFT system Structure

Blue: External clients that use service, oblivious to Raft (will implement in Lab3)

Green: Involved in restoring state post-crash

Purple: Interface between Raft and State Machine to start and apply new commands, and between Raft servers to reach agreement

Red: Involved in InstallSnapshot RPCs



```

type Raft struct {
    mu          sync.Mutex          // Lock to protect shared access to this peer's state
    peers       []*labrpc.ClientEnd // RPC end points of all peers
    persister   *Persister          // Object to hold this peer's persisted state
    me          int              // this peer's index into peers[]

    // Your data here (2A, 2B, 2C).
    // Look at the paper's Figure 2 for a description of what
    // state a Raft server must maintain.

    state      int
    voteCount  int

    // Persistent state on all servers.
    currentTerm int
    votedFor    int
    log         []LogEntry

    // Volatile state on all servers.
    commitIndex int
    lastApplied int

    // Volatile state on leaders.
    nextIndex []int
    matchIndex []int

    // Channels between raft peers.
    chanApply      chan ApplyMsg
    chanGrantVote  chan bool
    chanWinElect   chan bool
    chanHeartbeat  chan bool
}

```

```

type ApplyMsg struct {
    CommandValid bool
    CommandIndex int
    Command       interface{}

    UseSnapshot bool
    Snapshot     []byte
}

```


Lecture 8: Distrubted transactions

Lecture 9-10: Blockchain Technology

E-commerce in Egypt

It is expected that by 2020 Egypt will generate \$1.9 billion in e-commerce, with 35percent annually forecasted growth rate [29].

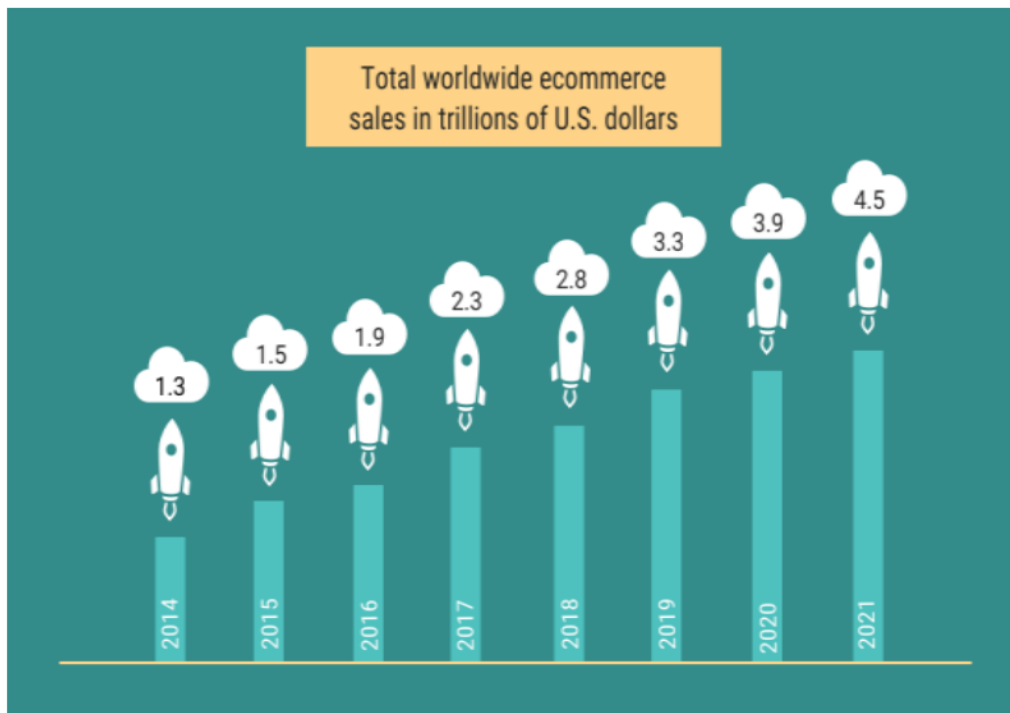


Fig.2: E-commerce revenue expectation until 2021
Source: [26]

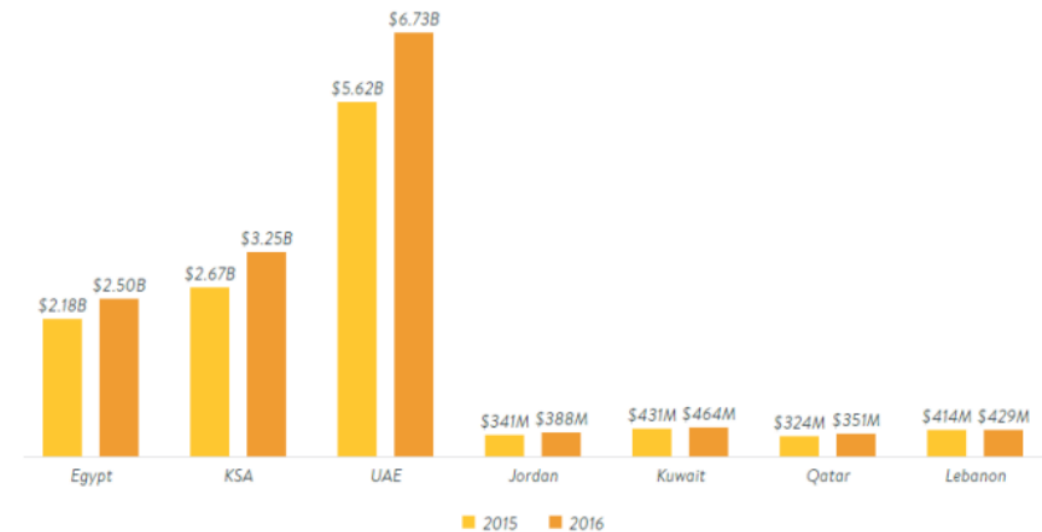


Fig.5: Middle East countries arrangement
Source: [28]

Definitions and Concepts

- Electronic commerce describes the process of buying, selling, transferring, or exchanging products, services, or information through computer networks, including the Internet.
- **Electronic business (e-business)** is a somewhat broader concept. In addition to the buying and selling of goods and services, e-business refers to servicing customers, collaborating with business partners, and performing electronic transactions within an organization.

Types of E-commerce

The three major electronic commerce categories are :

- **Business-to-consumer (B2C)** electronic commerce involves retailing products and services to individual shoppers. Amazon, Walmart, and iTunes are examples of B2C commerce. BarnesandNoble.com, which sells books, software, and music to individual consumers, is an example of B2C e-commerce.
- **Business-to-business (B2B)** electronic commerce involves sales of goods and services among businesses. Elemica's website for buying and selling chemicals and energy is an example of B2B e-commerce.
- **Consumer-to-consumer (C2C)** electronic commerce involves consumers selling directly to consumers. For example, eBay, the giant web auction site.

TABLE 10.5 INTERNET BUSINESS MODELS

CATEGORY	DESCRIPTION	EXAMPLES
E-tailer	Sells physical products directly to consumers or to individual businesses.	Amazon Blue Nile
Transaction broker	Saves users money and time by processing online sales transactions and generating a fee each time a transaction occurs.	ETrade.com Expedia
Market creator	Provides a digital environment where buyers and sellers can meet, search for products, display products, and establish prices for those products; can serve consumers or B2B e-commerce, generating revenue from transaction fees.	eBay Priceline.com Exostar Elemica
Content provider	Creates revenue by providing digital content, such as news, music, photos, or video, over the web. The customer may pay to access the content, or revenue may be generated by selling advertising space.	WSJ.com GettyImages.com iTunes.com MSN Games
Community provider	Provides an online meeting place where people with similar interests can communicate and find useful information.	Facebook Twitter
Portal	Provides initial point of entry to the web along with specialized content and other services.	Yahoo MSN AOL
Service provider	Provides applications such as photo sharing, video sharing, and user-generated content as services; provides other services such as online data storage and backup.	Google Docs Photobucket.com Dropbox

TABLE 7.1 E-Commerce Business Models

Online direct marketing	Manufacturers or retailers sell directly to customers. Very efficient for digital products and services. (www.amazon.ca)
Electronic tendering system	Businesses request quotes from suppliers. Uses B2B with a reverse auction mechanism
Name-your-own-price	Customers decide how much they are willing to pay. An intermediary tries to match a provider (www.priceline.com)
Find-the-best-price	Customers specify a need; an intermediary compares providers and shows the lowest price. Customers must accept the offer in a short time, or they may lose the deal (www.hotwire.com)
Affiliate marketing	Vendors ask partners to place logos (or banners) on partner's site. If customers click on a logo, go to a vendor's site, and make a purchase, then the vendor pays commissions to the partners
Viral marketing	Recipients of your marketing notices send information about your product to their friends
Group purchasing (e-coops)	Small buyers aggregate demand to create a large volume; the group then conducts tendering or negotiates a low price
Online auctions	Companies run auctions of various types on the Internet. Very popular in C2C, but gaining ground in other types of EC as well (www.ebay.com)
Product customization	Customers use the Internet to self-configure products or services. Sellers then price them and fulfill them quickly (<i>build-to-order</i>) (www.dell.com)
Electronic marketplaces and exchanges	Transactions are conducted efficiently (more information to buyers and sellers, lower transaction costs) in electronic marketplaces (private or public)
Bartering online	Intermediary administers online exchange of surplus products or company receives "points" for its contribution, which it can use to purchase other needed items (www.bbu.com)
Deep discounters	Company offers deep price discounts. Appeals to customers who consider only price in their purchasing decisions (www.ebay.com)
Membership	Only members can have access to the content and services (www.netflix.com)

Consumer-to-consumer (C2C): European Mobile Payment Systems Association

Provider	Country	Joined
Bancontact Payconiq Company	 Belgium	2019
Bluecode [de]	 Germany  Austria	2019
MobilePay	 Finland  Denmark	2019
SIBS/MB WAY [pt]	 Portugal	2019
Swish	 Sweden	2019
TWINT [de]	 Switzerland	2019
Vipps	 Norway	2019
Bancomat Pay	 Italy	2020 ^[4]
BLIK	 Poland	2020
PLICK	 Italy  Spain	2020 ^[5]
FLIK	 Slovenia	2020
KVIKO	 Bosnia and Herzegovina	2020
BORICA AD [bg]	 Bulgaria	2021
Currence iDEAL	 Netherlands	2021

Payment System

A Method to Amend and Record Entries on Ledgers for Money
Authorizing, Clearing and Recording Final Settlement



Personal Check
Thomas Jefferson
1809



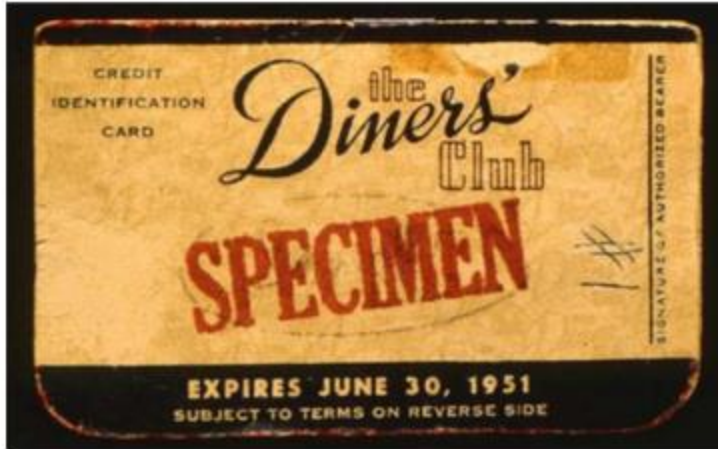
Western Union
Telegram
1873



Image by [ajmexico](#) on flickr. CC BY

Telex
1950s – 1970s

Credit Cards



First General Merchant Card
Diners' Club
1949



American Express
First Plastic Card
1959



Bank of America
First General Purpose
Credit Card
1966

Credit Card Processing



Slide Card Imprinter
1950s

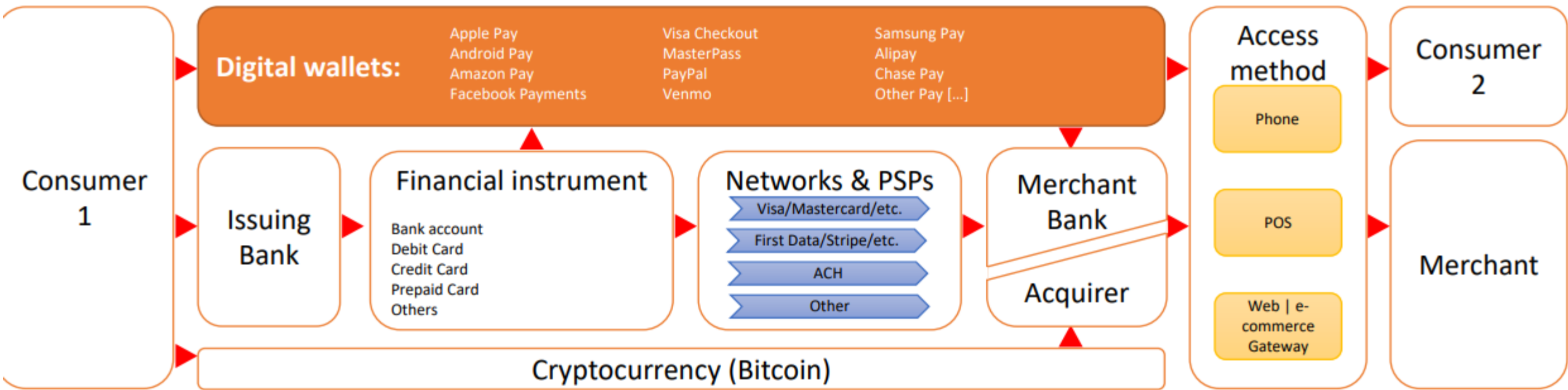


Visa Imprinter
1979

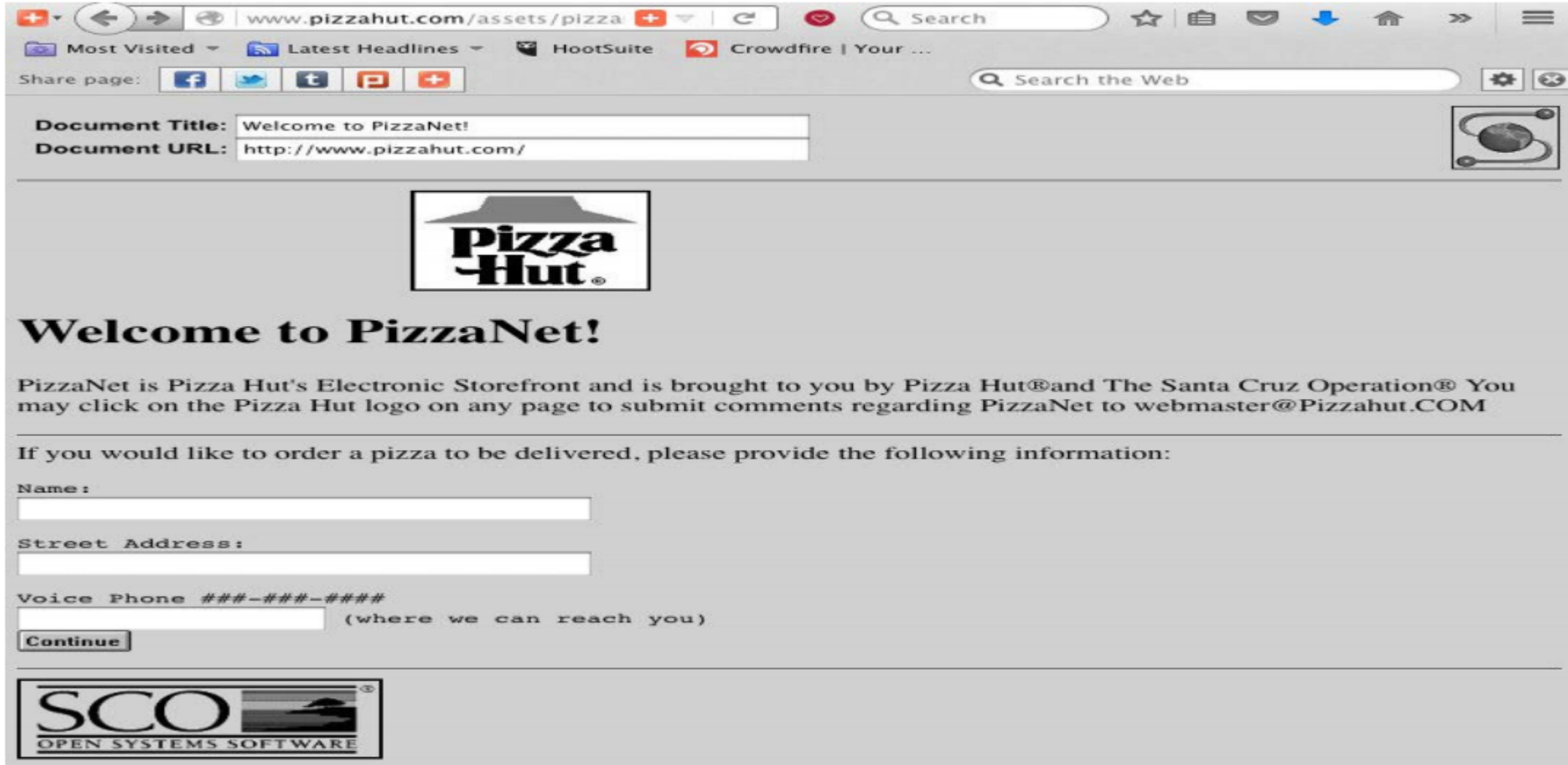


Payment Terminal
2018

Modern Payment Systems



First online sale: Pizza Hut in 1994



A screenshot of a web browser displaying the Pizza Hut website. The browser's address bar shows the URL `www.pizzahut.com/assets/pizza`. The page features the Pizza Hut logo at the top, followed by the heading "Welcome to PizzaNet!". Below this, a paragraph explains that PizzaNet is Pizza Hut's Electronic Storefront, created by Pizza Hut and The Santa Cruz Operation. It invites users to click the Pizza Hut logo to submit comments to `webmaster@Pizzahut.COM`. A section titled "If you would like to order a pizza to be delivered, please provide the following information:" contains three input fields for "Name:", "Street Address:", and "Voice Phone ###-###-####". A "Continue" button is located below the phone number field. At the bottom left, there is a logo for "SCO OPEN SYSTEMS SOFTWARE". The browser interface includes a search bar, social media sharing buttons, and a "Search the Web" field.

Document Title: Welcome to PizzaNet!
Document URL: http://www.pizzahut.com/

Pizza Hut

Welcome to PizzaNet!

PizzaNet is Pizza Hut's Electronic Storefront and is brought to you by Pizza Hut® and The Santa Cruz Operation®. You may click on the Pizza Hut logo on any page to submit comments regarding PizzaNet to `webmaster@Pizzahut.COM`.

If you would like to order a pizza to be delivered, please provide the following information:

Name:

Street Address:

Voice Phone ###-###-#### (where we can reach you)

SCO OPEN SYSTEMS SOFTWARE

But money changed hands with delivery

Early Cryptographic Digital Currencies Failed

- DigiCash (David Chaum) – 1989
- Mondex (National Westminster Bank) - 1993
- CyberCash (Lynch, Melton, Crocker & Wilson) – 1994

The Internet: Cryptographic protocols

SSL / TLS – 1996



HTTP- 1990



TCP/IP - 1974



Ethernet - 1974



Electronic Payment Mechanisms

1. When you purchase a book from Amazon, for example, your credit card information and purchase amount are encrypted in your browser. This procedure ensures the information is safe while it is “travelling” on the Internet to Amazon.
2. When your information arrives at Amazon, it is not opened. Rather, it is transferred automatically (in encrypted form) to a *clearinghouse*, where it is decrypted for verification and authorization.
3. The clearinghouse asks the bank that issued you your credit card (the card issuer bank) to verify your credit card information.
4. Your card issuer bank verifies your credit card information and reports this to the clearinghouse.
5. The clearinghouse reports the result of the verification of your credit card to Amazon.
6. Amazon reports a successful purchase and amount to you.
7. Your card issuer bank sends funds in the amount of the purchase to Amazon’s bank.
8. Your card issuer bank notifies you (either electronically or in your monthly statement) of the debit on your credit card.
9. Amazon’s bank notifies Amazon of the funds credited to its account.

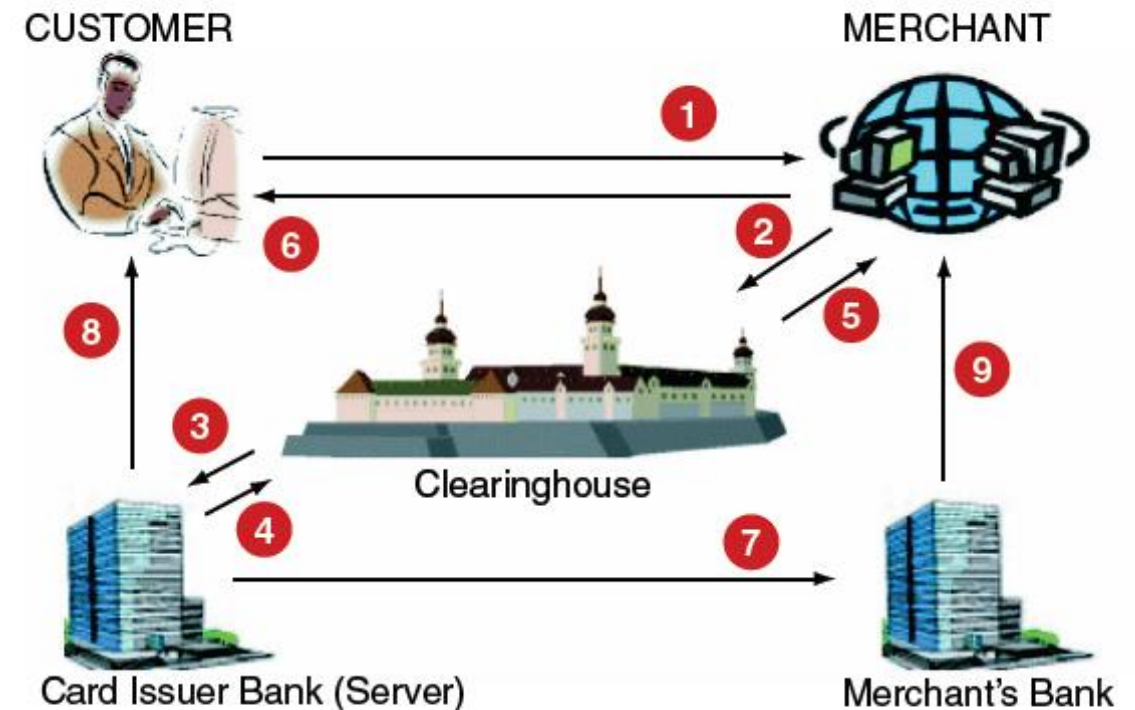


FIGURE 7.1 How e-credit cards work. (The numbers 1–9 indicate the sequence of activities.)

Further Cryptographic Digital Currencies Failed

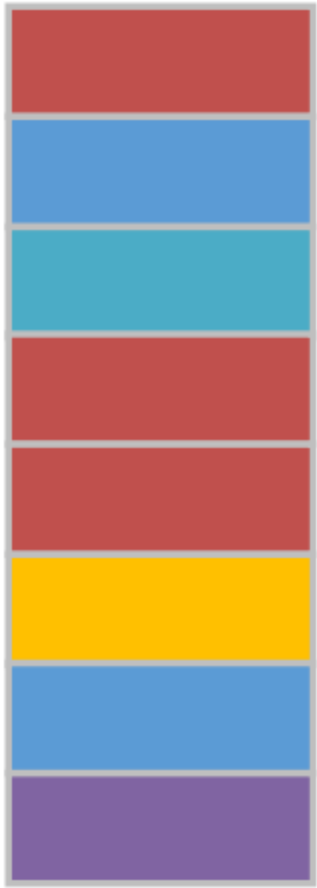
- DigiCash (David Chaum) – 1989
- Mondex (National Westminster Bank) - 1993
- CyberCash (Lynch, Melton, Crocker & Wilson) – 1994
- E-gold (Gold & Silver Reserve) – 1996
- Hashcash (Adam Back) – 1997
- Bit Gold (Nick Szabo) – 1998
- B-Money (Wei Dai) - 1998
- Lucre (Ben Laurie) – 1999

Blockchain Technology

- Verifiably moves 'data' on a decentralized network
- The 'data' can represent value or computer code
- Thus it goes directly to the plumbing of the financial sector and money
- Broad adoption rests on addressing technical, commercial and public policy hurdles
- It can be a catalyst for change in the world of finance and money

What is a blockchain?

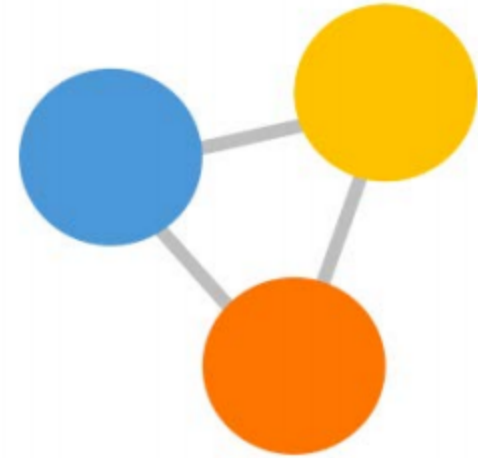
timestamped
append-only log



auditable database



consensus protocol



Secured via cryptography

- Hash functions for **tamper resistance** and **integrity**
- Digital signatures for **consent**
- Consensus for **agreement**

Addresses '**cost of trust**'
(Byzantine Generals problem)

- Permissioned
- Permissionless

Cryptography:

Communications in the presence of adversaries

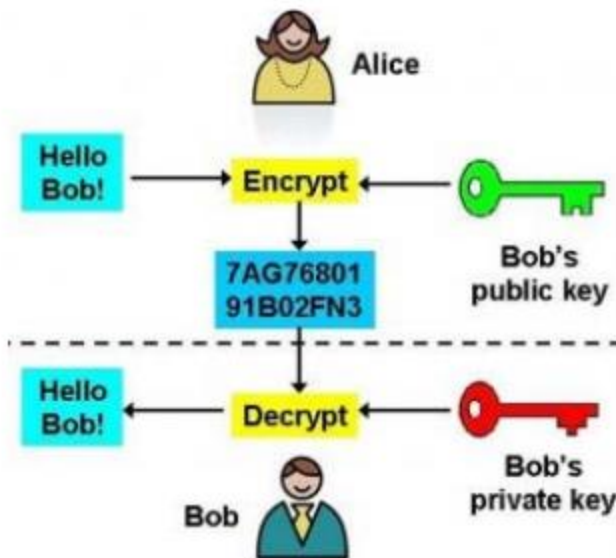


Scytale Cipher Ancient Times



Enigma Machine

1920s - WWII



Asymmetric Cryptography

1976 to today

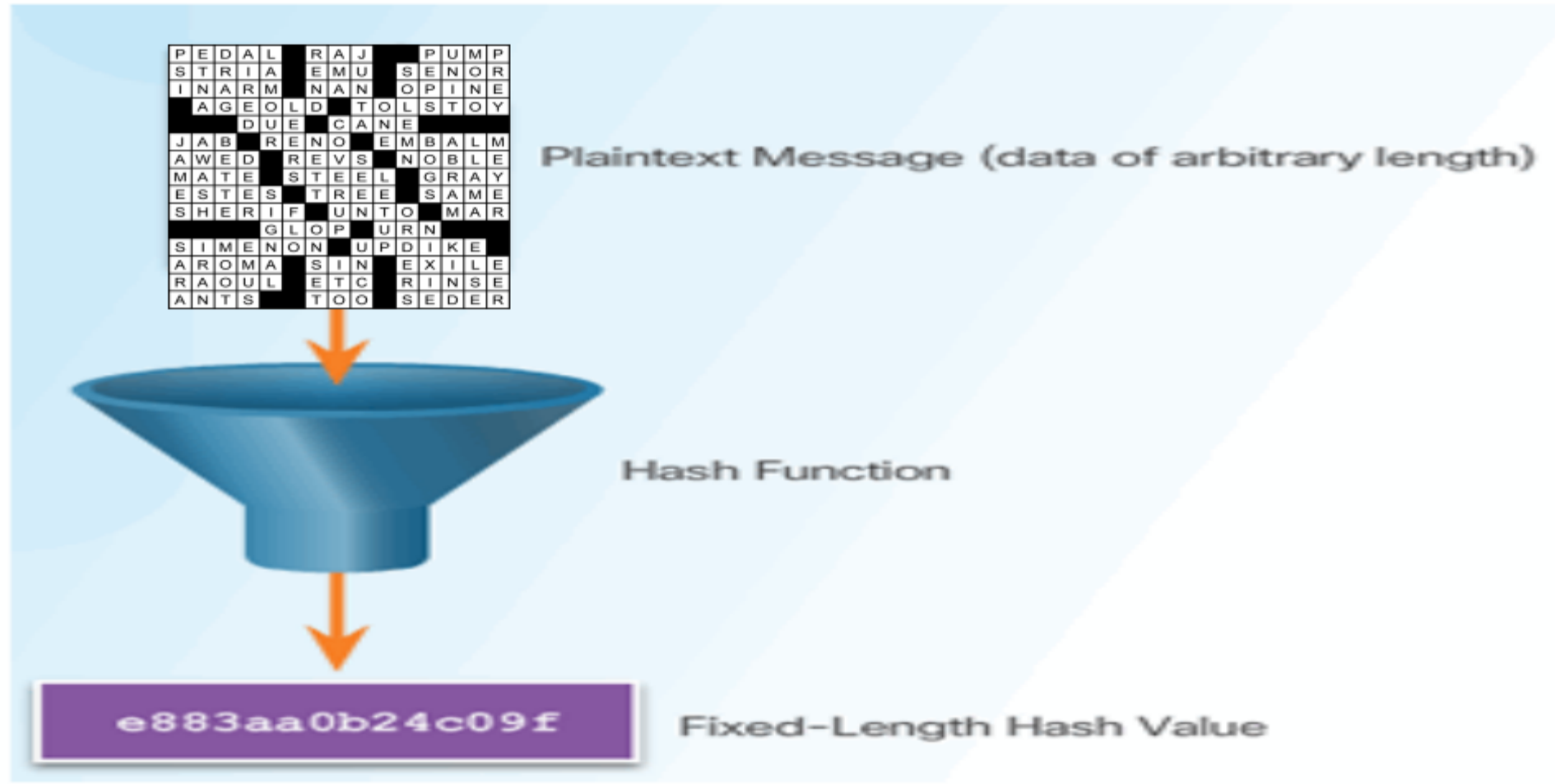
© Luringen on Wikimedia Commons. License CC BY-SA. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>

Image by the CIA and is in the public domain via Wikimedia Commons.

Image is in the public domain via Wikipedia.

Cryptographic Hash Functions

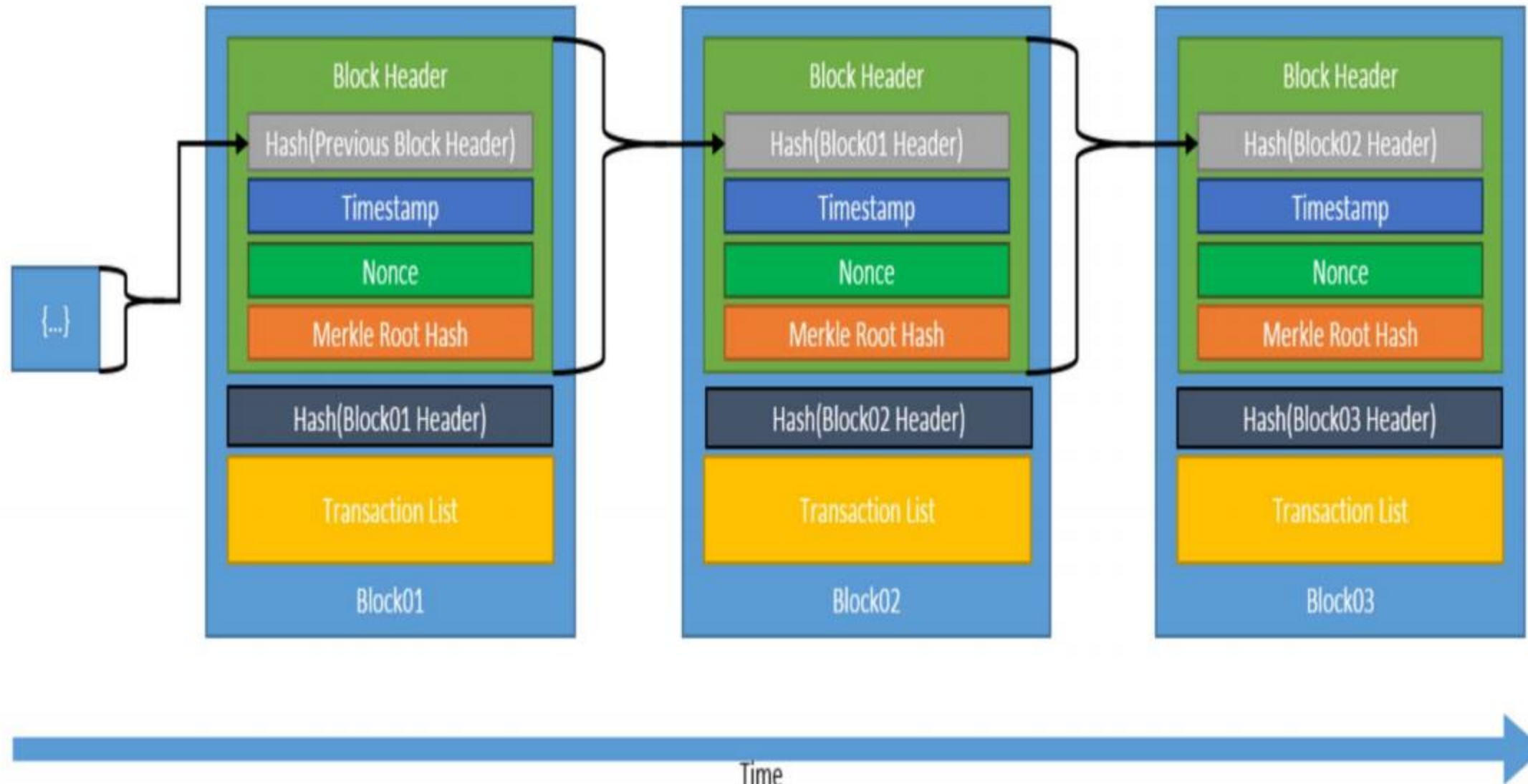
One-Way Data Compression



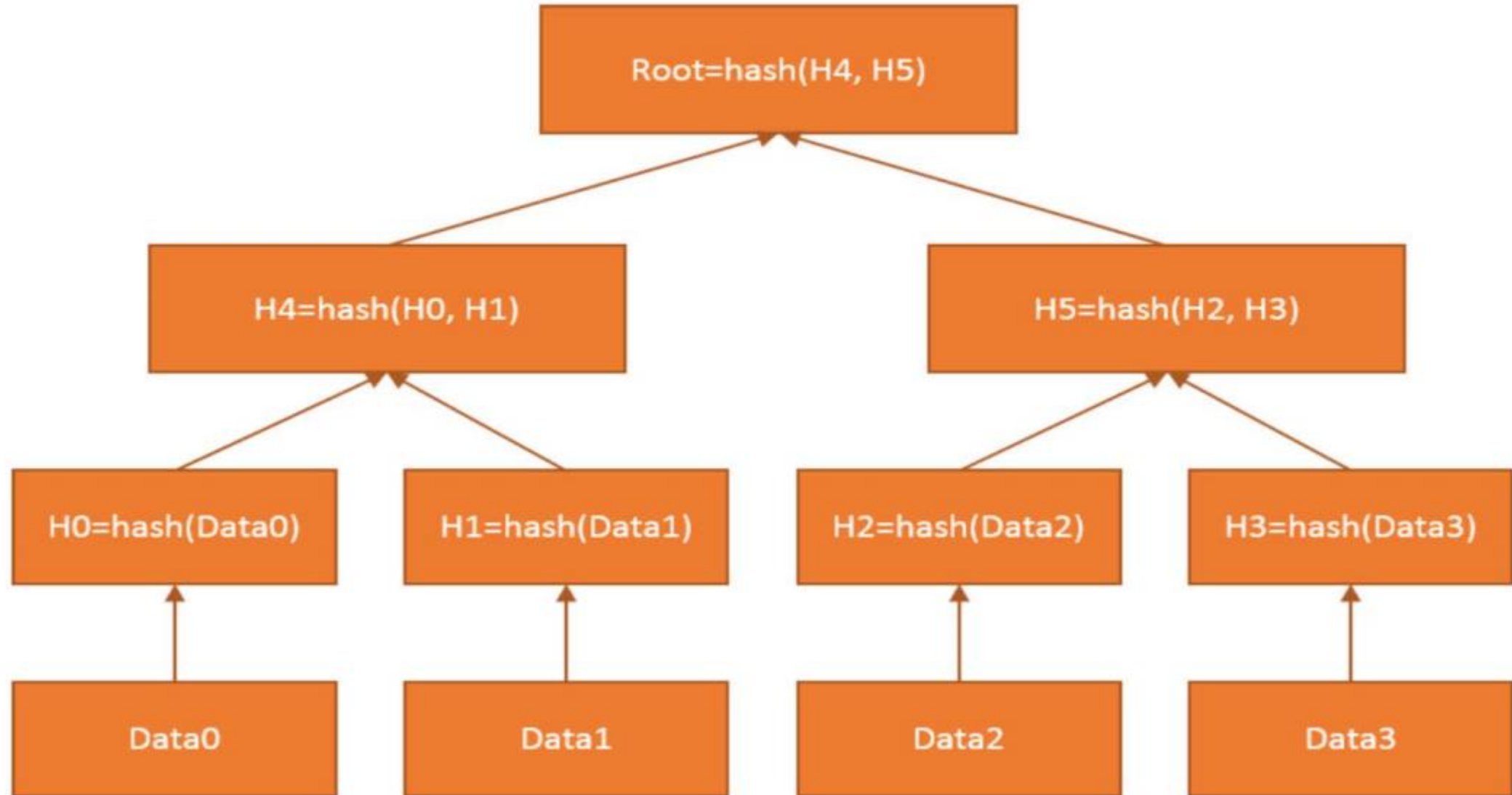
Data Commitment

© Source unknown. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>

Timestamped Append-only Log - Blockchain



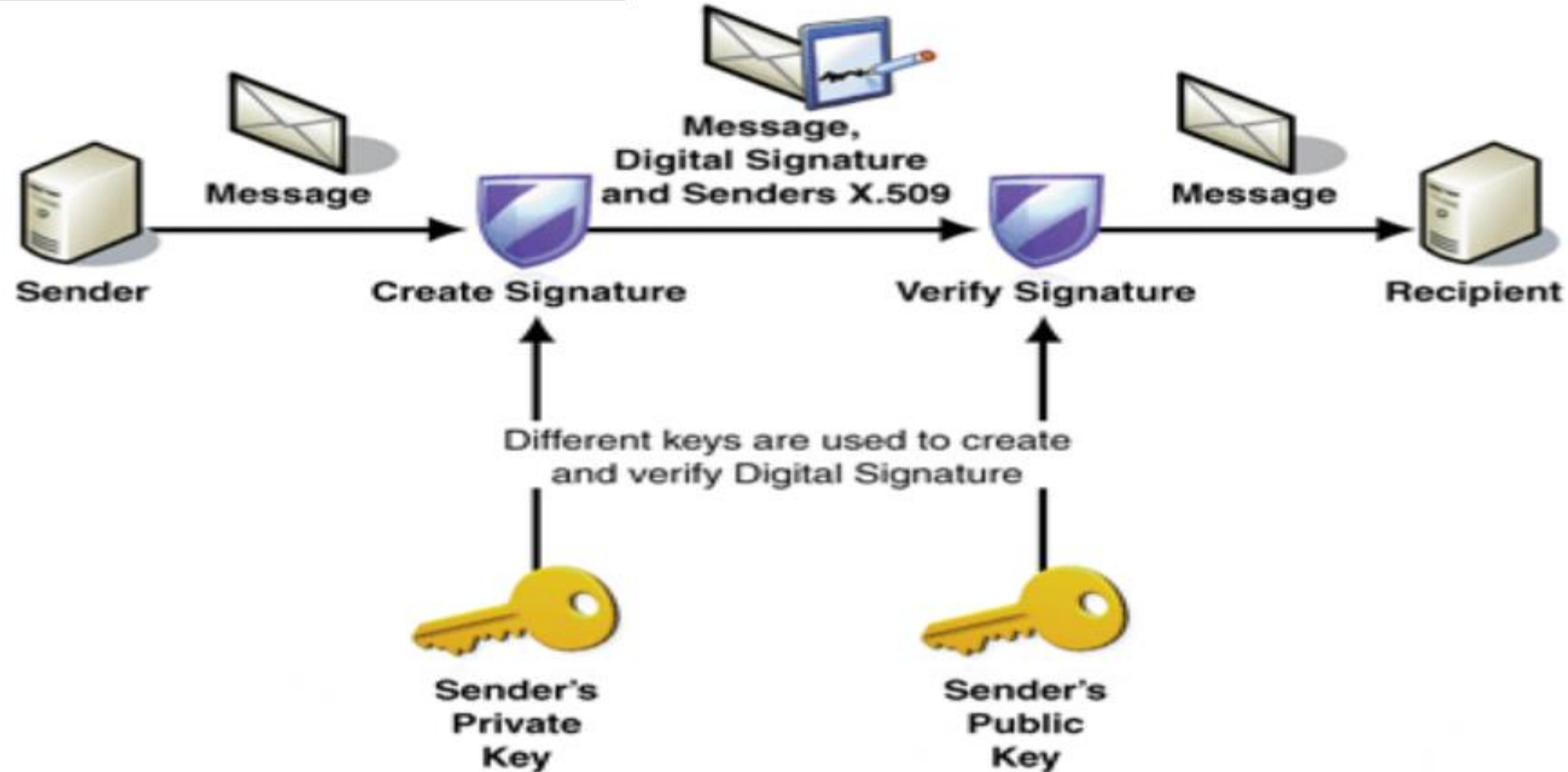
Merkle Tree – Binary Data Tree with Hashes



Asymmetric Cryptography & Digital Signatures

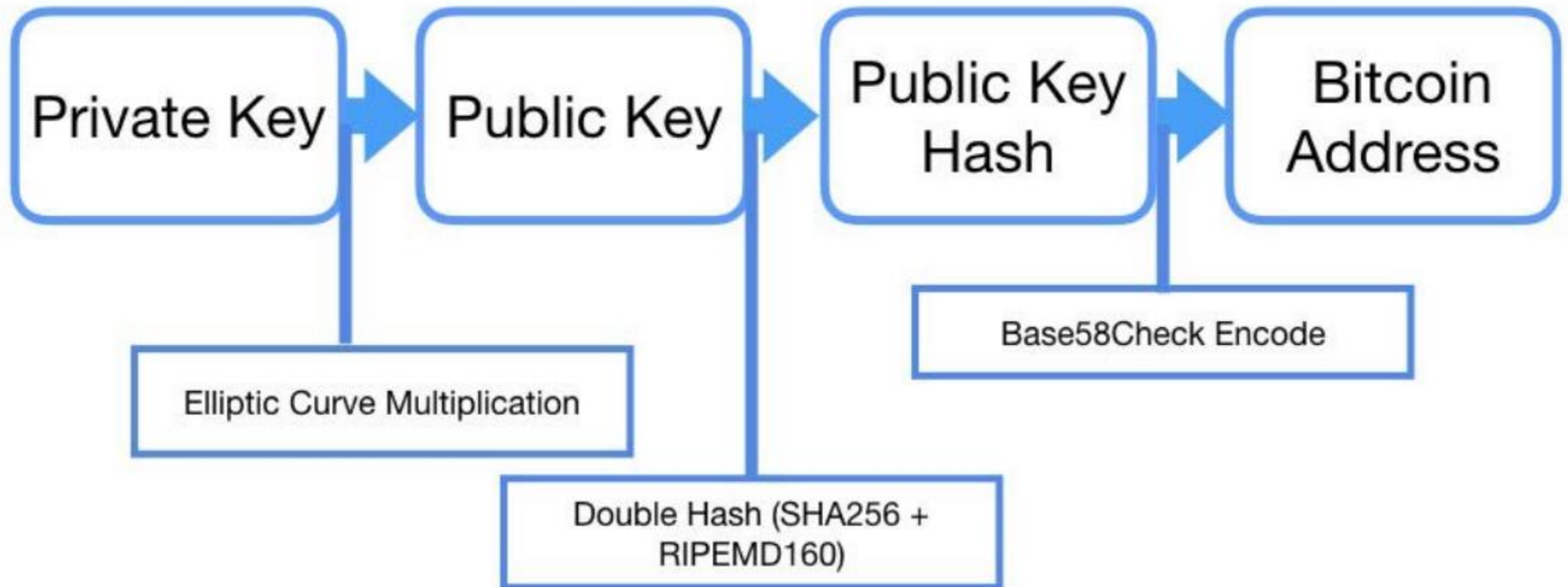
Guarding against Tampering & Impersonation

Digital Signature without Hash



Bitcoin Address

Determined by – but not identical to - Public Key

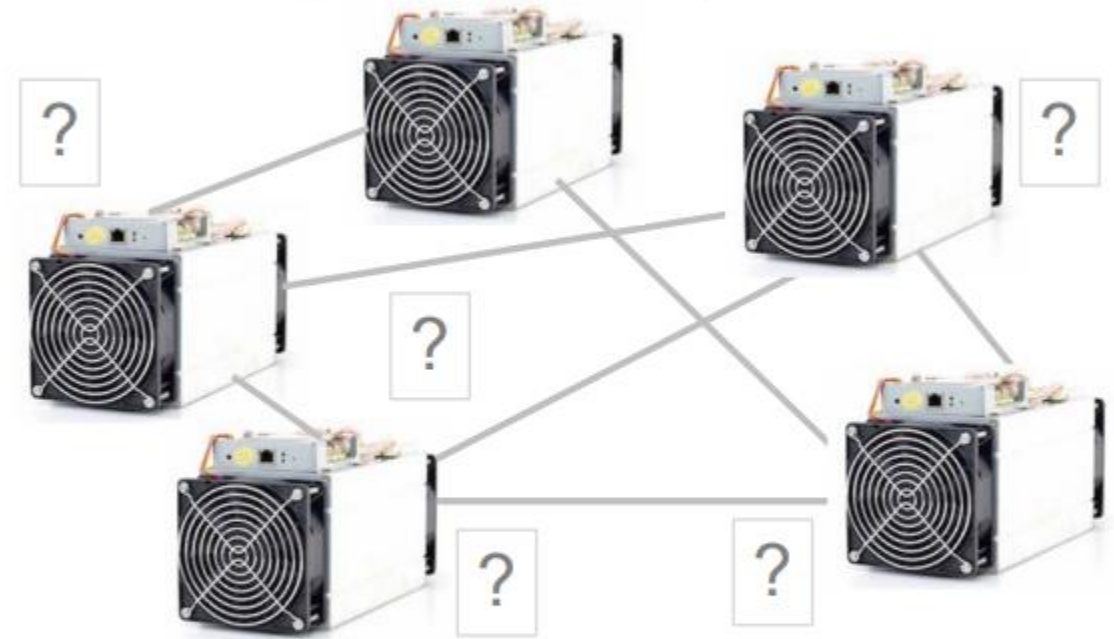


Decentralized Networks

Byzantine Generals Problem



Permissionless Blockchains - Unknown participants



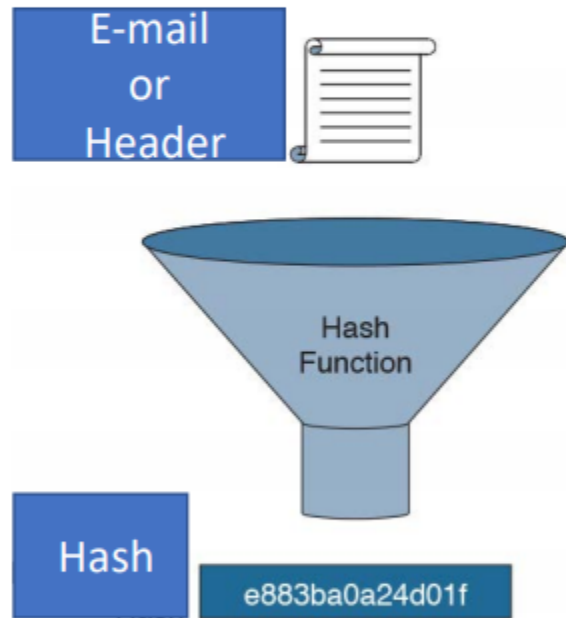
Security based on:

- Consensus protocol &
- Native currency

Hashcash – Proof of Work (Adam Back, 1997)

Proposed to address E-mail Spam and Denial of Service attacks

- Requires computational work to find a hash within predetermined range



© Source unknown. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>

- Difficulty defined by Hash outputs' # of leading zeros
- Proof of Work can be Efficiently Verified¹⁵

Blockchain – Proof of Work

Innovation – Chained Proof of Work for Distributed Network Consensus & Timestamping

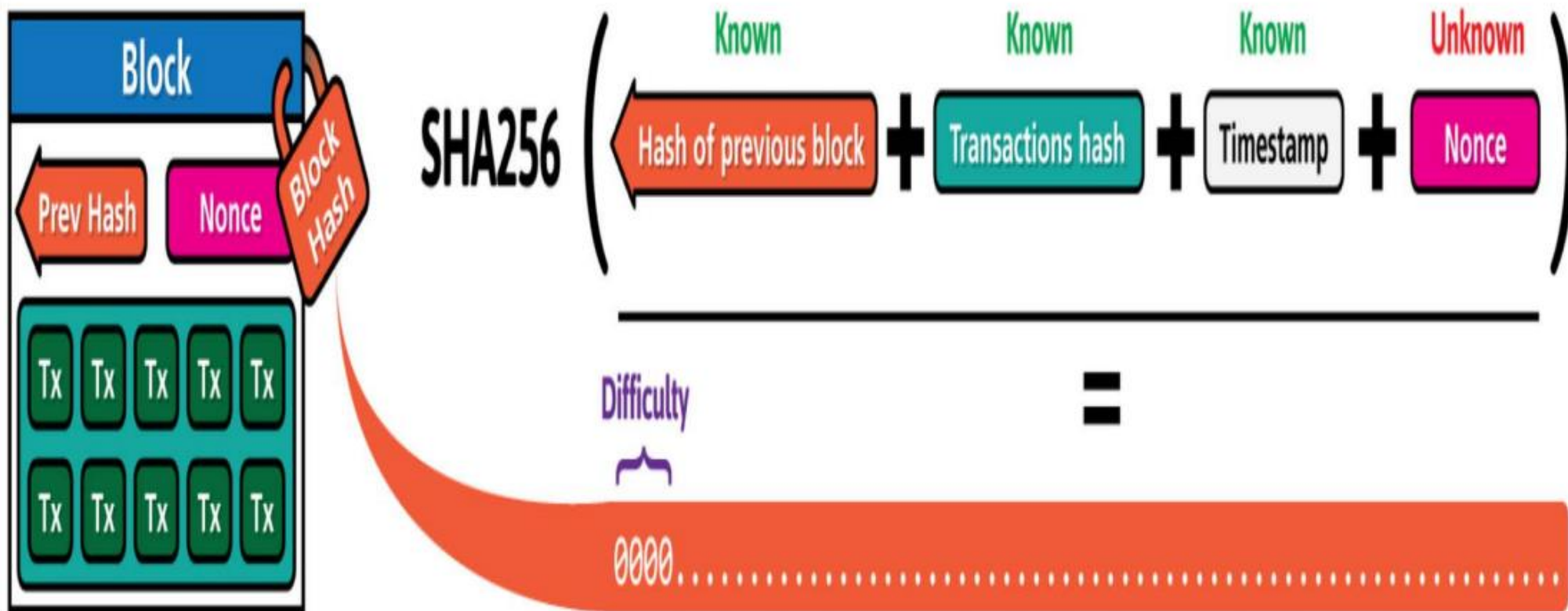


Illustration by CryptoGraphics.info

Blockchain – Proof of Work

Block: # 3

Nonce: 933

Coinbase: \$ 100.0 -> Ander

Tx:

\$		From:		->	
\$		From:		->	
\$		From:		->	

Prev: 0000a5a24dd8f977c06df9:

Hash: 0000053903659cdf61b0721

Mine

Block: # 4

Nonce: 35558

Coinbase: \$ 100.0 -> Ander

Tx:

\$		From:		->	
\$		From:		->	
\$		From:		->	

Prev: 0000053903659cdf61b0721

Hash: 0000e5196a011b80e7c79d

Mine

Block: # 5

Nonce: 11396

Coinbase: \$ 100.0 ->

Tx:

\$		From:	
\$		From:	
\$		From:	
\$		From:	

Prev: 0000e5196a011b80e7c79d

Hash: 0000c288488f4295:

Mine

Blockchain – Proof of Work

Block: # 3

Nonce: 933

Coinbase: \$ 100.0 -> Ander

Tx:

\$		From:		->	
\$		From:		->	
\$		From:		->	

Prev: 0000a5a24dd8f977c06df9:

Hash: 0000053903659cdf61b072i

Mine

Block: # 4

Nonce: 35558

Coinbase: \$ 100.0 -> Gary

Tx:

\$		From:		->	
\$		From:		->	
\$		From:		->	

Prev: 0000053903659cdf61b072i

Hash: f41546725027895cb31bd8'

Mine

Block: # 5

Nonce: 11396

Coinbase: \$ 100.0 ->

Tx:

\$		From:	
\$		From:	
\$		From:	
\$		From:	

Prev: f41546725027895c

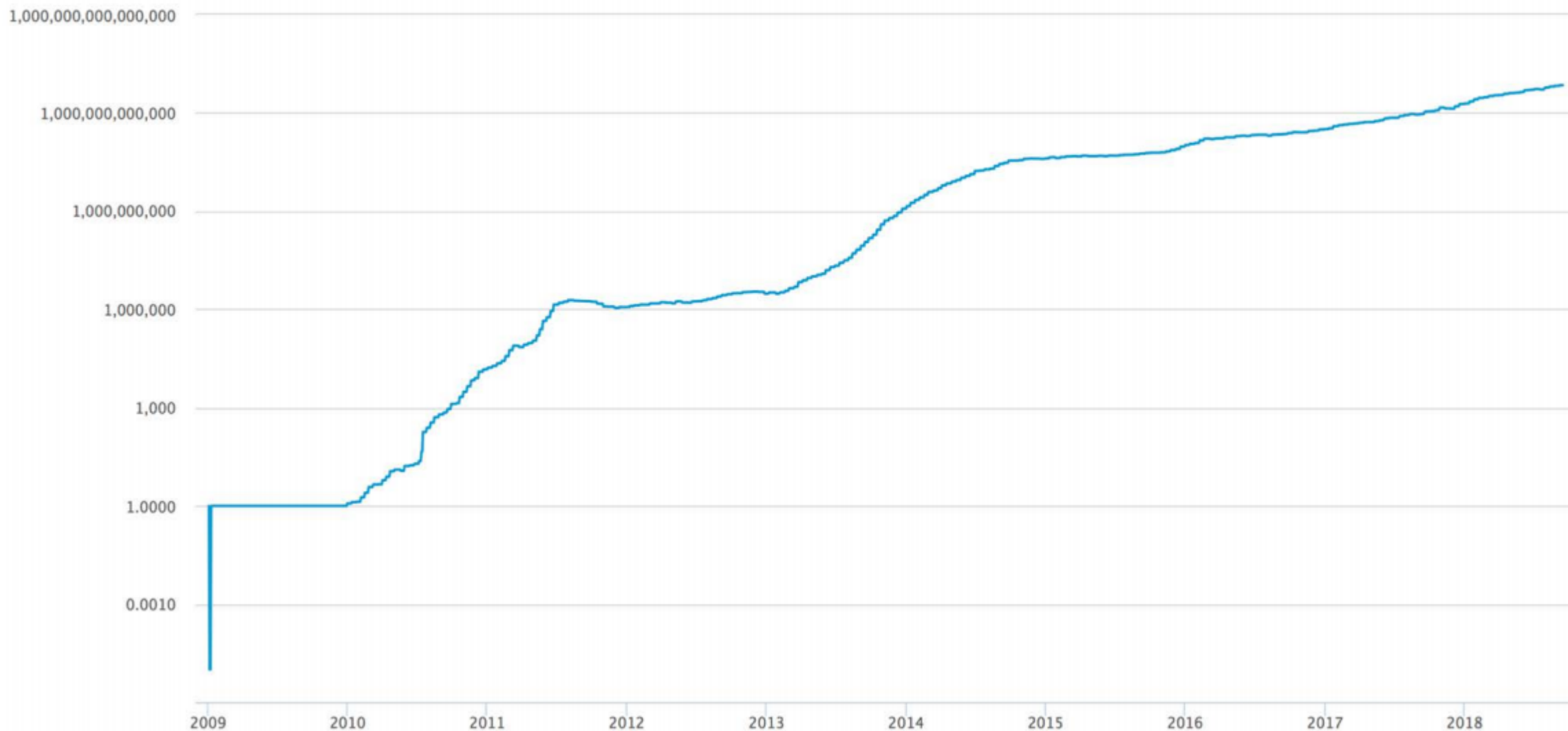
Hash: 5ef7430059da23f3

Mine

Bitcoin Proof of Work Difficulty

- Targets 10 minute average block generation time
- Defined by the # of leading zeros Hash output requires to solve proof of work
- Adjusts every 2016 blocks - about every two weeks
- Currently, ≥ 18 leading zeros (out of 64 hexadecimal characters)
- Block 541974 (9/18/18)- 18 leading zeros
0000000000000000000000001104a863046dfbad1a2941128815669623ff93c2a3945f
- Genesis Block (1/3/09) – 10 leading zeros, though only required 8
000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f

Bitcoin Mining Difficulty



Bitcoin Mining Evolution



Central Processing Units
(CPUs) 2009 – 2010
2 - 20 MH/S



Graphics Processing Units
(GPUs) 2010 – 2013
20 - 300 MH/S

Image is in the [public domain](#).



Application Specific Integrated Circuit
(ASICs) 2013 – 2018
4 – 16 TH/S

Image by [InstagramFOTOGRAFIN](#) on Pixabay.



Modern Mining Factory